



UNIT CODE: PRT582
SOFTWARE ENGINEERING: PROCESS AND TOOLS

Group Assignment: Software Project
Warehouse Inventory System

Submitted By: Group 3 - WIS

Student Name	Student ID	Email
Synthia Islam	# S375728	S375728@students.cdu.edu.au
Hussein Salami	#S371192	S371192@students.cdu.edu.au
Muhammad Asad Nawaz	#S380509	S380509@students.cdu.edu.au
Musrat Jahan	#S380098	S380098@students.cdu.edu.au

Table of Contents

Abstract:	3
Introduction:	3
Objectives:	3
Tasks:	3
Scope:	4
Requirements and User Stories:	5
User Stories	5
Warehouse Manager/Staff	5
Functional Requirements	6
User Stories	7
Non-Functional Requirements:	8
Software/system Requirements:	8
Team Composition and Task Allocation:	9
Non-Functional Testing:	9
➤ Usability Test:	14
Moderated and in-person usability testing	14
Lab usability testing:	15
Guerrilla testing:	15
Moderated and remote usability testing	15
Phone interviews:	15
Card sorting:	15
Unmoderated and remote usability testing	15
Session recordings:	15
Online testing tools and platforms:	15
Unmoderated and in-person usability testing	16
Different software tools that can be used for usability testing:	16
1. First-click Test:	16
Results:	17
Five-second Test:	21
➤ Performance Test:	23
Types of performance testing for software:	23
1. Load testing:	23
2. Stress testing:	23
3. Scalability testing:	23

4. Breakpoint testing:.....	23
5. Endurance testing:.....	23
6. Spike testing:	23
Different tools available to conduct performance testing:.....	23
1. Web LOAD:	23
2. Load Ninja:.....	23
3. Load View:	23
4. Apache JMeter:	24
5. Micro Focus LoadRunner:	24
6. Rational Performance Tester:	24
7. Neo Load:.....	24
8. Loadster:.....	24
9. Page Speed:	24
Creating performance test using JMeter:	24
➤ Security Test:.....	31
Types of Software Security Testing:	31
Software Security Testing tools:	31
Software Security Testing Tools Used in Warehouse Inventory System:.....	32
➤ Automation Test plan:	45
Test strategy:.....	45
Features to be tested:	45
Features not to be tested:	45
Test deliverables and schedule:.....	45
Responsibilities:.....	46
Comparison and justification of automation tool	47
Comparison of automated web testing tool:.....	47
Results and Report:.....	52
Ethical consideration:	73
Further work:	74
Lesson learned:.....	74
Conclusion:.....	74
Appendix	75

PRT582 Software Group Project

Abstract:

The purpose of this unit and assignment two is to develop the skills required for designing and developing structured software solutions for the testing environment of open-source applications. Analysis of both problems associated with code and the techniques for solution testing. Finally, a critical evaluation of all ethical considerations faced in development.

Introduction:

The Warehouse Inventory System is an online platform designed for warehouse admin and staff to manage users, products, sales, sales reports, and product categories efficiently. A comprehensive testing strategy to ensure the delivery of a high-quality system to end users that leverages advanced automation tools and a skilled testing team is crucial. This approach guarantees smooth operations, accurate tracking, and streamlined inventory management, resulting in a reliable and effective solution for managing warehouse operations.

Objectives:

The aim is to create a testing plan that outlines all activities and responsibilities, communication mechanisms, a bug-tracking system, acceptance criteria, and deliverables for each testing phase.

Tasks:

The testing team must verify that all executable features function as specified in this functional testing scope. In addition, it is vital to provide input on how the system might be enhanced to provide a better user experience.

The primary tasks would be defined as below:

Task ID	User story	Acceptance criteria
1	Allow the staff/managers to Login or Signup.	- System allows creating a new user - Registered users can log in to the system
2	Can add, remove, and manage the users.	- A new user can be added to the database - Users can be deleted - Users' information can be edited.
3	Can add new products or edit existing products	- A new product can be added to the database or updating the existing products.
4	Can add new stocks and update the buying and selling information	- New stock can be added, and buying and selling prices can be updated in the database.
5	Can add, remove, and manage the product category.	- A new category can be added to the database. - Categories can be deleted. - Categories can be edited.

6	Can upload new media, view and delete	- New media files can be uploaded. - Media images can be viewed. - Media files can be deleted.
7	Can store, modify and delete the sales records.	- The sales records can be stored to the database. - Sales records can be modified. - Sales records can be deleted.
8	Generate Sales Report	- For a specific data range, daily sales report can be generated. - For a specific month, monthly sales report can be generated.

Table 1: Primary Task

Scope:

This project scope is listed below.:

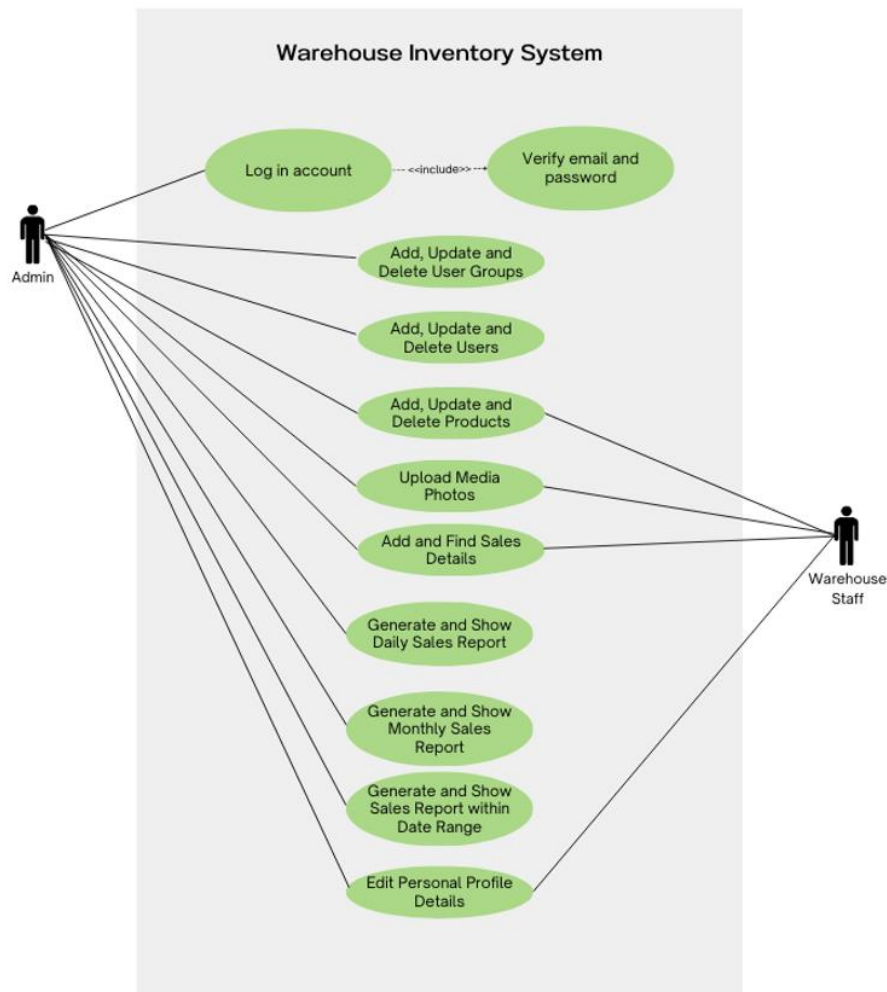
1. Dashboard:
 - It shows an overview of total users, categories, products, and sales.
2. User Management:
 - Users can be added, and existing users can be edited and deleted.
 - User groups also can be added, edited, and deleted.
3. Product Management:
 - New products can be added; existing products can be edited (categories and images can be assigned from categories and media sections) and deleted.
 - New stock can be added, and buying and selling prices can be updated here.
4. Categories:
 - For Categorizing products, new categories can be added, and existing categories can be modified and deleted.
5. Media Management:
 - New media upload, view, and delete product photos.
6. Sales Management:
 - Each sales record will be stored here. Modify sales and delete them here.
7. Sales Record:
 - For a specific date range, daily sales and monthly sales records can be generated.

Requirements and User Stories:

User Stories

The application *Warehouse Inventory System* is designed for use by warehouse staff and managers to manage the inventory process. The functional requirements focus on warehouse management roles. The non-functional and system requirements are demonstrated with a use case diagram below.

Use Case Diagram:



Warehouse Manager/Staff

The warehouse manager registers new users (e.g., staff, managers), adds and maintains product details, manages product categories, records sales, generates sales reports, and oversees stock levels to ensure timely replenishment. The manager also deletes outdated or unwanted items from the inventory system, ensuring smooth and efficient warehouse operations.

Functional Requirements

The functional requirements for this project are given below:

User Group	Requirements
Admin	Add, update, and delete inventory items.
	Manage user roles and permissions.
	Generate detailed reports on inventory levels, sales, and orders.
	Monitor system activity and logs for security and auditing purposes.
	Configure system settings and preferences.
Warehouse Staff	Update inventory status (e.g., received, shipped, damaged)
	Manage and process orders (create, track, fulfill).
	View real-time stock levels and inventory status.
	Perform stock take and adjust inventory counts.
	Receive notifications about low stock levels and pending orders.
	Scan barcodes for quick inventory updates.
Guest	View the product catalogue and check stock availability
	Search for products using various filters (e.g., category, name, SKU).
	Request information about products and availability.
Delivery Staff	Access delivery schedules and routes.
	Update the status of deliveries (e.g., in transit, delivered).
	Confirm receipt of goods at delivery locations.
IT Support	Perform system maintenance and updates.
	Troubleshoot technical issues and resolve them quickly to ensure downtime remains at a minimum.
	Ensure regular data backups are performed, and restore operations work correctly.

Table 2: Functional Requirements

User Stories

STORY ID: (US-001 - US-005)

STORY ID	USER STORY	IMPORTANCE	ESTIMATE	ACCEPTANCE CRITERIA	TYPE
US-001	As a: warehouse staff member	4/5	3/5	Scenario 1: Add inventory item - Logged in as warehouse staff, navigate to "Inventory Management", click on Add new, fill a form with item details. Scenario 2: Edit an item - On "Inventory Management" screen, click edit on an item, modify and save changes. Scenario 3: Remove an item - On "Inventory Management", delete an item after confirmation.	MANAGE DATA ✓
	I want: to add, update, and delete inventory items				
	So that: I can maintain accurate and up-to-date inventory records				
US-002	As a: warehouse staff	4/5	3/5	Scenario 1: Add new order - Log in, navigate to Order Management, create an order with customer and shipping details. Scenario 2: Tracking an order - View an order's current shipping status. Scenario 3: Fulfilling an order - Mark an order as fulfilled, update status, and send notifications.	WORKFLOW ✓
	I want: to manage and process orders				
	So that: I can address customer requests effectively				
US-003	As a: admin	5/5	2/5	Scenario 1: Add a new user - As an admin, add a user with username, role, and permissions. Scenario 2: Edit user roles - Edit roles and permissions for a user. Scenario 3: Remove a user - Delete a user after confirmation.	MANAGE DATA ✓
	I want: to handle user roles and permissions				
	So that: access to different features of the system can be controlled				
US-004	As a: admin	4/5	4/5	Scenario 1: Run inventory report - Generate a report on product details and stocks. Scenario 2: Generate sales report - Produce sales data report. Scenario 3: Generate an order status report - View statuses of all orders.	REPORT/VIEW ✓
	I want: to be able to run inventory-level reports, sales figures, and order statuses				
	So that: better business decisions can be made				
US-005	As a: warehouse staff	3/5	2/5	Scenario 1: Generate Low Stock Alert - Receive a notification to reorder. Scenario 2: Notifications for pending orders - Get alerts on due orders.	ALERT ✓
	I want: to receive alerts on stock balance level and when stock is near threshold point				
	So that: I can make prompt actions on holding inventory and order fulfillment				

Table 3: User Stories

These user stories should help in the realization of the functionalities described in document Form B, along with the development and testing phases of the Warehouse Inventory System project.

Non-Functional Requirements:

Security	Usability	Performance
The system should use strong encryption and secure logins to protect sensitive inventory data and user information.	The interface should be easy to use and navigate, making it simple for administrators and staff to manage the system.	The software should handle more users and inventory items smoothly without slowing down.
User access should be tightly controlled to prevent unauthorized people from accessing inventory data.	The design should be accessible to people with disabilities, following all necessary standards.	The system should be fast, allowing quick searches, updates, and order processing even with large inventories.
All data sent between users and the server should be encrypted to stop anyone from intercepting it.	The interface should be straightforward with clear labels and instructions for all tasks.	The software should perform well even during busy times and when handling large amounts of data.
Regular security checks should be done to find and fix any vulnerabilities.	The system should work consistently across different devices and screen sizes.	The system should be reliable, with minimal downtime and high availability.
The system should regularly back up data to avoid any loss.	Users should have access to training materials and support to help them use all the features effectively.	The software should use resources efficiently to avoid slowdowns and crashes.
The system should meet all relevant data protection laws and standards.	The software should support multiple languages to cater to a diverse range of users.	The system should provide real-time updates and notifications to keep operations running smoothly
Any document sharing should be secure to prevent unauthorized access or leaks of sensitive information.	The system's collaboration features should be easy to understand, promoting teamwork among users.	The software should ensure all tasks, like inventory updates and order processing, are completed quickly without delays.

Table 4: Non-Functional Requirements

Software/system Requirements:

The Warehouse Inventory System is a web application designed to make managing inventory, orders, sales, and deliveries easier for businesses. It's built using PHP, CSS, JavaScript, and MySQL, which are popular technologies for creating reliable and efficient web applications.

➤ **Programming Language:**

- **PHP:** Handles the backend operations and interacts with the database.
- **CSS:** Styles the web pages to ensure they are user-friendly and visually appealing.
- **JavaScript:** Adds dynamic features to the web application, making it more interactive and responsive.
- **MySQL:** DBMS system to manage and store all the inventory data securely and efficiently.

➤ **Software Type:**

- **Web-based Application:** This means users can access it through any web browser on different devices, so there's no need for any special software installations.

Team Composition and Task Allocation:

The report, presentation, and assignment were divided among all group members.

The main components and their assignment group member are as follows:

- Report, documentation, PowerPoint Presentation:
 - Report, Documentation - Synthia, Musrat
 - PowerPoint Presentation – All (Synthia, Musrat, Hussein, Asad)
- Non-Functional Testing:
 - Usability, Security & Performance Testing: All (Synthia, Musrat, Hussein, Asad)
- Functional Testing:
 - Automation Testing Using Robot Framework: - All (Synthia, Musrat, Hussein, Asad)
 - i. Login, Dashboard, Media – assigned to Musrat
 - ii. User Management (User, Group), Categories – assigned to Synthia
 - iii. Sales Management, Sales Report – assigned to Hussein
 - iv. Products and Profile Settings – assigned to Asad

Non-Functional Testing:

Non-functional testing is a type of software testing that verifies how software works and how well it works on non-functional parameters such as usability, performance, security, availability, maintainability, reliability, and accountability of the software. It is just as important as functional testing because it plays a crucial role in customer satisfaction, ensures that our product works as it should, and checks things not covered in functional tests.

➤ Persona and Scenario Testing

 Name: John Type of User: Warehouse Manager	Profile Background Age: 40 Gender: Male Experience: 6 years in warehouse management Goals and Tasks Keep track of goods, monitor their movement, and restock supplies as needed.
--	--

Anxieties and Motivations • Anxious about overstock and shortages of products. • Motivated to keep inventory levels at the perfect level and ensure efficient warehouse operations. Frustrations and Concerns • Frustrated by inaccurate stock data and delays in inventory updates. • Concerned about ineffective systems of supply replenishment. Needs and Expectations • Needs automated alerts for low inventory and pending replenishment requirements. • Expects an accurate stock level real-time inventory tracking system.	Scenario John must estimate future inventory demands, monitor current stock levels, and create reordering supply alerts to avoid stockouts.
---	--

Sarah Type of User: Warehouse Staff	Profile Background Age: 25 Gender: Female Experience: 3 years in Warehouse Operations & Inventory Management. Goals and Tasks Pick, pack, and ship orders in an exact and efficient manner, as well as ensuring correct inventory storage.
Anxieties and Motivations • Anxious about inventory loss and order fulfillment issues. • Motivated to ensuring shipments are shipped on time and inventory is efficiently handled. Frustrations and Concerns • Frustrated by disorganized storage and inefficient picking processes. • Concerned about about incorrect orders and shipment delays. Needs and Expectations • Needs quick access to order details and inventory locations. Expects an organized and effective approach to order selecting and packaging.	Scenario After completing orders, Sarah must update inventory records, make sure that everything is properly maintained in the warehouse, and pick and pack goods for shipping.

 Name: Oliver Type of User: Inventory Clerk	Profile Background Age: 27 Gender: Male Experience: 3 years in Inventory Management. Goals and Tasks • Perform inventory audits, help with stock movement, and correctly record changes to the inventory.
Anxieties and Motivations • Anxious about making errors during stock counts and data entry. • Motivated to maintain accurate inventory records and efficient stock management. Frustrations and Concerns • Frustrated by inaccurate stock data and expensive manual inventory procedures. • Concerned about mismatches between physical inventory and system records. Needs and Expectations • Needs effective instruments for stock adjustment and audits.	Scenario After a stock audit, Oliver needs to fix errors, update inventory records, and shift items according to updated inventory data.

• Expects a simple interface for entering and updating inventory information.	
---	--

 Name: Isa Type of User: Purchasing Manager	Profile Background Age: 30 Gender: Female Experience: 5 years in Procurement and Purchasing Goals and Tasks • To handle purchase orders, monitor supplier delivery, and make sure that supplies are acquired on time.
Anxieties and Motivations • Concerned about supply shortages and supplier delivery delays. • Motivated to establish a positive relationship with suppliers and quick inventory restocking. Frustrations and Concerns • Frustrated by supplier miscommunication and shipment delays.	Scenario Isa must evaluate supplier performance, monitor the status of scheduled shipments, and place a new purchase order for demanded products for accurate and quick shipment.

- Concerned about supply chain interruptions impacting stock levels. Needs and Expectations - Need to know the order status and supplier performance in real time. Expect to have a reliable system setup to track shipment and manage purchasing orders.	
--	--

➤ Usability Test:

Usability testing refers to the user interface that should be simple enough for representative users to understand and get relevant information without special training.

Usability testing done too late in the design phase is a typical mistake. If we wait until the very last minute before our product is delivered, we will not have the time or resources to address any problems and we will have wasted a great deal of time building our product the wrong way.

This testing cycle is completed when:

1. The product is ready.
2. Customers are asked to evaluate the product.
3. If any further changes are found.
4. The product is returned to the development team with feedback for changes required.
5. The product requires useability testing.
6. No changes are required.
7. The product is launched into the market.

Several types of usability testing are available, these include:

Moderated and in-person usability testing

Tests that are moderated and conducted in-person offer the most control. They require many resources, but they are great for gathering comprehensive data.

Lab usability testing:

This kind of usability testing is conducted inside a lab that has been specially designed for it. While a skilled moderator watches and observes, test participants work on computers or mobile devices to accomplish tasks.

Guerrilla testing:

In guerrilla testing, test subjects are randomly selected from a crowded area, typically a coffee shop, shopping centre, or airport. They are frequently requested to do a brief usability test in return for a gift card or another incentive.

Moderated and remote usability testing

A skilled moderator must conduct remote and moderated usability tests using a computer or phone. They allow you to choose from various testers while utilizing a moderator's expertise and depth.

Phone interviews:

In a phone usability test, the user's online activity is remotely recorded. At the same time, the moderator gives participants verbal instructions on how to accomplish tasks on their computers and gathers comments.

Card sorting:

Concepts are written on digital note cards, and then participants organize the cards into groups and categories. After sorting the cards, they discuss their reasoning in a debriefing session led by the moderator.

Unmoderated and remote usability testing

These passive testing techniques, which rely on computer programs, provided insights into how people interact with a website in its "natural environment."

Session recordings:

Software captures user behaviour on a website, including mouse clicks, movements, and scrolling.

Online testing tools and platforms:

Numerous online testing tools enable us to observe user activity on your website from a distance. Some of these platforms let us pay participants to take quick tests, while others monitor actual users' actions as they navigate our website.

5-second test:

This is a simple method for gathering qualitative information on users' initial reactions to your website.

First-click test:

First click testing helps gather information on user expectations and determine where the best places to put menus and buttons.

Unmoderated card sorting:

Users can take the test whenever it is convenient, and this method is quicker and less expensive than moderated sorting.

Unmoderated and in-person usability testing

Unmoderated in-person tests are administered in a controlled physical environment without a test administrator.

Observation:

Researchers can observe participants' facial expressions and body language during observational testing without the moderator's involvement.

Eye-tracking: Using a specialized pupil-tracking gadget attached to a computer, researchers track and analyse users' eye movements during eye-tracking examinations. The computer can produce heatmaps or movement pathway diagrams by examining where users focus their attention when asked to perform a job.

Different software tools that can be used for usability testing:

1. First click and Five Second Test by UsabilityHub. <https://usabilityhub.com>
2. Google Analytics. <https://marketingplatform.google.com/about/analytics/>
3. Google Forms. <https://docs.google.com/forms>
4. Google Mobile Friendly Tests. <https://search.google.com/test/mobile-friendly>
5. Usability Test using Nibbler. <https://nibbler.insites.com/>

For our warehouse inventory system software, we have chosen the “*Unmoderated and remote*” methodology to perform usability testing that relies on computer programs. These passive testing techniques offer perception into how visitors engage with a website in their “natural environment.” And we will create two types of unmoderated and remote usability testing for users that are warehouse admin or staff in our case. The first is the First click test, and the second is the 5-second test.

1. First-click Test:

First-click testing aims to determine whether users can quickly identify the location they must navigate to finish a task. The software tracks the participant's mouse movement as they answer questions like “Where would you click to buy this product?”. We created different first-click test cases for our software in Usability Hub (<https://usabilityhub.com>), a user research and usability testing platform. In the free version of UsabilityHub, we can only create two test cases on a single link.

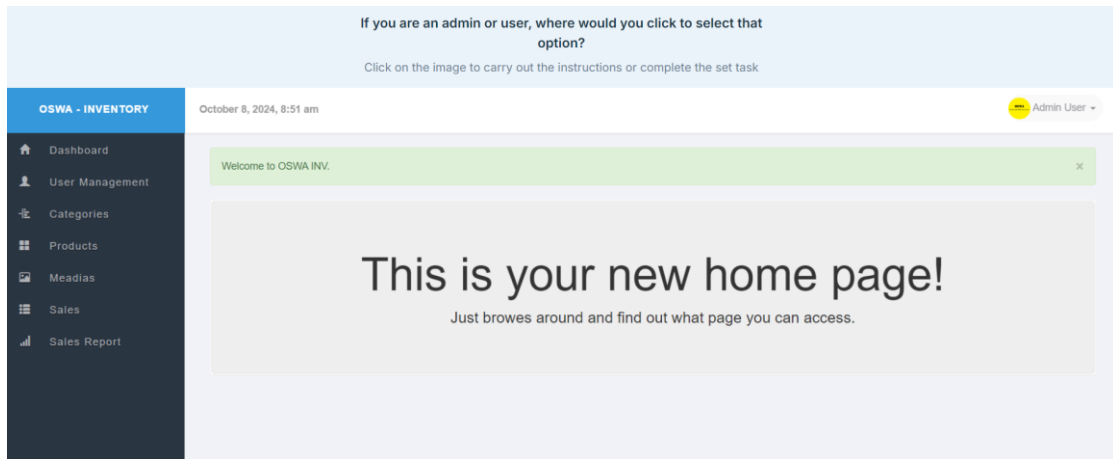
We designed a usability test case for the home page using first-click method and generated a shareable test link to get user feedback.

Test 1:

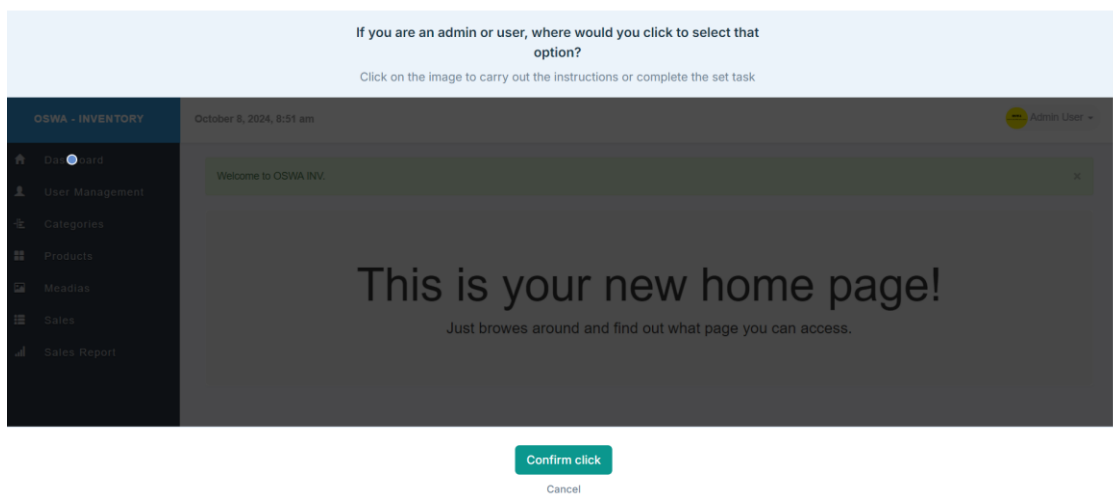
Test link: <https://app.lyssna.com/do/1fb21d3e0180/107d>

Below are the steps to take the tests:

On the first page, we are asking a question on the top “If you are an admin or user, where would you click to select that option??”



Then the users must click on the image according to the instructions.



Then the user was asked questions based on the user's first impression of the software.

A screenshot of the same web application interface as above, but with a survey form overlaid on the right side. The form contains the question "what do you think this website is for?" and four radio button options: "For admin to oversee inventory?", "For workers to locate and manage stock?", "Both for admin and workers?", and "Other". There is a "Pass" button at the bottom right of the form.

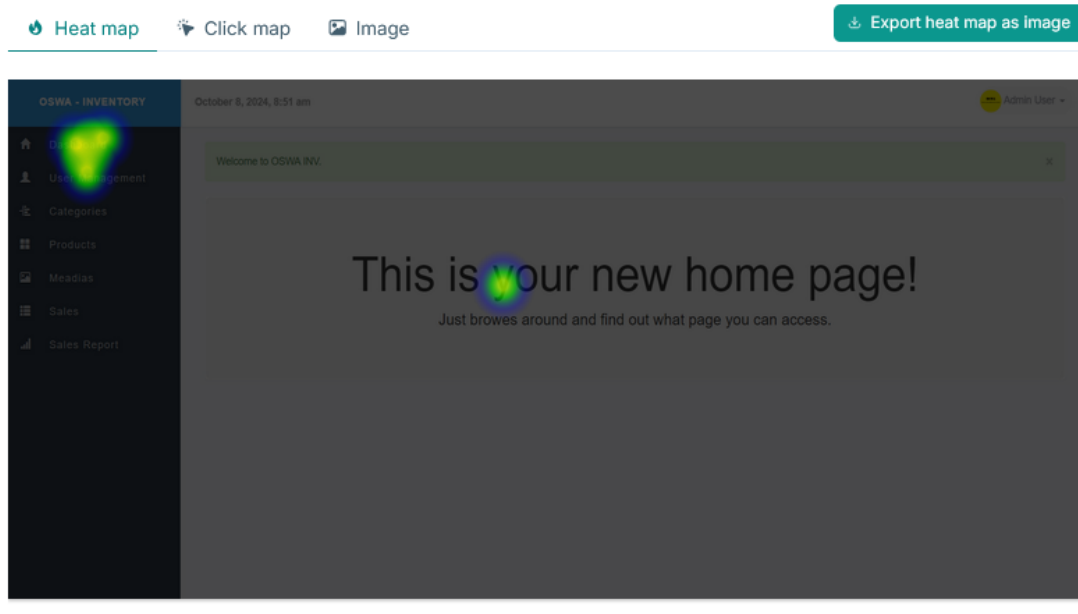
Results:

We requested some friends to participate in the tests, and below are the test results based on the questions we asked.

Test result link: <https://app.lyssna.com/tests/1fb21d3e0180/results>

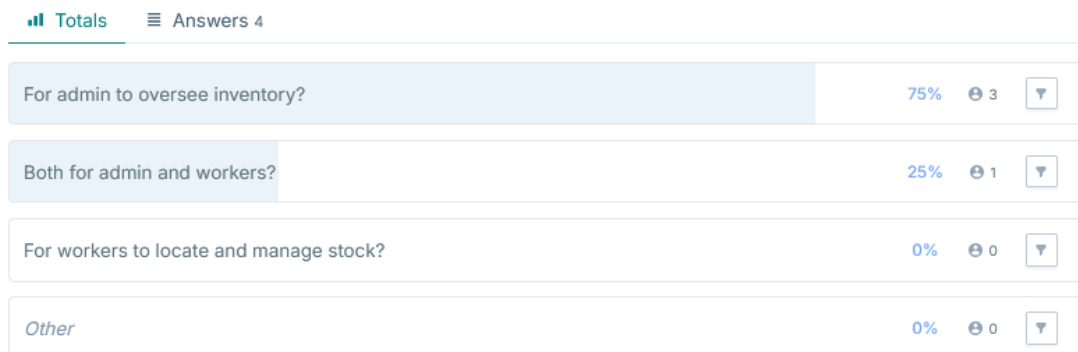
1. First click

If you are an admin or user, where would you click to select that option?



1.1. Single Choice question

what do you think this website is for?

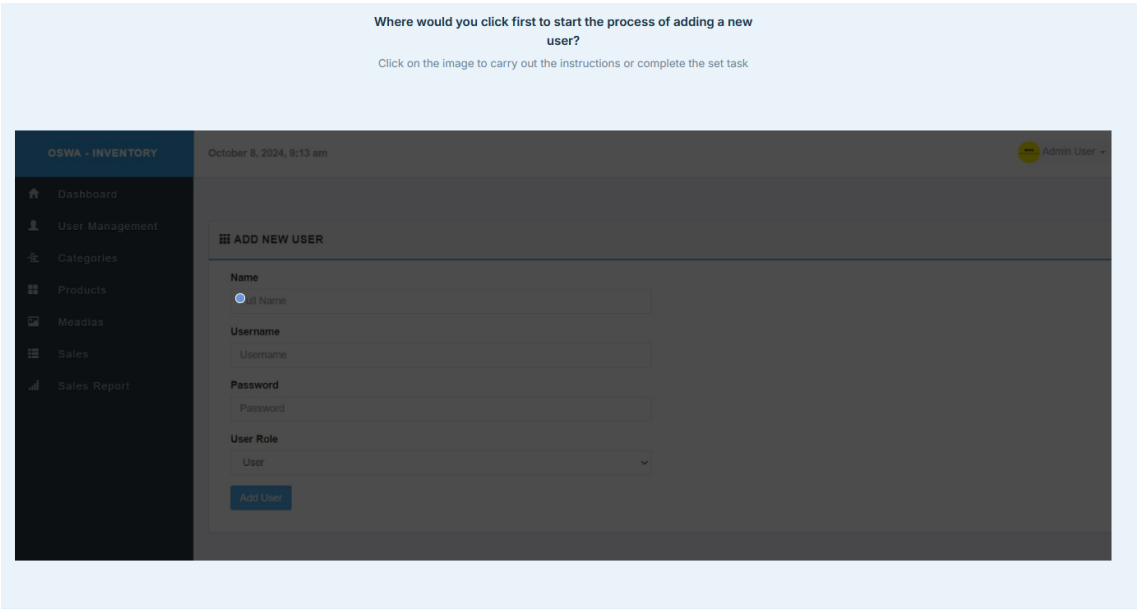
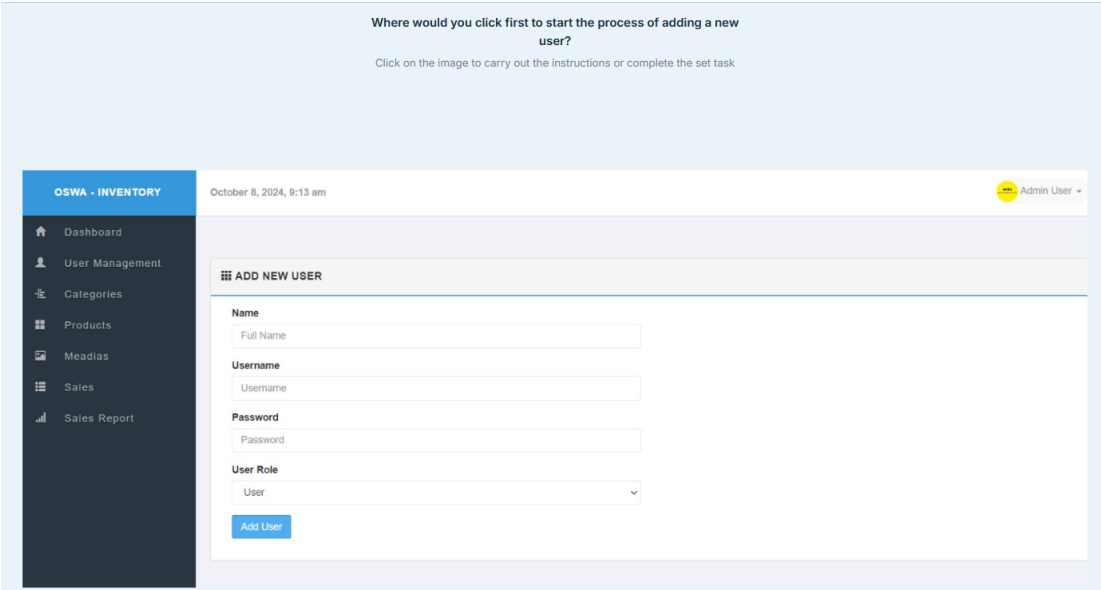


Although our sample size is not enough to decide based on the data shown in the results, most users can quickly identify the location they must navigate to finish their task. Some users answer that this website is for students and staff. However, the system is designed for the warehouse inventory admin and staff. Therefore, we should mention that in our software.

We created 3 more usability tests for the staff/admin using the first-click method and five-second tests in the same link one after another, generating a shareable test link that is used to get feedback from users.

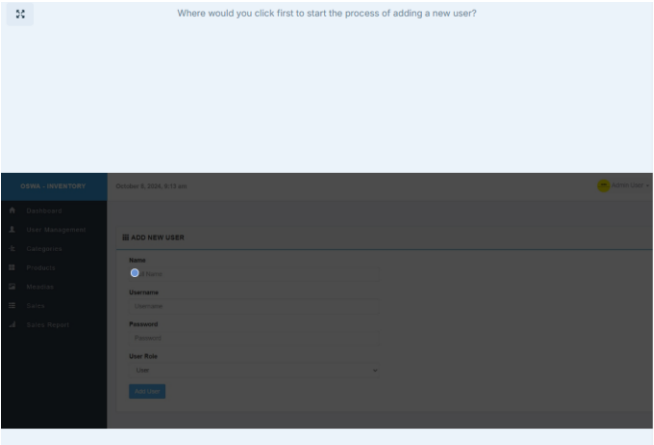
Test link: <https://app.usabilityhub.com/do/6ce0ceec4cb1/8bbe>

Test 2: Below are the steps we followed as same as the above test:



Confirm click

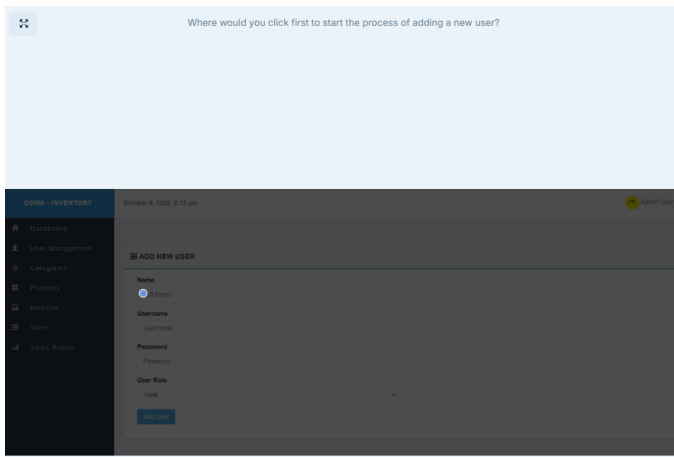
Cancel



How easy was it to find where to start filling in the form?

0	1	2	3	4	5
---	---	---	---	---	---

Pass

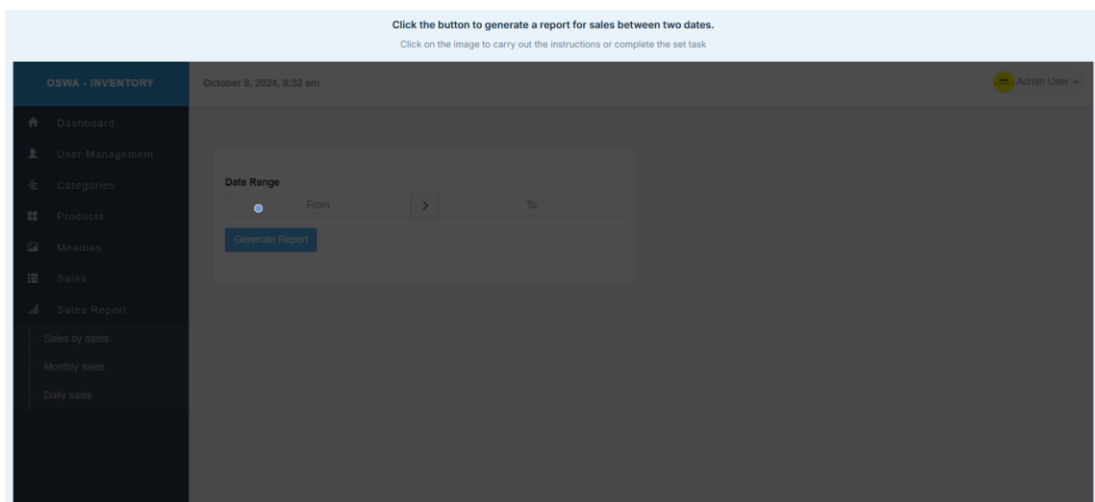
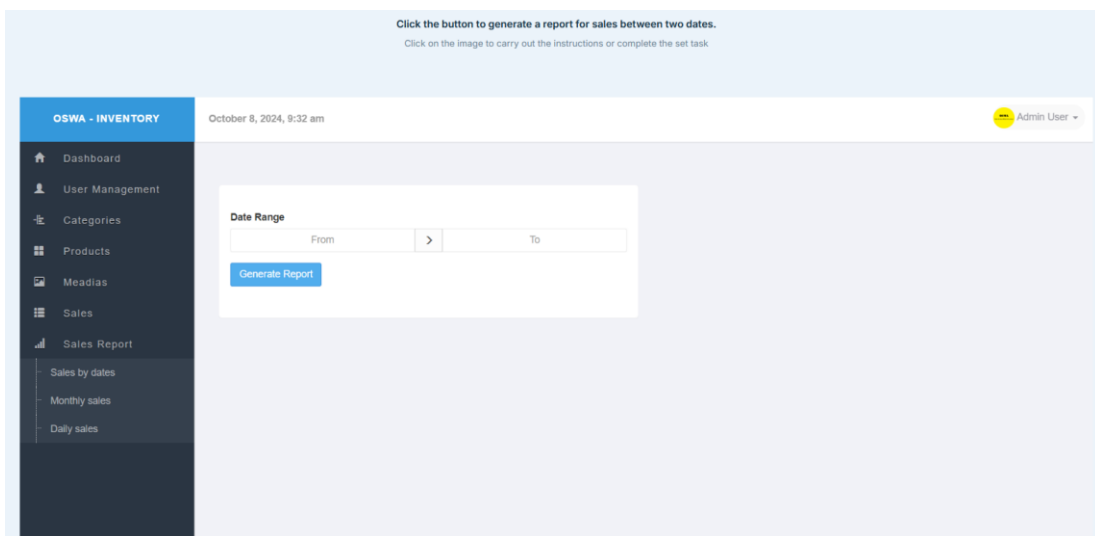


How easy was it to find where to start filling in the form?

0	1	2	3	4	5
---	---	---	---	---	---

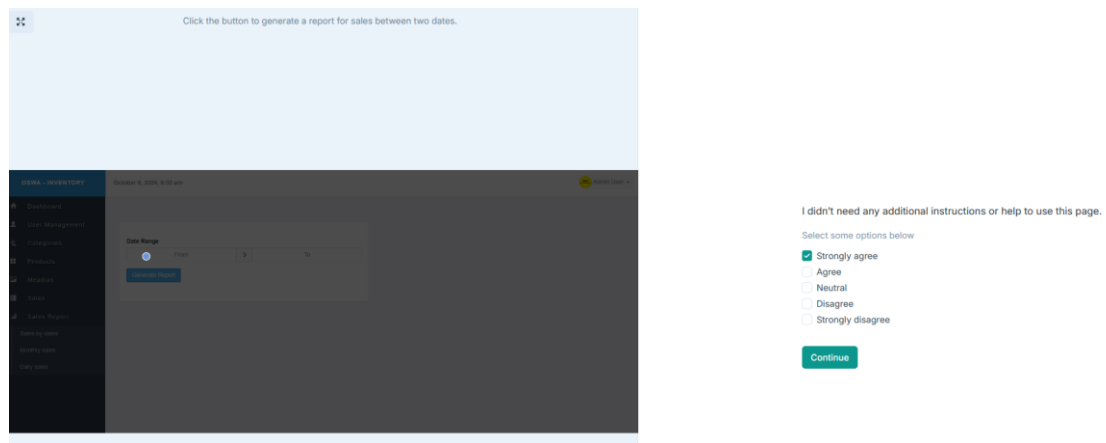
Continue

Test 3: Below are the steps we followed as same as the above test:



Confirm click

Cancel



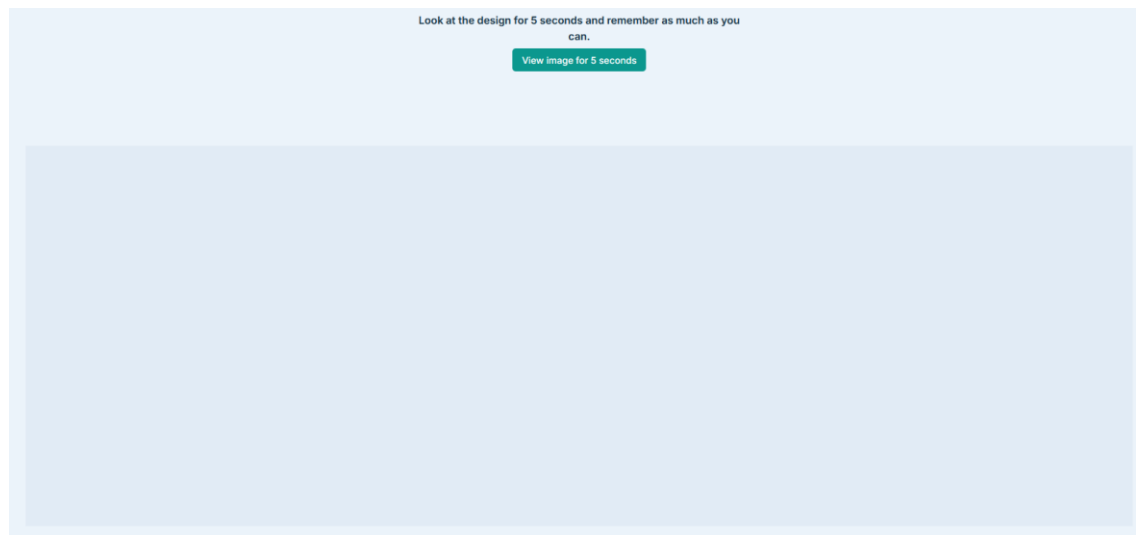
The design for the admin page seems reasonable, as the users were selected in the correct position. Although the sample size is not big enough to conclude the decision, overall, it looks good as it is easy to find the interface components that the user is looking for.

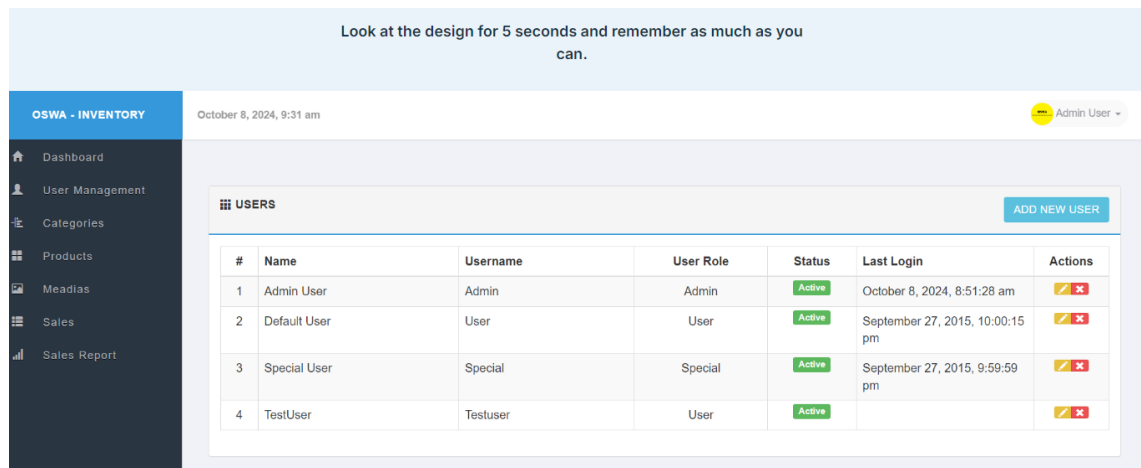
Five-second Test:

Five-second tests are a user research technique that allows us to assess the impression that users form of a design within the first five seconds of viewing it. They are frequently used to determine if web pages effectively convey the message they are meant to.

We created a 5-second test for our software in Usability Hub (<https://usabilityhub.com>) and generated a shareable test link to get user feedback.

Below are the steps we followed for 5-second test:





What color was used to represent the status of each user?

Select some options below

- ☐ Green
- ☐ Red
- ☐ Blue

Pass

What color was used to represent the status of each user?

Select some options below

- ☒ Green
- ☐ Red
- ☐ Blue

Continue

How many users were listed on this page?

3

Continue

Most of the test takers rated the software from 3 to 5 out of five in terms of design which is not bad. Still, there is a scope to recheck the design and fix if there are any issues, or we could take another test where we could ask the test takers what things or components they think can be improved.

➤ Performance Test:

Performance testing is a non-functional software testing technique that assesses how well an application performs in speed, scalability, responsiveness, and stability under expected workloads. Finding bugs or flaws in software is not the main point. Various performance testing methodologies measure outcomes by standards and benchmarks. Developers can identify bottlenecks and remove them with the help of performance testing.

Types of performance testing for software:

1. Load testing:

Load testing assesses how well a system performs as the workload grows. That workload could indicate concurrent users or transactions. The system is evaluated to measure response time and system endurance as the workload increases.

2. Stress testing:

Stress testing is typically done to determine the system's maximum capacity. This test assesses the system's robustness under extreme load and aids application administrators in deciding if the system will function well if the current load is significantly higher than the anticipated maximum.

3. Scalability testing:

The software's ability to handle increasing workloads is evaluated through scalability testing. This can be determined by continually improving the user load or data volume while monitoring system performance. Additionally, even though resources like CPUs and memory are revised, the workload could remain unchanged.

4. Breakpoint testing:

Stress testing and breakpoint testing are related. Over time, a gradually increasing load is supplied while the system is watched for certain failure circumstances.

5. Endurance testing:

Ensuring the program can endure the anticipated load over an extended period.

6. Spike testing:

It evaluates how the system responds to unexpectedly high user load spikes.

Different tools available to conduct performance testing:

1. Web LOAD:

A web application load and enterprise standard performance testing tool. Web LOAD is the tool of choice for businesses with a high user load and complicated testing requirements. Creating load from the cloud and on-premises servers enables us to do load and stress testing on any internet application.

2. Load Ninja:

We can quickly construct scripless advanced load tests using Load Ninja by Smart Bear, which also decreases testing time in half, replaces load emulators with actual browsers, and obtains actionable browser-based metrics.

3. Load View:

The load and stress testing process is hassle-free with the help of Load View, a fully managed, on-demand load testing tool.

4. Apache JMeter:

It is a Java platform program that is an open-source load-testing tool. It can be incorporated with the test plan and is thought of as a performance testing tool. You can make a functional test plan in addition to the load test plan. This tool can be installed into a server or network to monitor performance and examine how it operates in various scenarios. It was initially designed to test web applications, but over time, its applicability expanded.

5. Micro Focus LoadRunner:

It is possible to utilize this Micro Focus product as a Performance Testing tool. Its Micro Focus software department offers this as a Micro Focus product. Additionally, it is helpful in figuring out how the system will operate and perform under actual load.

6. Rational Performance Tester:

Rational performance tester is an automated performance testing tool that can be used for a web application or a server-based application where input and output are involved.

7. Neo Load:

Neo Load is the most automated performance testing platform for enterprise organizations that regularly test applications and APIs.

8. Loadster:

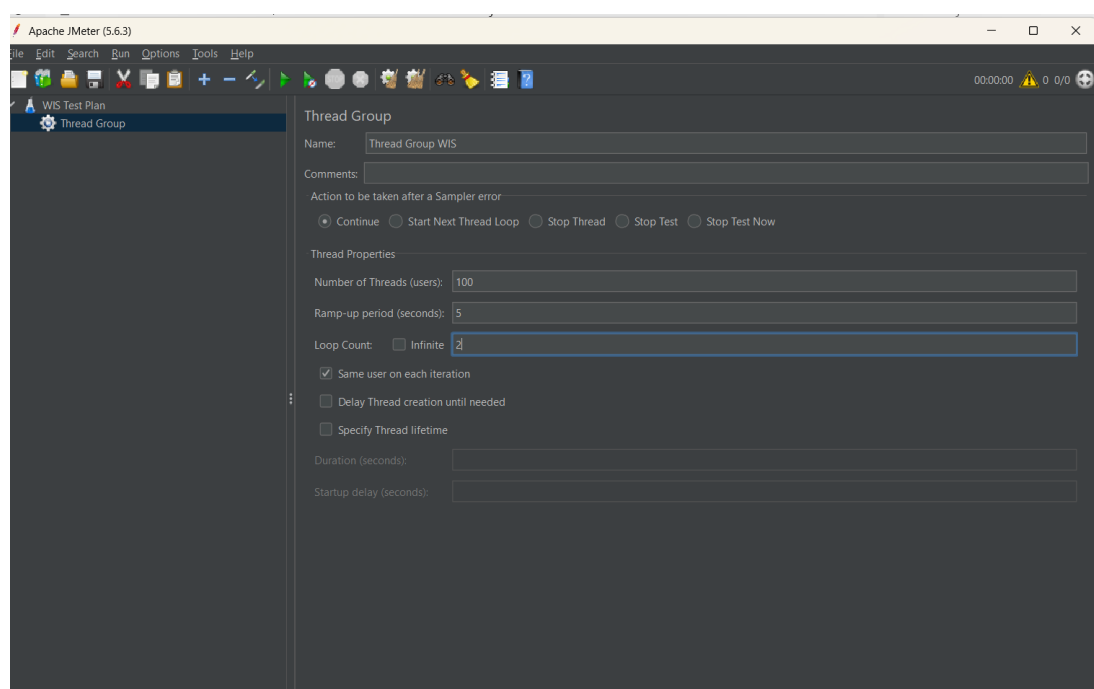
Loadster is a desktop-based advanced HTTP (Hypertext Transfer Protocol) load testing tool. The web browser can record scripts that are easy to use and record.

9. Page Speed:

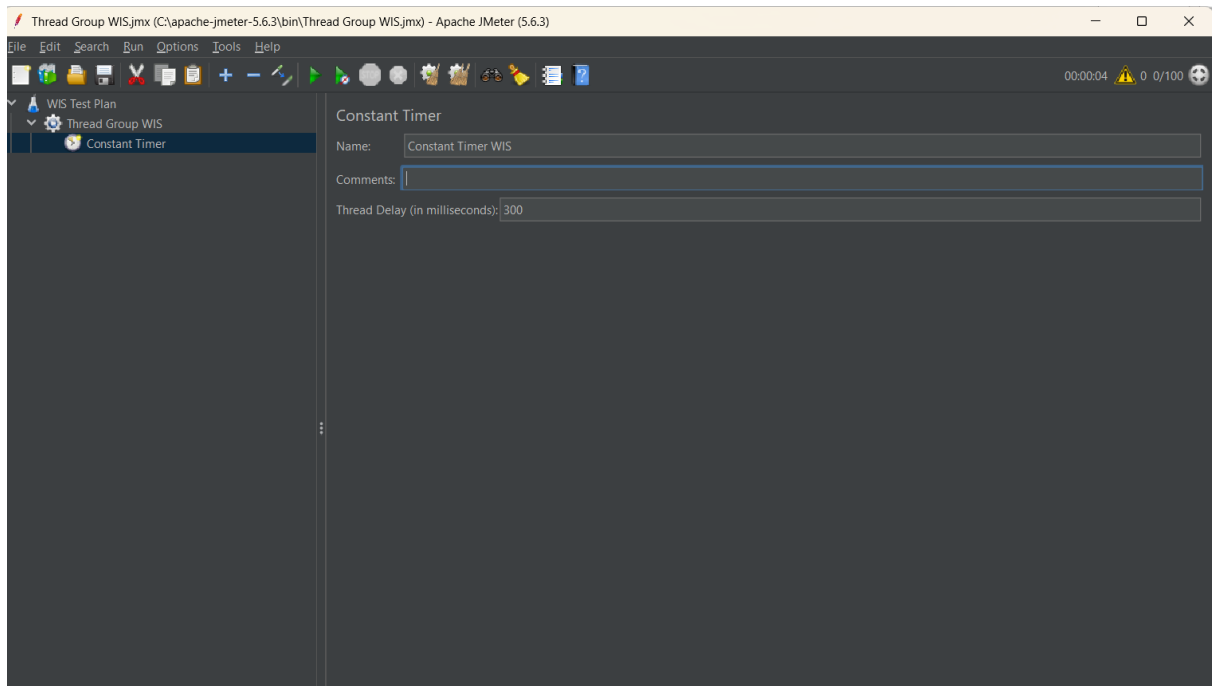
Page Speed Insights (PSI) reports a page's performance on desktop and mobile devices and offers recommendations for improving that page.

Creating performance test using JMeter:

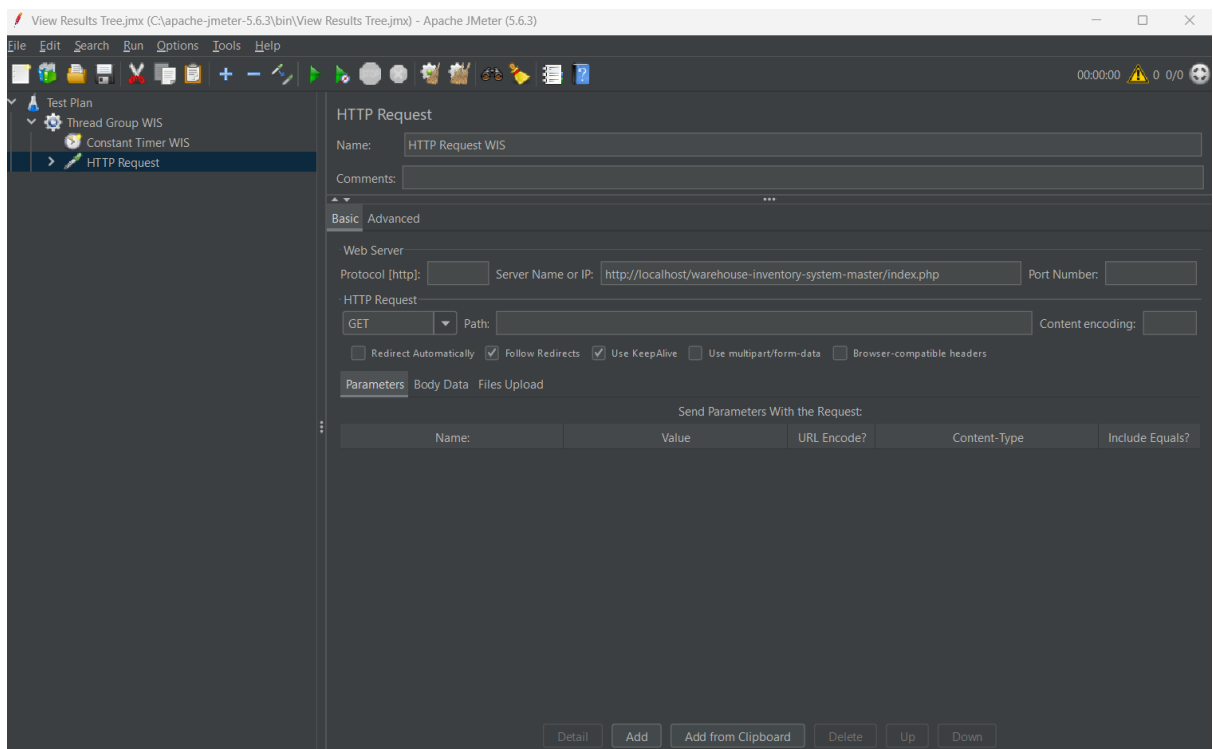
1. Creating a test plan and adding several threads:



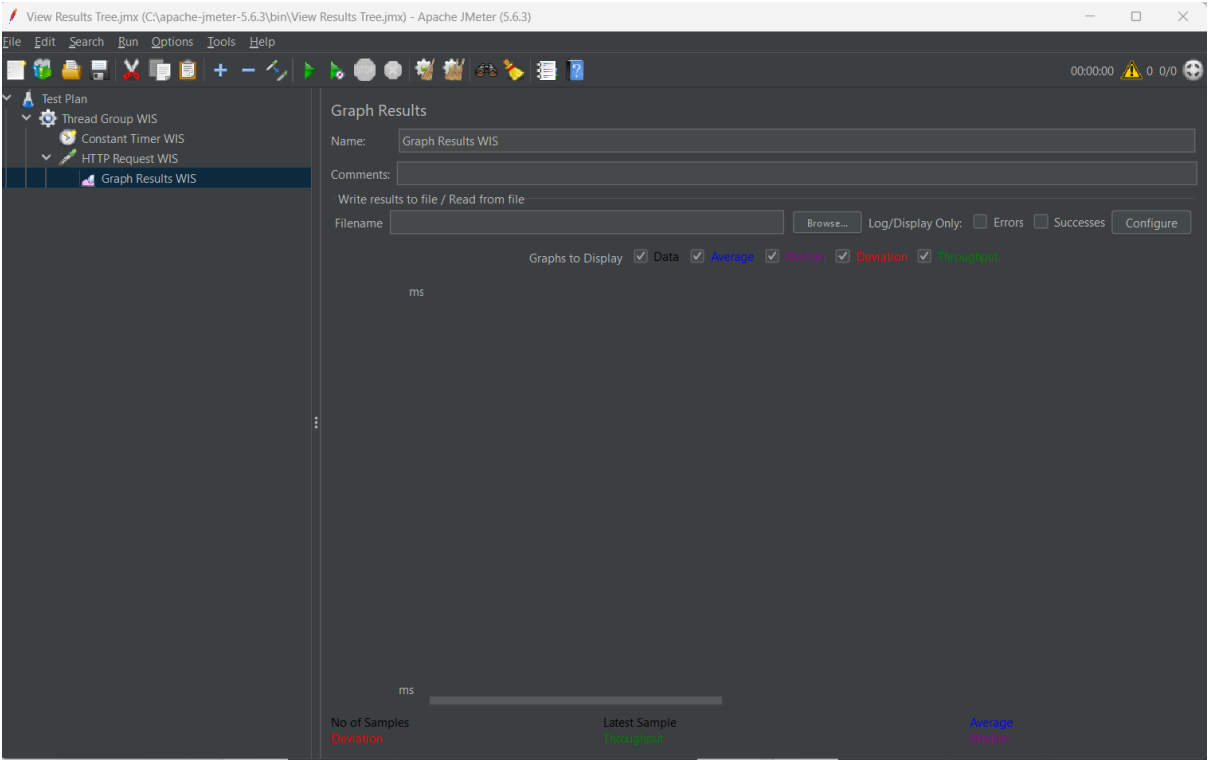
2. Adding Timer:



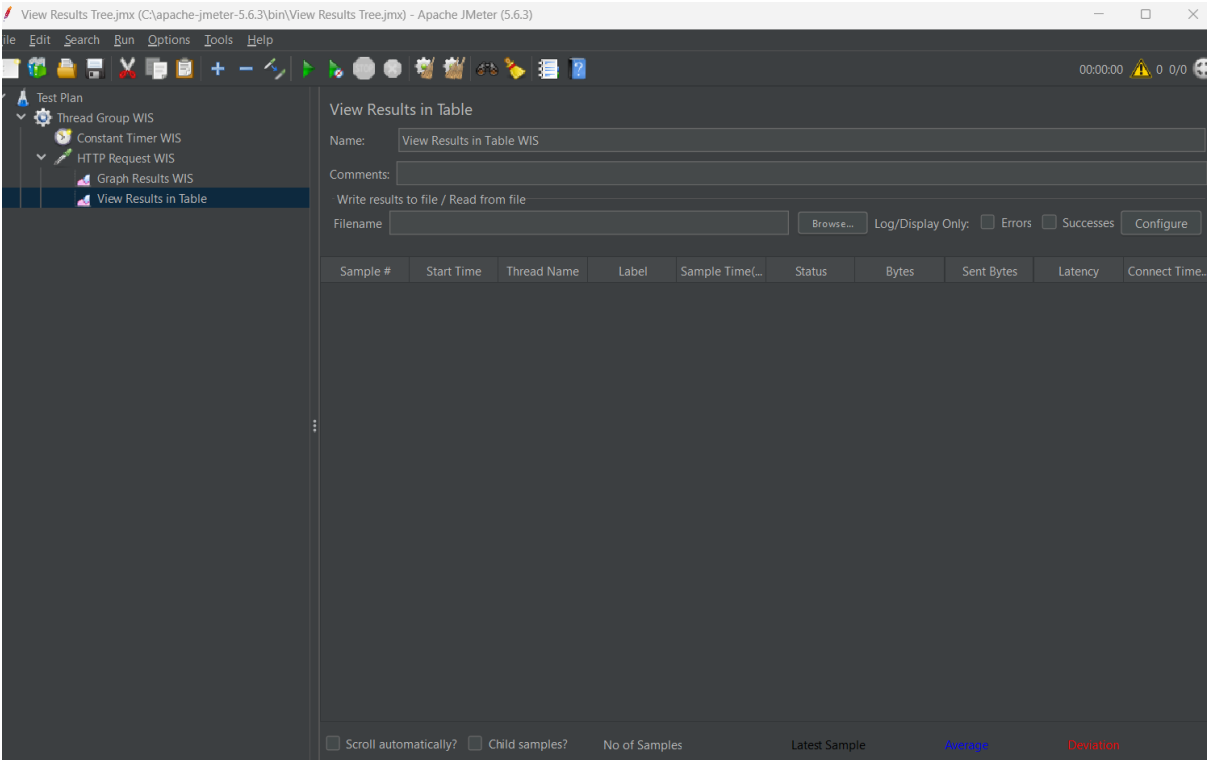
3. Adding HTTP request for home page(<http://localhost:8000/>):



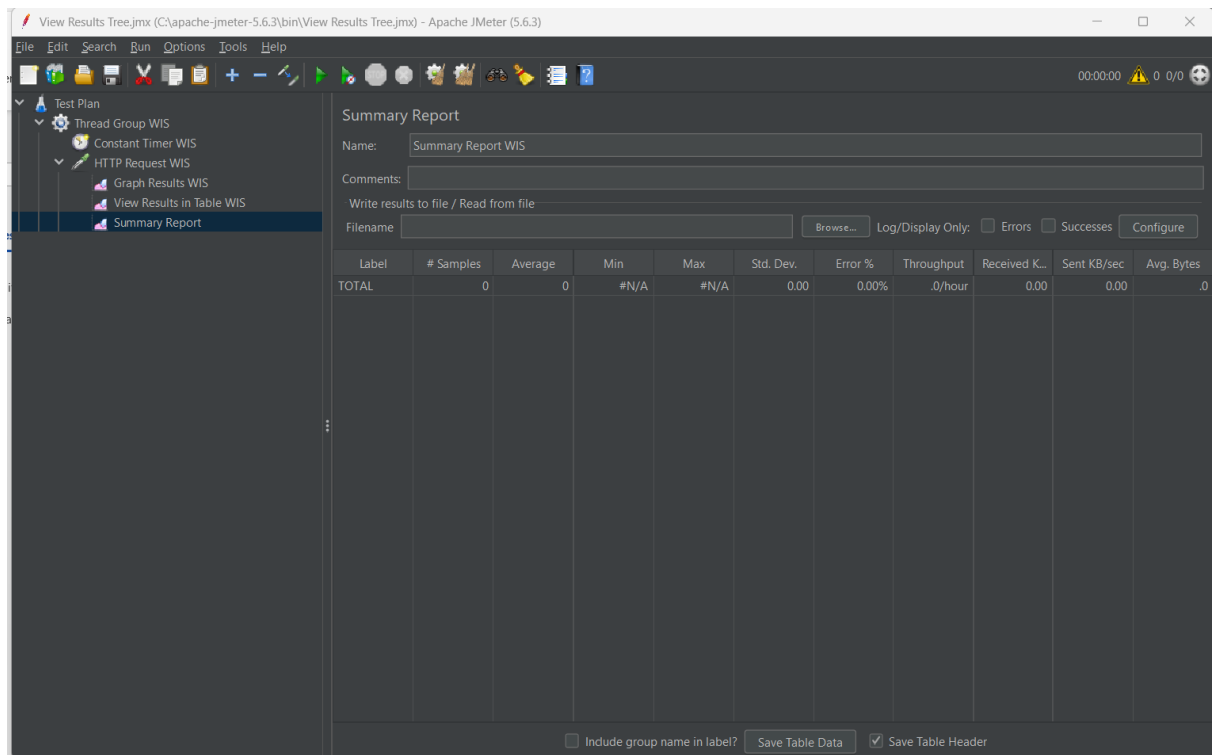
4. Adding graph listener to see output or result as a graph:



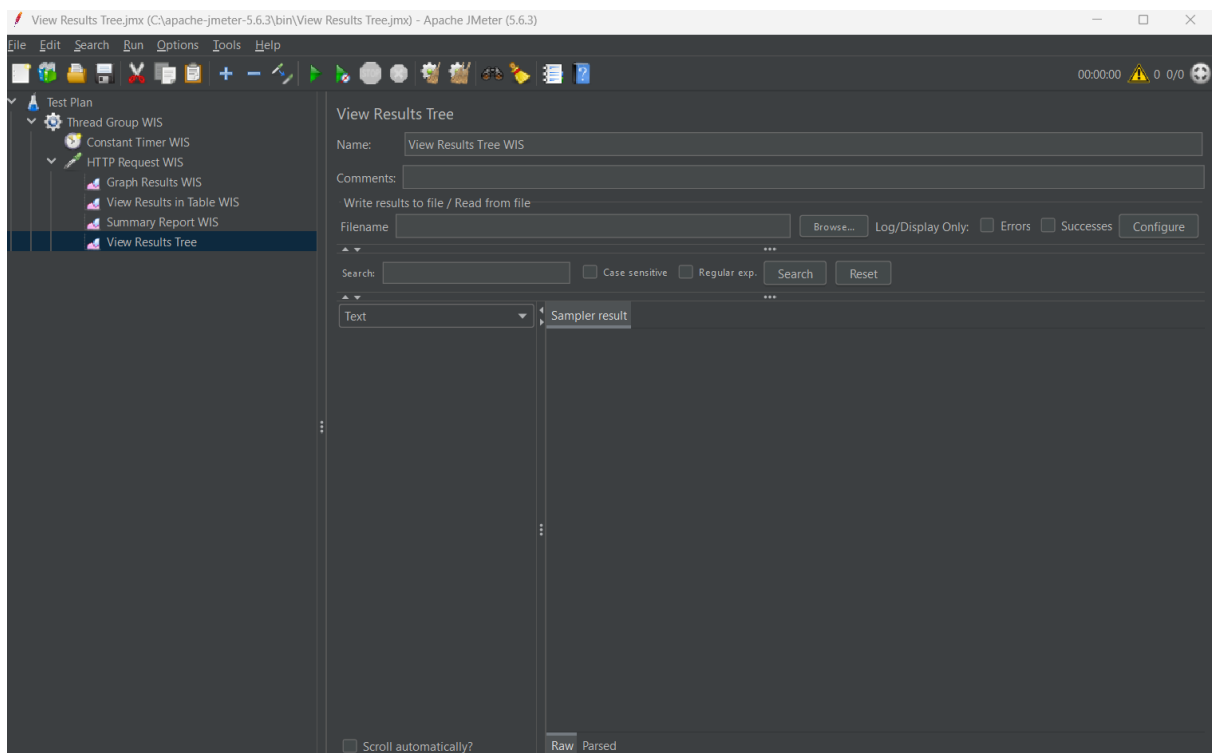
5. Adding another listener to see results in the table view:



6. Adding a listener for summary reports:

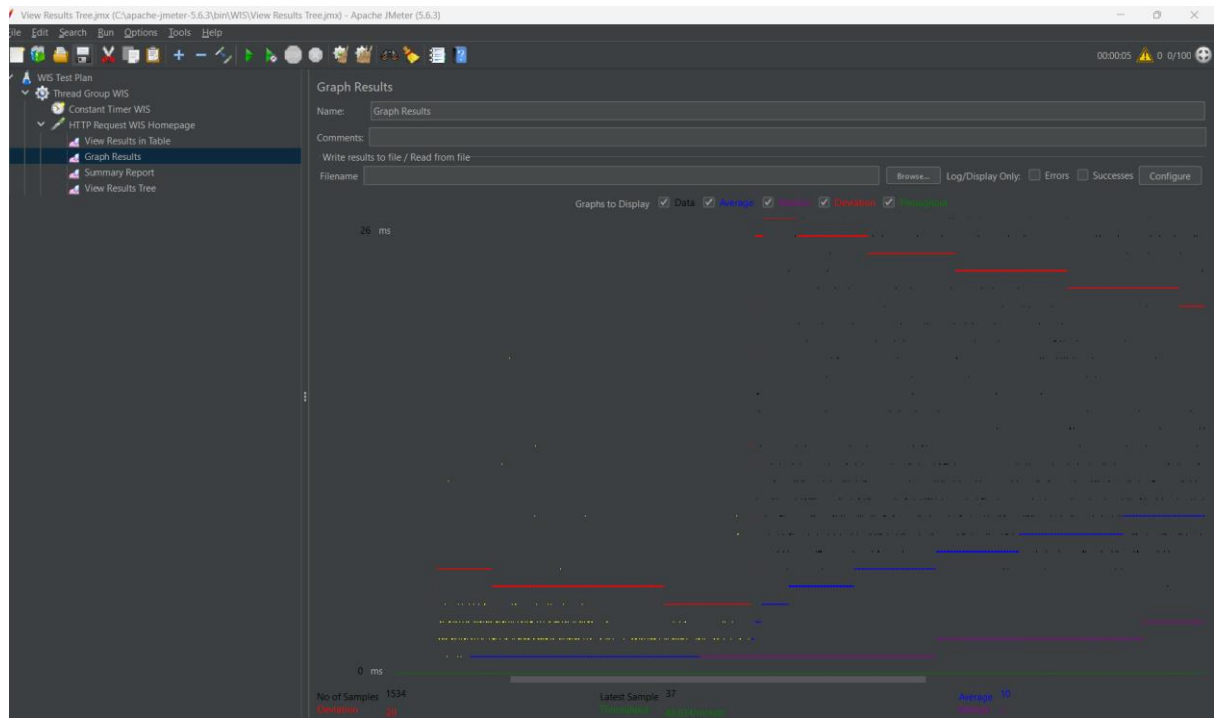


7. Adding a listener to view the results in a tree:



- **Running the Test:**

- **Graph Result:**

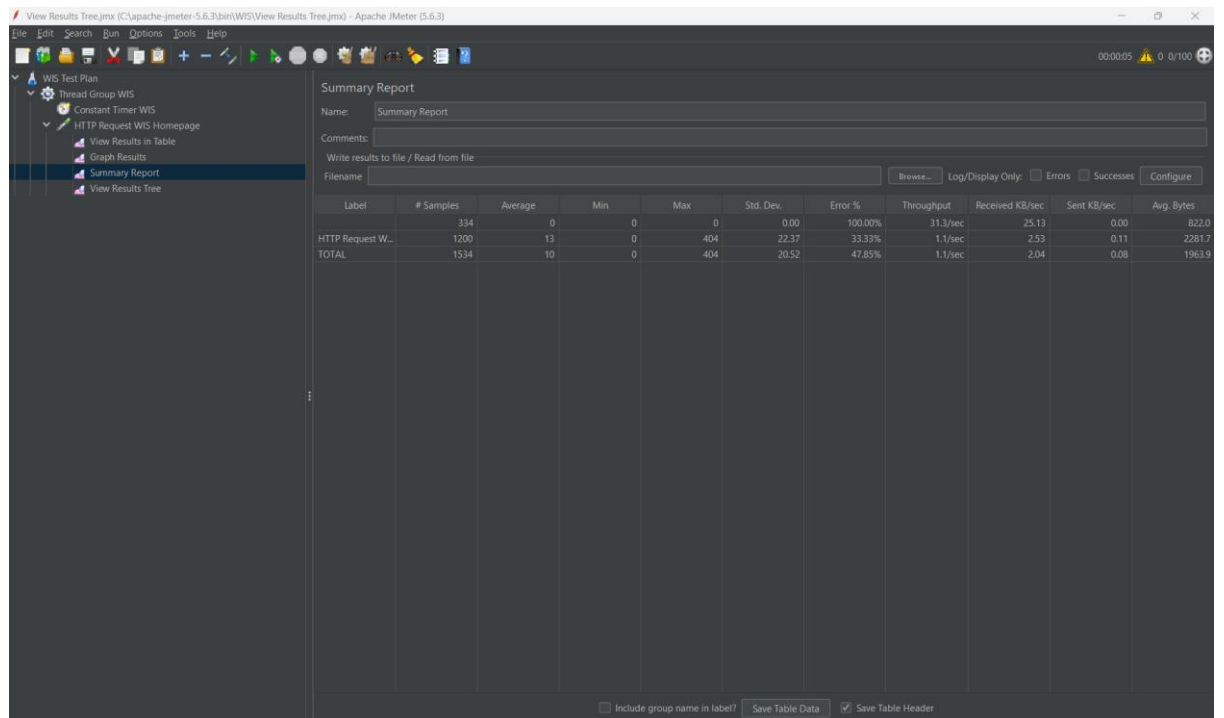


- **Results in the Table:**

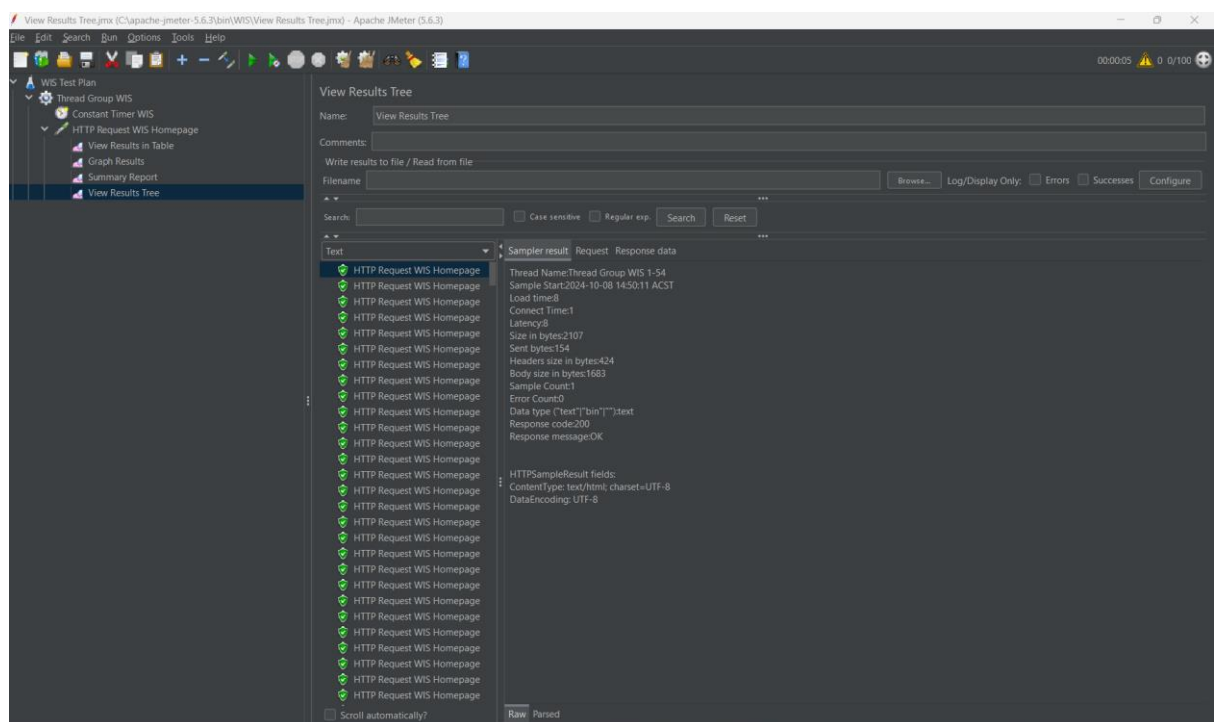
The screenshot shows the Apache JMeter interface with the 'View Results Tree' window open. The 'View Results in Table' view is selected, displaying a table of test results. The table has columns for Sample #, Start Time, Thread Name, Label, Sample Time(ms), Status, Bytes, Sent Bytes, Latency, and Connect Time(ms). The status bar at the bottom indicates that there are 1734 samples, the latest sample is 37, and the average response time is 10ms.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
733	14:42:32.151	Thread Group WIS...	HTTP Request WIS...	2	Success	2631	0	0	2
734	14:42:32.199	Thread Group WIS...	HTTP Request WIS...	1	Success	2631	0	0	1
735	14:48:32.453	Thread Group WIS...	HTTP Request WIS...	186	Success	2107	154	181	1
736	14:48:32.504	Thread Group WIS...	HTTP Request WIS...	135	Success	2107	154	131	1
737	14:48:32.404	Thread Group WIS...	HTTP Request WIS...	235	Success	2107	154	227	2
738	14:48:32.553	Thread Group WIS...	HTTP Request WIS...	86	Success	2107	154	78	1
739	14:48:32.600	Thread Group WIS...	HTTP Request WIS...	39	Success	2107	154	31	1
740	14:48:32.290	Thread Group WIS...	HTTP Request WIS...	349	Success	2107	154	341	1
741	14:48:32.337	Thread Group WIS...	HTTP Request WIS...	302	Success	2107	154	297	2
742	14:48:32.235	Thread Group WIS...	HTTP Request WIS...	404	Success	2107	154	396	4
743	14:48:32.641	Thread Group WIS...	HTTP Request WIS...	10	Success	2107	154	10	1
744	14:48:32.689	Thread Group WIS...	HTTP Request WIS...	16	Success	2107	154	16	2
745	14:48:32.752	Thread Group WIS...	HTTP Request WIS...	16	Success	2107	154	15	1
746	14:48:32.800	Thread Group WIS...	HTTP Request WIS...	35	Success	2107	154	35	2
747	14:48:32.848	Thread Group WIS...	HTTP Request WIS...	15	Success	2107	154	14	1
748	14:48:32.911	Thread Group WIS...	HTTP Request WIS...	13	Success	2107	154	13	2
749	14:48:32.943	Thread Group WIS...	HTTP Request WIS...	40	Success	2107	154	40	1
750	14:48:32.948	Thread Group WIS...	HTTP Request WIS...	45	Success	2107	154	44	1
751	14:48:32.948	Thread Group WIS...	HTTP Request WIS...	46	Success	2107	154	46	0
752	14:48:32.951	Thread Group WIS...	HTTP Request WIS...	44	Success	2107	154	44	1
753	14:48:32.943	Thread Group WIS...	HTTP Request WIS...	53	Success	2107	154	53	1
754	14:48:32.946	Thread Group WIS...	HTTP Request WIS...	54	Success	2107	154	54	1
755	14:48:32.991	Thread Group WIS...	HTTP Request WIS...	9	Success	2107	154	9	1
756	14:48:32.952	Thread Group WIS...	HTTP Request WIS...	54	Success	2107	154	54	3
757	14:48:32.945	Thread Group WIS...	HTTP Request WIS...	62	Success	2107	154	62	1
758	14:48:33.005	Thread Group WIS...	HTTP Request WIS...	8	Success	2107	154	8	2
759	14:48:32.945	Thread Group WIS...	HTTP Request WIS...	68	Success	2107	154	68	1
760	14:48:32.948	Thread Group WIS...	HTTP Request WIS...	93	Success	2107	154	93	0
761	14:48:33.052	Thread Group WIS...	HTTP Request WIS...	35	Success	2107	154	35	1

- Summary Report:



- View Results Tree:



- CSV report generated from JMeter:

A1	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	timeStamp	elapsed	label	responseCode	responseMessage	threadName	dataType	success	failureMessage	bytes	sentBytes	grpThreads	allThreads	URL	Latency	IdleTime	Connect			
2	1.73E+12	24	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	7	7	http://loca	24	0	2			
3	1.73E+12	12	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	8	8	http://loca	12	0	2			
4	1.73E+12	14	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	9	9	http://loca	14	0	1			
5	1.73E+12	22	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	10	10	http://loca	22	0	1			
6	1.73E+12	12	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	11	11	http://loca	12	0	2			
7	1.73E+12	17	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	12	12	http://loca	17	0	1			
8	1.73E+12	13	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	13	0	1			
9	1.73E+12	24	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	14	14	http://loca	24	0	0			
10	1.73E+12	29	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	29	0	0			
11	1.73E+12	8	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	8	0	0			
12	1.73E+12	9	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	9	0	1			
13	1.73E+12	14	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	14	14	http://loca	14	0	1			
14	1.73E+12	17	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	17	0	2			
15	1.73E+12	11	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	11	0	2			
16	1.73E+12	10	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	10	0	1			
17	1.73E+12	10	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	10	0	1			
18	1.73E+12	10	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	10	0	2			
19	1.73E+12	10	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	10	0	2			
20	1.73E+12	11	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	11	0	1			
21	1.73E+12	8	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	14	14	http://loca	8	0	0			
22	1.73E+12	35	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	14	14	http://loca	35	0	1			
23	1.73E+12	9	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	9	0	1			
24	1.73E+12	15	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	15	0	1			
25	1.73E+12	12	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	12	0	0			
26	1.73E+12	11	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	11	0	1			
27	1.73E+12	9	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	9	0	1			
28	1.73E+12	12	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	12	0	1			
29	1.73E+12	9	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	9	0	1			
30	1.73E+12	12	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	13	13	http://loca	12	0	1			
31	1.73E+12	37	HTTP Requ	200 OK		Thread Grc text		TRUE		2107	154	14	14	http://loca	37	0	1			

- Test for staff login page using cookies and cache:

- Failed Result:

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
700	14-02-31.195	Thread Group WIS...	HTTP Request WIS...	3	Failed	2631	0	0	3
701	14-02-31.244	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
702	14-02-31.255	Thread Group WIS...	HTTP Request WIS...	2	Failed	2631	0	0	1
703	14-02-31.295	Thread Group WIS...	HTTP Request WIS...	3	Failed	2631	0	0	3
704	14-02-31.298	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
705	14-02-31.347	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
706	14-02-31.353	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
707	14-02-31.396	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
708	14-02-31.400	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
709	14-02-31.447	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
710	14-02-31.447	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
711	14-02-31.500	Thread Group WIS...	HTTP Request WIS...	2	Failed	2631	0	0	1
712	14-02-31.501	Thread Group WIS...	HTTP Request WIS...	3	Failed	2631	0	0	3
713	14-02-31.546	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
714	14-02-31.546	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
715	14-02-31.609	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
716	14-02-31.609	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
717	14-02-31.659	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
718	14-02-31.657	Thread Group WIS...	HTTP Request WIS...	9	Failed	2631	0	0	9
719	14-02-31.704	Thread Group WIS...	HTTP Request WIS...	8	Failed	2631	0	0	7
720	14-02-31.719	Thread Group WIS...	HTTP Request WIS...	8	Failed	2631	0	0	6
721	14-02-31.732	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
722	14-02-31.751	Thread Group WIS...	HTTP Request WIS...	2	Failed	2631	0	0	2
723	14-02-31.799	Thread Group WIS...	HTTP Request WIS...	2	Failed	2631	0	0	2
724	14-02-31.815	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
725	14-02-31.847	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
726	14-02-31.852	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
727	14-02-31.894	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1
728	14-02-31.911	Thread Group WIS...	HTTP Request WIS...	1	Failed	2631	0	0	1

➤ Security Test:

To prevent the software from endangering the system and user data, software security testing verifies that the software is free of any potential flaws, hazards, or threats. Security testing proves that data and systems are secure and trustworthy and do not allow unauthorized inputs.

The fundamental objective of security testing is to determine the system's risks and assess any potential vulnerabilities so that threats can be encountered, and the system can continue to operate without being compromised. Additionally, it aids in identifying any security risks that might be present in the design and enables programmers to solve issues.

Types of Software Security Testing:

1. Static Code Analysis:

Most developers begin their security testing using this method because it is the oldest. This test can be done manually, and engineers can read the code to look for security flaws.

2. Compliance Testing:

Software must comply with a client's established policies, and we ensure this by conducting compliance tests. We evaluate a piece of software in these tests by contrasting it with the actual configurations.

3. Penetration Testing:

This software testing uses simulation attacks to find the weak points in newly built software. A developer fixes the bugs in the codes once they are discovered.

4. Load Testing:

This test evaluates how well a piece of software operates under a heavy load. The reason for this test is Distributed-Denial-of-Service (DDoS) attack, which aims to interfere with an application's availability by flooding the host infrastructure with traffic or other requests.

5. SQL Injection Testing:

A SQL Injection test can be run for quote marks, apostrophes, brackets, and commas. Simple mistakes like these invite spammer attacks. SQL injection attacks are essential because they provide attackers access to the server database and allow them to obtain crucial data.

Software Security Testing tools:

Zed Attack Proxy (ZAP): A multi-platform, open-source security testing tool for web applications developed by the Open Web Application Security Project (OWASP).

Key features of ZAP:

1. Automatic scanning
2. Easy to use
3. Multi-platform
4. Rest-based API
5. Support for authentication

Wfuzz: This tool's interface has no GUI and is developed using Python. The only drawback of this utility is that it can only be used via the command line.

Key features of Wfuzz:

1. Authentication support

2. Cookies fuzzing
3. Multi-threading
4. Multiple injection points
5. Support for proxy and SOCK

W3af: It is yet another acclaimed Python-built tool. This tool is ideal for web apps. W3af can find more than two hundred distinct kinds of security flaws.

Key features of W3af:

1. Blind SQL injection.
2. Buffer overflow.
3. Cross-site scripting.
4. Insecure DAV configurations.

Slap: It automates the discovery of a database vulnerability on a website and is completely free to use. SQL Map has a mighty testing engine to find different security threads.

Key features of SQL Map:

1. Automates the process of finding SQL injection vulnerabilities.
2. It can also be used for security testing on a website.
3. Robust detection engine.
4. Supports a range of databases, including MySQL, Oracle, and PostgreSQL.

Python Bandit: A source code security analysis tool called Bandit searches in Python programs for known vulnerabilities. It processes files to create an abstract syntax tree (AST), which is then used to run plugins against. It then produces a report on the results.

Software Security Testing Tools Used in Warehouse Inventory System:

From the above options, we used Zed Attack Proxy (ZAP) to conduct security tests for the Warehouse Inventory System. ZAP is an open-source web application security testing tool developed by OWASP. ZAP is flexible and extensible and provides a range of features suited for web application testing.

- **Creating security test using ZAP:**

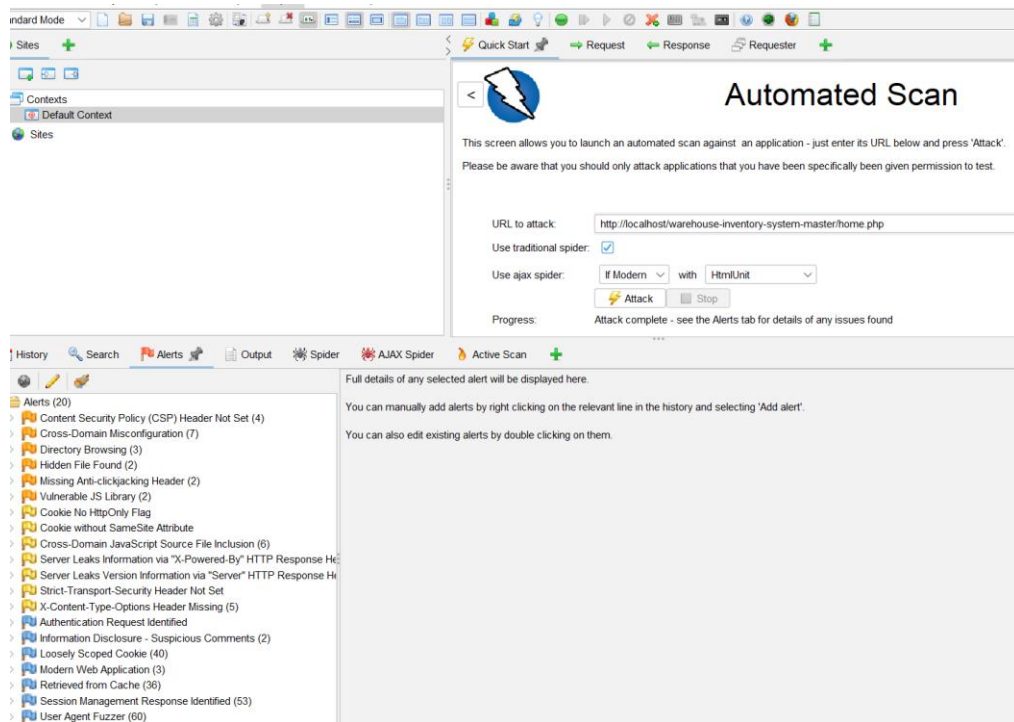
Firefox:

The screenshot shows the ZAP 2.15.0 interface. The top menu bar includes File, Edit, View, Analyse, Report, Tools, Import, Export, Online, and Help. The left sidebar shows the 'Sites' tab with 'Default Context' selected. The main panel displays the 'Automated Scan' configuration window. The 'URL to attack' field is set to 'http://localhost/warehouse-inventory-system-master/home.php'. The 'Use traditional spider' checkbox is checked. The 'Use ajax spider' dropdown is set to 'If Modern' with 'Firefox' selected. The 'Attack' button is highlighted. The 'Progress' bar shows 'Attack complete - see the Alerts tab for details of any issues found'. Below the configuration window, the 'Alerts' tab is active, showing a list of 20 alerts. The 'User Agent Fuzzer' alert is selected, showing details such as 'URL: http://localhost/warehouse-inventory-system-master', 'Risk: Informational', 'Confidence: Medium', 'Parameter: Header User-Agent', 'Attack: Mozilla/4.0 (compatible; MSE 8.0; Windows NT 6.1)', 'Evidence: CWE ID: 0, WASC ID: 0', 'Source: Active (10104 - User Agent Fuzzer)', and 'Input Vector:'. The 'Description' field states: 'Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and hashcode of the response body with the original response.'

Chrome Headlines:

The screenshot shows the ZAP 2.15.0 interface. The top menu bar includes File, Edit, View, Analyse, Report, Tools, Import, Export, Online, and Help. The left sidebar shows the 'Sites' tab with 'Default Context' selected. The main panel displays the 'Automated Scan' configuration window. The 'URL to attack' field is set to 'http://localhost/warehouse-inventory-system-master/home.php'. The 'Use traditional spider' checkbox is checked. The 'Use ajax spider' dropdown is set to 'If Modern' with 'Chrome Headless' selected. The 'Attack' button is highlighted. The 'Progress' bar shows 'Manually stopped'. Below the configuration window, the 'Alerts' tab is active, showing a list of 20 alerts. The 'User Agent Fuzzer' alert is selected, showing details such as 'URL: http://localhost/warehouse-inventory-system-master', 'Risk: Informational', 'Confidence: Medium', 'Parameter: Header User-Agent', 'Attack: Mozilla/4.0 (compatible; MSE 8.0; Windows NT 6.1)', 'Evidence: CWE ID: 0, WASC ID: 0', 'Source: Active (10104 - User Agent Fuzzer)', and 'Input Vector:'. The 'Description' field states: 'Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and hashcode of the response body with the original response.'

HtmlUnit:



Results:

Risk Level	Count
Medium Risk	6
Low Risk	7
Informational	7
Total Alerts = 20	

Medium Risk:

1. Content Security Policy (CSP) Header Not Set

The screenshot shows the Burp Suite interface. The top pane displays an HTTP response with a 404 Not Found status. The response body contains HTML code indicating the requested URL was not found. The bottom pane shows a list of alerts, with 'Content Security Policy (CSP) Header Not Set' highlighted. The details for this alert are shown on the right, including the URL, risk level (Medium), confidence (High), and a description of the vulnerability.

Content Security Policy (CSP) Header Not Set

URL: http://localhost/sitemap.xml
Risk: Medium
Confidence: High
Parameter:
Attack:
Evidence:
CWE ID: 893
WASC ID: 15
Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)
Alert Reference: 10038-1
Input Vector:
Description:
Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML, fra

Solution: Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Solution: We need to ensure that our web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

2. Cross-Domain Misconfiguration

The screenshot shows the Burp Suite interface. The top pane displays an HTTP response with a 200 OK status. The response body contains CSS code for a datepicker. The bottom pane shows a list of alerts, with 'Cross-Domain Misconfiguration' highlighted. The details for this alert are shown on the right, including the URL, risk level (Medium), confidence (Medium), and a description of the vulnerability.

Cross-Domain Misconfiguration

URL: https://cdnjs.cloudflare.com/ajax/libs/bootstrap-datepicker/1.3.0/css/datepicker3.min.css
Risk: Medium
Confidence: Medium
Parameter:
Attack:
Evidence: Access-Control-Allow-Origin: *
CWE ID: 264
WASC ID: 14
Source: Passive (10098 - Cross-Domain Misconfiguration)
Input Vector:
Description:
Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

Solution: Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance). Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

Solution: We need to ensure that sensitive data is not available in an unauthenticated manner (using IP address whitelisting, for instance). We need to configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains or remove all CORS headers entirely to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

3. Directory Browsing

The screenshot shows the ZAP 2.15.0 interface. The top pane displays an HTTP response from `http://localhost/warehouse-inventory-system-master/libs/css/`. The response is an HTML directory listing for `/warehouse-inventory-system-master/libs/css/`. The bottom pane shows the 'Alerts' list with 'Directory Browsing (3)' selected. The details pane on the right provides information about the alert, including the URL, risk level (Medium), confidence (Medium), and a description: 'It is possible to view the directory listing. Directory listing may reveal hidden scripts, include files, backup source files, etc. which can be accessed to read sensitive information.'

Solution: Disable directory browsing. If this is required, make sure the listed files do not induce risks.

4. Hidden File Found

The screenshot shows the ZAP 2.15.0 interface. The top pane displays an HTTP response from `http://localhost/server-status`. The response is an HTML page showing server status information, including 'Server Version: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12' and 'Server PMM: N/A'. The bottom pane shows the 'Alerts' list with 'Hidden File Found (2)' selected. The details pane on the right provides information about the alert, including the URL, risk level (Medium), confidence (High), and a description: 'A sensitive file was identified as accessible or available. This may leak administrative, configuration, or credential information which can be leveraged by a malicious individual to further attack the system or conduct social engineering efforts.'

Solution: We need to consider whether the component is actually required in production or not. If it is not required, then it needs to be disabled. Otherwise, we need to ensure access to it, which requires appropriate authentication and authorization, or limit exposure to internal systems or specific source IPs, etc.

5. Missing Anti-clickjacking Header

The screenshot shows the ZAP 2.15.0 interface. The top pane displays the HTTP response for a request to `http://localhost/warehouse-inventory-system-master/home.php`. The response headers include `HTTP/1.1 200 OK`, `Date: Sat, 05 Oct 2024 15:09:46 GMT`, `Server: Apache/2.4.58 (Ubuntu)`, `OpenSSL/3.1.3`, `PHP/8.2.12`, `X-Powered-By: PHP/8.2.12`, `Expires: Thu, 19 Nov 1981 08:52:00 GMT`, `Cache-Control: no-store, no-cache, must-revalidate`, `Pragma: no-cache`, and `Content-Length: 1683`. The body of the response is an HTML document with a title "Simple Inventory System" and several CSS links.

The bottom pane shows a list of alerts. The "Missing Anti-clickjacking Header" alert is selected, showing the following details:

- URL:** `http://localhost/warehouse-inventory-system-master/home.php`
- Risk:** Medium
- Confidence:** Medium
- Parameter:** `x-frame-options`
- Attack:**
- Evidence:**
 - CWE ID: 1021
 - WASC ID: 15
 - Source: Passive (10020 - Anti-clickjacking Header)
 - Alert Reference: 10020-1
 - Input Vector:
 - Description: The response does not protect against "ClickJacking" attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
- Other Info:**
- Solution:** Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the

Solution: Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. We need to ensure one of them is set on all web pages returned by our website/app. If we expect the page to be framed only by pages on our server (e.g., it is part of a FRAMESET), then we will want to use SAMEORIGIN. Otherwise, if you never expect the page to be framed, we should use DENY. Alternatively, we can consider implementing the Content Security Policy's "frame-ancestors" directive.

6. Vulnerable JS Library

The screenshot shows the ZAP 2.15.0 interface. The top pane displays the HTTP request for a request to `https://maxcdn.bootstrapcdn.com/bootstrap/3.3.4/js/bootstrap.min.js`. The request headers include `GET https://maxcdn.bootstrapcdn.com/bootstrap/3.3.4/js/bootstrap.min.js HTTP/1.1`, `host: maxcdn.bootstrapcdn.com`, `Connection: keep-alive`, `sec-ch-ua-platform: "Windows"`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0 Safari/`, `sec-ch-ua: "Google Chrome";v="129", "Not-A.Brand";v="8", "Chromium";v="129"`, `sec-ch-ua-mobile: ?0`, `Accept: */*`, `Sec-Fetch-Site: cross-site`, `Sec-Fetch-Mode: no-cors`, `Sec-Fetch-Dest: script`, `Referer: http://localhost/`, and `Accept-Language: en-US,en;q=0.9`.

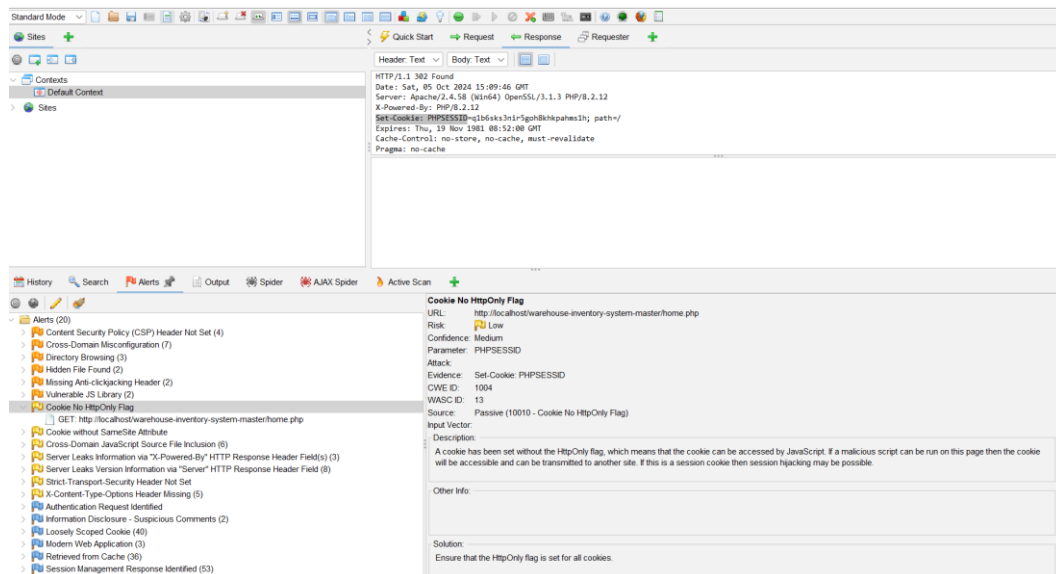
The bottom pane shows a list of alerts. The "Vulnerable JS Library" alert is selected, showing the following details:

- URL:** `https://maxcdn.bootstrapcdn.com/bootstrap/3.3.4/js/bootstrap.min.js`
- Risk:** Medium
- Confidence:** Medium
- Parameter:**
- Attack:**
- Evidence:** `/3.3.4/js/bootstrap.min.js`
- CWE ID:** 829
- WASC ID:**
- Source:** Passive (10003 - Vulnerable JS Library (Powered by Retire.js))
- Input Vector:**
- Description:** The identified library bootstrap, version 3.3.4 is vulnerable.
- Other Info:**
 - CVE-2018-14041
 - CVE-2019-8331
 - CVE-2018-20677
- Solution:** Please upgrade to the latest version of bootstrap.

Solution: We need to upgrade to the latest version of Bootstrap.

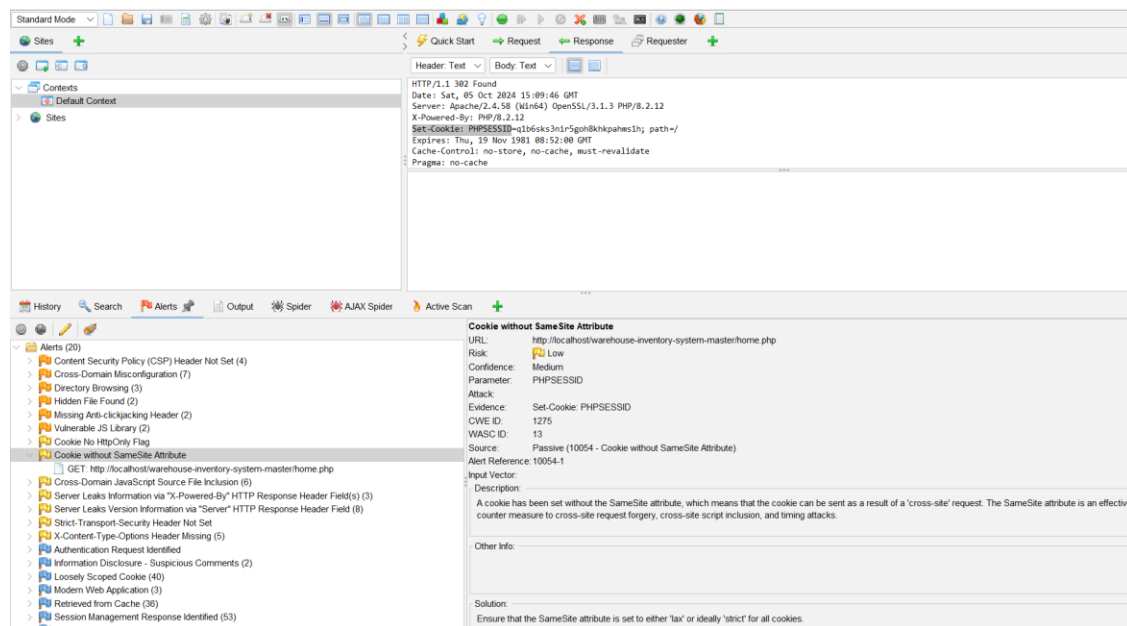
Low Risk:

1. Cookie No HttpOnly Flag



Solution: We need to ensure that the HttpOnly flag is set for all cookies.

2. Cookie without SameSite Attribute



Solution: We need to ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.

3. Cross-Domain JavaScript Source File Inclusion

The screenshot shows the Burp Suite interface. The top pane displays the HTTP response for a request to `http://localhost/warehouse-inventory-system-master/home.php`. The response includes headers like `HTTP/1.1 200 OK`, `Date: Sat, 05 Oct 2024 15:09:46 GMT`, and `X-Powered-By: PHP/8.2.12`. The body contains HTML code with a form and a script tag: `<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js"></script>`. The bottom pane shows the Alerts list with 20 alerts. The selected alert is "Cross-Domain JavaScript Source File Inclusion (6)". The details pane on the right shows the alert information: URL is `http://localhost/warehouse-inventory-system-master/home.php`, Risk is Low, Confidence is Medium, and the attack evidence is the script tag from the response body.

Solution: We need to ensure JavaScript source files are loaded from only trusted sources, and the sources can't be controlled by end users of the application.

4. Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)

The screenshot shows the Burp Suite interface. The top pane displays the HTTP response for a request to `http://localhost/warehouse-inventory-system-master/auth.php`. The response includes headers like `HTTP/1.1 302 Found`, `Date: Sat, 05 Oct 2024 15:09:46 GMT`, and `X-Powered-By: PHP/8.2.12`. The bottom pane shows the Alerts list with 20 alerts. The selected alert is "Server Leaks Information via 'X-Powered-By' HTTP Response Header Field(s) (3)". The details pane on the right shows the alert information: URL is `http://localhost/warehouse-inventory-system-master/auth.php`, Risk is Low, Confidence is Medium, and the attack evidence is the `X-Powered-By: PHP/8.2.12` header from the response.

Solution: We need to ensure that our web server, application server, load balancer, etc., is configured to suppress "X-Powered-By" headers.

5. Server Leaks Version Information via "Server" HTTP Response Header Field

The screenshot shows the Burp Suite interface. The top pane displays an HTTP response from `http://localhost/warehouse-inventory-system-master/libs/js/functions.js`. The response header includes `Server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12`. The left sidebar shows a list of alerts, with `Server Leaks Version Information via "Server" HTTP Response Header Field (8)` selected. The right pane displays the details of this alert, including the URL, risk level (Low), confidence (High), and a description of the vulnerability.

Server Leaks Version Information via "Server" HTTP Response Header Field

URL: `http://localhost/warehouse-inventory-system-master/libs/js/functions.js`

Risk: Low

Confidence: High

Parameter:

Attack: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12

Evidence: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12

CWE ID: 200

WASC ID: 13

Source: Passive (10036 - HTTP Server Response Header)

Input Vector:

Description: The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

Other Info:

Solution: Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.

Solution: We need to ensure that our web server, application server, load balancer, etc., is configured to suppress the "Server" header or provide generic details.

6. Strict-Transport-Security Header Not Set

The screenshot shows the Burp Suite interface. The top pane displays an HTTP response from `https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js`. The response header includes `Strict-Transport-Security: max-age=31536000; includeSubDomains`. The left sidebar shows a list of alerts, with `Strict-Transport-Security Header Not Set` selected. The right pane displays the details of this alert, including the URL, risk level (Low), confidence (High), and a description of the vulnerability.

Strict-Transport-Security Header Not Set

URL: `https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js`

Risk: Low

Confidence: High

Parameter:

Attack:

Evidence:

CWE ID: 319

WASC ID: 15

Source: Passive (10035 - Strict-Transport-Security Header)

Alert Reference: 10035-1

Input Vector:

Description: HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL). HSTS is an IETF standards track protocol and is specified in RFC 6797.

Other Info:

Solution: Ensure that your web server, application server, load balancer, etc. is configured to enforce Strict-Transport-Security.

Solution: We ensure that our web server, application server, load balancer, etc. is configured to enforce Strict-Transport-Security.

7. X-Content-Type-Options Header Missing

The screenshot shows the Burp Suite interface. The top pane displays the HTTP request details for a JavaScript file. The bottom pane shows a list of alerts, with 'X-Content-Type-Options Header Missing (5)' selected. The alert details pane on the right provides the following information:

- URL:** http://localhost/warehouse-inventory-system-master/libs/js/functions.js
- Risk:** Low
- Confidence:** Medium
- Parameter:** x-content-type-options
- Attack:**
- Evidence:** CWE ID: 693, WASC ID: 15
- Source:** Passive (10021 - X-Content-Type-Options Header Missing)
- Input Vector:**
- Description:** The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
- Other Info:** This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At 'High' threshold this scan rule will not alert on client or server error responses.
- Solution:** Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Solution: We need to ensure that the application/web server sets the Content-Type header appropriately and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, we also can ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all or that can be directed by the web application/web server not to perform MIME-sniffing.

Informational Risk:

1. Authentication Request Identified

The screenshot shows the Burp Suite interface. The top pane displays the HTTP request details for a POST request to the authentication endpoint. The bottom pane shows a list of alerts, with 'Authentication Request Identified' selected. The alert details pane on the right provides the following information:

- URL:** http://localhost/warehouse-inventory-system-master/auth.php
- Risk:** Informational
- Confidence:** Low
- Parameter:** username
- Attack:** password
- Evidence:** CWE ID: 693, WASC ID: 15
- Source:** Passive (10111 - Authentication Request Identified)
- Input Vector:**
- Description:** The given request has been identified as an authentication request. The 'Other Info' field contains a set of key-value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to 'Auto-Detect' then this rule will change the authentication to match the request identified.
- Other Info:** userParam=username, userValue=, passwordParam=password
- Solution:** This is an informational alert rather than a vulnerability and so there is nothing to fix.

Solution: This is an informational alert rather than a vulnerability, so there is nothing to fix.

2. Information Disclosure – Suspicious Comments:

The screenshot shows the Burp Suite interface. The top panel displays the HTTP response headers and body. The body contains a JavaScript file with a comment that is highlighted in red. The comment is a multi-line string that appears to be a mix of code and comments, possibly a developer's note or a debug statement. The bottom panel shows the 'Alerts' tab with a list of detected issues. One alert is selected, titled 'Information Disclosure - Suspicious Comments'. The details of this alert are shown on the right, including the URL, risk level, confidence, and a description of the issue. The description states that the response appears to contain suspicious comments which may help an attacker, and provides a note about matches made within script blocks or files.

Solution: We need to remove all comments that return information that may help an attacker and fix any underlying problems they refer to.

3. Loosely Scoped Cookie

The screenshot shows the Burp Suite interface. The top panel displays the HTTP response headers and body. The body contains a JavaScript file with a comment that is highlighted in red. The comment is a multi-line string that appears to be a mix of code and comments, possibly a developer's note or a debug statement. The bottom panel shows the 'Alerts' tab with a list of detected issues. One alert is selected, titled 'Loosely Scoped Cookie'. The details of this alert are shown on the right, including the URL, risk level, confidence, and a description of the issue. The description states that cookies can be scoped by domain or path, and this check is only concerned with domain scope. The domain scope applied to a cookie determines which domains can access it. For example, a cookie can be scoped strictly to a subdomain e.g. www.notrusted.com, or loosely scoped to a parent domain e.g. notarusted.com. In the latter case, any subdomain of notarusted.com can access the cookie. Loosely scoped cookies are common in mega-applications like google.com and live.com. Cookies set

Solution: Always scope cookies to an FQDN (Fully Qualified Domain Name).

4. Modern Web Application

The screenshot displays the Burp Suite interface. On the left, a list of alerts is visible, including 'Hidden File Found (2)', 'Missing Anti-clickjacking Header (2)', and 'Modern Web Application (3)'. The main panel shows the details of the 'Modern Web Application' alert. The URL is `http://localhost/warehouse-inventory-system-master/home.php`. The risk is 'Informational' and the confidence is 'Medium'. The attack evidence shows a script tag for jQuery. The description states: 'The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard spider.' The solution section is empty.

Solution: This is an informational alert and so no changes are required.

5. Retrieved from Cache

The screenshot displays the Burp Suite interface. On the left, a list of alerts is visible, including 'Retrieved from Cache (36)'. The main panel shows the details of the 'Retrieved from Cache' alert. The URL is `https://cdnjs.cloudflare.com/ajax/libs/bootstrap-datepicker/1.3.0/css/datepicker3.min.css`. The risk is 'Medium' and the confidence is 'Medium'. The attack evidence shows the 'Age' header is 357797. The description states: 'The content was retrieved from a shared cache. If the response data is sensitive, personal or user-specific, this may result in sensitive information being leaked. In some cases, this may even result in a user gaining complete control of the session of another user, depending on the configuration of the caching components in use in their environment. This is primarily an issue where caching servers such as "proxy" caches are configured on the local network. This configuration is typically found in:'. The solution section provides instructions to validate the response does not contain sensitive information and to use specific HTTP response headers to limit or prevent the content from being stored and retrieved from the cache.

Solution: We need to validate that the response does not contain sensitive, personal, or user-specific information. If it does, consider the use of the following HTTP response headers, to limit or prevent the content from being stored and retrieved from the cache by another user:

- Cache-Control: no-cache, no-store, must-revalidate, private
- Pragma: no-cache
- Expires: 0

This configuration directs both HTTP 1.0 and HTTP 1.1 compliant caching servers to not store the response and to not retrieve the response (without validation) from the cache in response to a similar request.

6. Session Management Response Identified

The screenshot displays the Burp Suite interface. On the left, the 'Contexts' pane shows 'Default Context' and 'Sites'. The main pane shows a list of 53 GET requests to 'http://localhost/warehouse-inventory-system-master/home.php'. The right pane shows the details of the selected alert, 'Session Management Response Identified'.

Session Management Response Identified

URL: http://localhost/warehouse-inventory-system-master/home.php
Risk: Informational
Confidence: Medium
Parameter: PHPSESSID
Attack:
Evidence: q1b6sks3nir5goh8khpahms1h
CVE ID:
WASC ID:
Source: Passive (10112 - Session Management Response Identified)
Input Vector:
Description:
The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to 'Auto-Detect' then this rule will change the session management to use the tokens identified.
Other Info:
cookie:PHPSESSID
Solution:
This is an informational alert rather than a vulnerability and so there is nothing to fix.

Solution: This is an informational alert rather than a vulnerability, so there is nothing to fix.

7. User Agent Fuzzer

The screenshot displays the Burp Suite interface. On the left, the 'Contexts' pane shows 'Default Context' and 'Sites'. The main pane shows a list of 80 GET requests to 'http://localhost/warehouse-inventory-system-master' and 'http://localhost/warehouse-inventory-system-master/libs'. The right pane shows the details of the selected alert, 'User Agent Fuzzer'.

User Agent Fuzzer

URL: http://localhost/warehouse-inventory-system-master
Risk: Informational
Confidence: Medium
Parameter: Header User-Agent
Attack: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence:
CVE ID: 0
WASC ID: 0
Source: Active (10104 - User Agent Fuzzer)
Input Vector:
Description:
Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.
Other Info:
Solution:
Reference:
<https://owasp.org/wstt>

Functional Testing

Automation Test plan:

Test strategy:

Various technology tools and frameworks are extensively supporting Test Automation in software development. With the appropriate technology and approach, QA teams can drastically reduce testing time and effort while maintaining comprehensive test coverage.

Additionally, the LMS application has been released to the public with various features and tested manually. However, this would heavily cost most of the time for re-testing all features when having an updated version release. Thus, applying automated tests is essential not only to maintain a high standard of quality of the product but also to keep receiving valuable feedback from end users and product owners. Automated test optimizes the team's efforts and resources

Features to be tested:

Feature ID	Features	Descriptions
1	Home page	Website home page
2	Login	User login page with username and password
3	Dashboard	Showing user, product, categories, users.
4	Log Out	Log the user out of the system.

Features not to be tested:

Test deliverables and schedule:

The testing team to develop and deliver all important documents during the testing activity, and they will align with the work package and milestones below:

Work ID	Deliverable	Descriptions	Milestone
1	Test Plan Document	The testing plan is developed by the project starting and can be updated during the development phases.	Finished by week one
2	Test Cases Document + Manual test case + Automation test scripts	The test report documents automated test scripts and manual tests. Besides, a manual test is necessary when it needs humans for some testing.	Finished by week nine
3	Test Incident Reports Document	Execute test cases, and log bugs into the defect tracking system.	Finished by week ten
4	Test Summary Reports Documents	A system summary report includes fixed bugs and some carried-over bugs to subsequent upgrade version development phases.	Finished by week eleven

Responsibilities:

Student No	Member	Role	Responsibilities
1	Charles Yeo	QA Manager	Review test cases, review and approve the issues
2	Synthia Islam	QA Lead	Assign tasks to track the testing progress; Prepare Documentation; Develop and Execute Automation Test, Usability Test, Performance Test, Security Test scripts;
3	Musrat Jahan	QA	Set up the test environment, develop and execute Automation Test, Usability Test, Performance Test, Security Test scripts
4	Hussein Salami	QA	Develop and Execute Automation Test, Usability Test, Performance Test, Security Test scripts
5	Muhammad Asad Nawaz	QA	Develop and Execute Automation Test, Usability Test, Performance Test, Security Test scripts

Risks and mitigations:

The below shows the identified risks with descriptions and mitigations.

ID No.	R1
Risk	Delays in delivery of test items
Description	The developer team delivering test items later might require increased night shift scheduling to meet the delivery date.
Response	The developer and tester team work closely to keep progress informed. Communicate and negotiate with the project manager and product owner to adjust each sprint's schedule or release items.

ID No.	R2
Risk	Uncleared requirement
Description	An unclear requirement may highly cost both of developer and tester effort to fix the error.
Response	It is necessary to have communication between developers, testers, and stakeholders to have the feature developed as required.

ID No.	R3
Risk	Automated team member absent
Description	The unplanned absence of an automated tester may cause delays in product item delivery.
Response	Keep close communication and oversee team members' issues to have in-time support.

ID No.	R4
Risk	An automated team member lacks experience
Description	Less experience automated testers may cause delays in developing testing scripts.
Response	Develop a training strategy for project team members. Assign test scripts based on team member's skill level and feature complexity. Members should be required to work longer hours to ensure that the test items are delivered on time.

Comparison and justification of automation tool

Automation Testing necessitates purchasing new devices and software, which requires allocating time and resources to determine which tools/software are optimal.

Comparison of automated web testing tool:

No.	Robot Framework	Selenium
1	Test cases are easy to develop with Python	Test cases are easy to create with Python, java
2	For Acceptance Test-Driven Development, the Robot framework is employed.	Selenium enables the programming of automation by the user.
3	It incorporates a variety of libraries to facilitate automation.	It is a web driver without an integrated library.
4	It can execute test cases without the need for external automation unwrapped. Test cases can be API or element based.	For executing selenium-written test cases, automation unwrapped is necessary.
5	It is used to automate every sort of test, including acceptability testing and others.	Automation is limited to regression and performance testing only.
6	Robot framework may be utilized to test both web and desktop apps.	Using selenium, web-based software and apps are tested.
7	Users can define individualized test scenarios.	There is no custom test case functionality available.
8	It may be utilized on all systems.	Selenium cannot operate on mobile devices platform
9	Dictionary and list variables are supported.	It does not support variables.

10	The robot framework runs with Selenium web driver for web application testing. Test scripts are easy to develop with a natural language textbase.	Developing Selenium test cases somehow requires experience with XPath access on a web application, which is complex work for whom has less experience.
----	---	--

Those automated web testing tools have numerous advantages and disadvantages. However, the selection of the right tool depends on several factors, such as project requirements, programming expertise, and budget. Our automated testing team has chosen to use Robot Framework for the *Warehouse Inventory System* based on the following considerations:

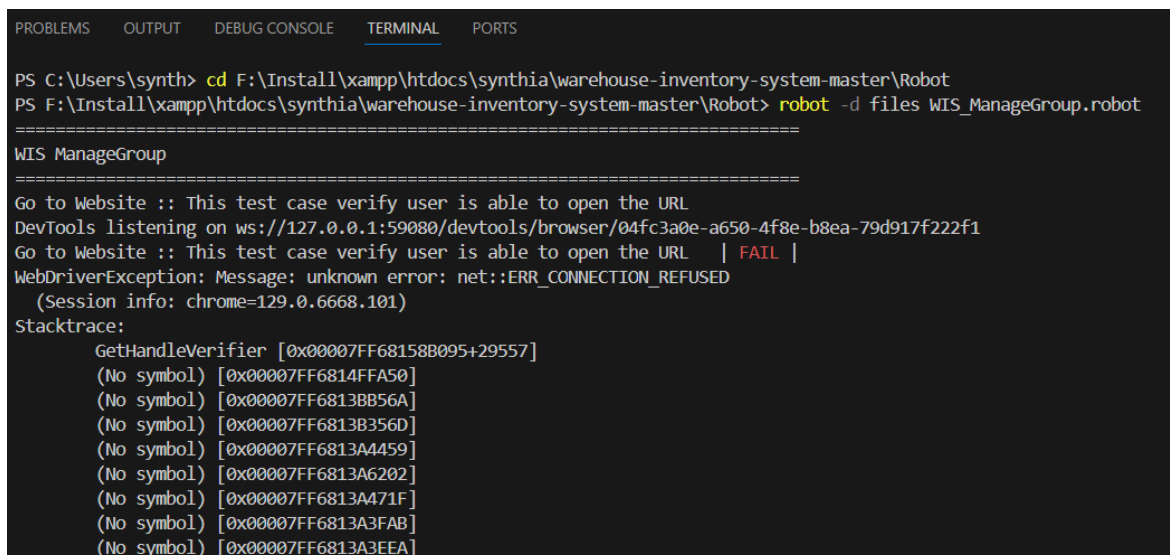
1. The Warehouse Inventory System is built using PHP
2. Most of the features are straightforward.
3. Robot Framework is familiar to our team from previous projects
4. Test scripts from previous projects can be re-used
5. Testing timelines are short, with only three weeks.
6. A new team member with limited experience
7. Cross-platform and cross-browser testing capabilities

Environmental Needs for Robot Framework:

1. Python 3.x
2. Robot Framework
3. Selenium Library
4. Browser: Chrome, Firefox, Edge, Opera, etc.

Pre-Conditions:

1. We need to run XAMPP first as our project is based on PHP. Otherwise, the robot script will not run. It will show errors like below:



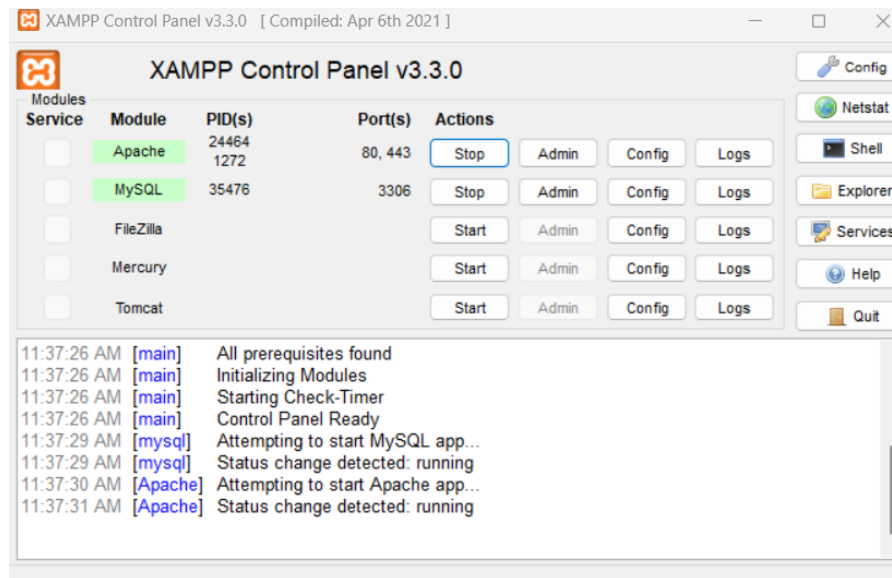
```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\synth> cd F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_ManageGroup.robot
=====
WIS_ManageGroup
=====
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:59080/devtools/browser/04fc3a0e-a650-4f8e-b8ea-79d917f222f1
Go to Website :: This test case verify user is able to open the URL | FAIL |
WebDriverException: Message: unknown error: net::ERR_CONNECTION_REFUSED
(Session info: chrome=129.0.6668.101)
Stacktrace:
  GetHandleVerifier [0x00007FF68158B095+29557]
  (No symbol) [0x00007FF6814FFA50]
  (No symbol) [0x00007FF6813BB56A]
  (No symbol) [0x00007FF6813B356D]
  (No symbol) [0x00007FF6813A4459]
  (No symbol) [0x00007FF6813A6202]
  (No symbol) [0x00007FF6813A471F]
  (No symbol) [0x00007FF6813A3FAB]
  (No symbol) [0x00007FF6813A3EEA]

```

Therefore, we will run XAMPP and start Apache and MySQL first to execute Robot script code successfully.



Test cases for functional requirements conducted by automated testing using Robot Framework:

Keywords used from Robot Framework and Selenium Library:

- Go to Website
 - Open Browser
 - Maximize Browser Window
 - Wait Until Element Is Visible
 - Input Text (renamed as Input username and Input passwords in the code)
 - Click Element
 - Click link
 - Click Button
 - Element Should Be Visible
 - Submit Credentials
 - Page Should Contain
 - Sleep 1 seconds
 - Go Back
 - Close Browser
 - Chrome
 - xpath://a[@class='toggle' and contains(span, 'Admin User')] (for Click Admin User Dropdown)
 - xpath://li[@class='last']/a (for Click Logout Link)
 - xpath://table//tr[4]//td[5]//div[@class='btn-group']//a[@data-original-title='Remove']
- **Login Module: - (Musrat Jahan)**

Allowing the staff/User to log in to the WIS:

We have created 12 test cases for User Login using all combinations of passwords:

- i. Verify Successful Login to Warehouse Inventory System
- ii. Failed Login Test Invalid Username and Valid Password
- iii. Failed Login Test Valid Username and Invalid Password
- iv. Failed Login Test Invalid Username and Invalid Password
- v. Failed Login Test Empty Username and Valid Password

- vi. Failed Login Test Valid Username and Empty Password
- vii. Failed Login Test Empty Username and Empty Password
- viii. Verify Successful Login to Warehouse Inventory System
- ix. Different case for Upper Username and Valid password
- x. Different case for Lower Username and Valid password
- xi. Multiple Failed Attempts
- xii. Logout

Writing automated testing code using Robot Framework:

```
C:\> Users > Musra > Downloads > Semester 2,2024 > Software Process > Project > Automation > WIS_login_resource.robot
Load in Interactive Console
1 *** Settings ***
2 Library SeleniumLibrary
3
Load in Interactive Console
4 *** Variables ***
5 ${URL} http://localhost/warehouse-inventory-system-master/index.php
6 ${BROWSER} chrome
7
8 *** Keywords ***
Load in Interactive Console
9 Go to Website
10 [Documentation] Open the login page
11 Open Browser ${URL} ${BROWSER}
12 Maximize Browser Window
13 Page Should Contain welcome
14
Load in Interactive Console
15 Input username
16 [Arguments] ${username}
17 Input Text username ${username}
18
Load in Interactive Console
19 Input passwords
20 [Arguments] ${password}
21 Input Text password ${password}
22
Load in Interactive Console
23 Submit Credentials
24 Click Button Login
...
```

```
WIS_loginrobot X WIS_login_resource.robot WIS_logout.robot WIS_Dashboard.robot
C:\> Users > Musra > Downloads > Semester 2,2024 > Software Process > Project > Automation > WIS_login_resource.robot > {} Settings
Run Suite | Debug Suite | Load in Interactive Console
1 *** Settings ***
2 [Documentation] Separate Test Cases for Invalid, Valid, Empty Login, Case Sensitivity, and Multiple Failed Attempts
3 Resource WIS_login_resource.robot
4 Suite Setup Go to Website
5 Suite Teardown Close Browser
6
Load in Interactive Console
7 *** Variables ***
8 ${VALID_USER} admin
9 ${VALID_PASSWORD} admin
10 ${INVALID_USER} invalid_user
11 ${INVALID_PASSWORD} wrong_pw
12 ${EMPTY} ${EMPTY}
13
14 *** Test Cases ***
15
Run | Debug | Run in Interactive Console
16 Invalid Username and Valid Password
17 [Documentation] Test invalid username with valid password
18 Login With Invalid Credentials Should Fail ${INVALID_USER} ${VALID_PASSWORD}
19
Run | Debug | Run in Interactive Console
20 Valid Username and Invalid Password
21 [Documentation] Test valid username with invalid password
22 Login With Invalid Credentials Should Fail ${VALID_USER} ${INVALID_PASSWORD}
23
Run | Debug | Run in Interactive Console
24 Invalid Username and Invalid Password
25 [Documentation] Test invalid username and invalid password
26 Login With Invalid Credentials Should Fail ${INVALID_USER} ${INVALID_PASSWORD}
27
28
Run | Debug | Run in Interactive Console
29 Empty Username and Valid Password
30 [Documentation] Test empty username with valid password
31 Login With Invalid Credentials Should Fail ${EMPTY} ${VALID_PASSWORD}
32
Run | Debug | Run in Interactive Console
33 Valid Username and Empty Password
34 [Documentation] Test valid username with empty password
35 Login With Invalid Credentials Should Fail ${VALID_USER} ${EMPTY}
36
Run | Debug | Run in Interactive Console
37 Empty Username and Empty Password
38 [Documentation] Test empty username and empty password
39 Login With Invalid Credentials Should Fail ${EMPTY} ${EMPTY}
40
41
```

```

41
42 # Valid Login Scenario
43 Run | Debug | Run in Interactive Console
44 Valid Username and Valid Password
45 [Documentation] Test valid username and valid password
46 Login With Valid Credentials Should Succeed ${VALID_USER} ${VALID_PASSWORD}
47 Logout As Admin User
48
49 Run | Debug | Run in Interactive Console
50 Different Case Username and Valid Password
51 [Documentation] Test login functionality with a username in different case and valid password
52 Login With Valid Credentials Should Succeed ${VALID_USER.upper()} ${VALID_PASSWORD}
53 Logout As Admin User
54 Login With Valid Credentials Should Succeed ${VALID_USER.lower()} ${VALID_PASSWORD}
55 Logout As Admin User
56
57 Run | Debug | Run in Interactive Console
58 Multiple Failed Login Attempts
59 [Documentation] Test login functionality after multiple failed login attempts
60 FOR ${index} IN RANGE 3
61     Login With Invalid Credentials Should Fail ${INVALID_USER} ${INVALID_PASSWORD}
62 END
63 Login With Valid Credentials Should Succeed ${VALID_USER} ${VALID_PASSWORD}
64 Logout As Admin User
65
66 Run | Debug | Run in Interactive Console
67 Logout
68 [Documentation] Logout from system
69 Login With Valid Credentials Should Succeed ${VALID_USER} ${VALID_PASSWORD}
70 Click Admin User Dropdown
71 Click Logout Link
72
73 *** Keywords ***
74
75 Load in Interactive Console
76 Login With Invalid Credentials Should Fail
77 [Arguments] ${username} ${password}
78 Input username ${username} # Updated locator for username
79 Input passwords ${password} # Updated locator for password
80 Submit Credentials
81
82 # Check specific cases for empty username and password
83 Run Keyword If '${username}' == '' and '${password}' == ''
84 ... Page Should Contain Username can't be blank.
85
86 # Check case for empty username only
87 Run Keyword If '${username}' == '' and '${password}' != ''
88 ... Page Should Contain Username can't be blank.
89
90 # Check case for empty password only
91 Run Keyword If '${username}' != '' and '${password}' == ''
92 ... Page Should Contain Password can't be blank.
93
94 # Check for invalid username and password case
95 Run Keyword If '${username}' != '' and '${password}' != ''
96 ... Page Should Contain Sorry Username/Password incorrect.
97
98 Load in Interactive Console
99 Login With Valid Credentials Should Succeed
100 [Arguments] ${username} ${password}
101 Input username ${username} # Updated locator for username
102 Input passwords ${password} # Updated locator for password
103 Submit Credentials
104 Page Should Contain Welcome
105
106 Load in Interactive Console
107 Click Admin User Dropdown
108 [Documentation] Click on the Admin User dropdown to reveal the logout option.
109 Click Element //a[@class='toggle' and contains(span, 'Admin User')]
110
111 Load in Interactive Console
112 Click Logout Link
113 [Documentation] Click on the Logout link in the dropdown menu.
114 Click Element //li[@class='last']/a
115
116 Load in Interactive Console
117 Logout As Admin User
118 [Documentation] Perform logout by clicking on the Admin User dropdown and then Logout.
119 Click Admin User Dropdown
120 Click Logout Link

```


Executing robot file:

```

=====
DEBLOGMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ROBOT DOCUMENTATION  ROBOT OUTPUT
=====
PS C:\Users\Vsura\New folder\htdocs\warehouse-inventory-system-master\Robot> robot -d results WIS_login.robot
[ ERROR ] Error in file 'C:\Users\Vsura\New folder\htdocs\warehouse-inventory-system-master\Robot\WIS_login.robot' on line 12: Setting variable '${EMPTY}' failed: Recursive variable definition.
=====
WIS login :: Separate Test Cases for Invalid, Valid, Empty Login, Case Sens...
=====
[ WARN ] The chromedriver version (128.0.6613.119) detected in PATH at C:\Python310\Scripts\chromedriver-win64\chromedriver-win64\chromedriver.exe might not be compatible with the detected chrome vers
on (129.0.6668.90); currently, chromedriver 129.0.6668.89 is recommended for chrome 129.*, so it is advised to delete the driver in PATH and retry
=====
DevTools listening on ws://127.0.0.1:58608/devtools/browser/170ac50c-0b15-47e1-949e-df625e58e3cc
Invalid Username and Valid Password :: Test invalid username with ... | PASS |
-----
Valid Username and Invalid Password :: Test valid username with in... | PASS |
-----
Invalid Username and Invalid Password :: Test invalid username and... | PASS |
-----
Empty Username and Valid Password :: Test empty username with vali... | PASS |
-----
Valid Username and Empty Password :: Test valid username with empt... | PASS |
-----
Empty Username and Empty Password :: Test empty username and empty... | PASS |
-----
Valid Username and Valid Password :: Test valid username and valid... | PASS |
-----
Different Case Username and Valid Password :: Test login functiona... | PASS |
-----
Multiple Failed Login Attempts :: Test login functionality after m... | PASS |
-----
Logout :: Logout from system | PASS |
-----
WIS login :: Separate Test Cases for Invalid, Valid, Empty Login, ... | PASS |
10 tests, 10 passed, 0 failed
=====
Output: C:\Users\Vsura\New folder\htdocs\warehouse-inventory-system-master\Robot\results\output.xml
Log: C:\Users\Vsura\New folder\htdocs\warehouse-inventory-system-master\Robot\results\log.html
Report: C:\Users\Vsura\New folder\htdocs\warehouse-inventory-system-master\Robot\results\report.html
PS C:\Users\Vsura\New folder\htdocs\warehouse-inventory-system-master\Robot>

```

After running the code, we can see that all the test cases passed.

Results and Report:

Home Page

WVS login Report

File

C:\Users\Musra\New%20folder\htdocs\warehouse-inventory-system-master\Robot\results\report.html#totals

Warehouse

Pharmacyadmin

Login Registration

All Bookings

LOG

Test Details

All

Suites

Search

Status:

10 tests total, 10 passed, 0 failed, 0 skipped

Total Time:

00:00:05:386

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WVS login: Different Case Username and Valid Password	Test login functionality with a username in different case and valid password		PASS		00:00:01:101	20241005 23:58:32:045 20241005 23:58:31:146
WVS login: Empty Username and Empty Password	Test empty username and empty password		PASS		00:00:00:174	20241005 23:58:32:361 20241005 23:58:32:535
WVS login: Empty Username and Valid Password	Test empty username with valid password		PASS		00:00:00:203	20241005 23:58:31:943 20241005 23:58:32:146
WVS login: Invalid Username and Invalid Password	Test invalid username and invalid password		PASS		00:00:00:233	20241005 23:58:31:709 20241005 23:58:31:942
WVS login: Invalid Username and Valid Password	Test invalid username with valid password		PASS		00:00:00:349	20241005 23:58:31:109 20241005 23:58:31:458
WVS login: Logout	Logout from system		PASS		00:00:00:495	20241005 23:58:36:001 20241005 23:58:36:496
WVS login: Multiple Failed Login Attempts	Test login functionality after multiple failed login attempts		PASS		00:00:01:853	20241005 23:58:34:148 20241005 23:58:36:001
WVS login: Valid Username and Empty Password	Test valid username with empty password		PASS		00:00:00:216	20241005 23:58:32:145 20241005 23:58:32:361
WVS login: Valid Username and Invalid Password	Test valid username with invalid password		PASS		00:00:00:252	20241005 23:58:31:458 20241005 23:58:31:710
WVS login: Valid Username and Valid Password	Test valid username and valid password		PASS		00:00:00:510	20241005 23:58:32:538 20241005 23:58:33:045

- **Dashboard Module: - (Musrat Jahan)**

Allowing the staff/User to explore dashboard in WIS:

Created 3 test cases for this scenario:

- Go to website
- Login with Admin username
- Go to Dashboard

Writing automated testing code using Robot Framework:

```
C:\> Users\Musra > Downloads\Semester 2,2024 > Software Process > Project > Automation > WIS_Dashboard.robot
```

```
Run Suite | Debug Suite | Load in Interactive Console
1  Settings *
2  Library SeleniumLibrary
3  Library Collections
4
5  Load in Interactive Console
6  * Variables *
7  ${BROWSER} chrome
8  ${URL} http://localhost/warehouse-inventory-system-master/home.php
9  ${ADMIN_USER} admin
10 ${ADMIN_PASS} admin
11
12 * Test Cases *
13 Run | Debug | Run in Interactive Console
14 Go to Website
15 [Documentation] This test case verify user is able to open the URL
16 Open Browser ${URL} ${Browser}
17 Page Should Contain Welcome
18
19 Run | Debug | Run in Interactive Console
20 Login to your account
21 Input Text username admin
22 Input Text password admin
23 Click Button Login
24
25 Run | Debug | Run in Interactive Console
26 Go to Dashboard
27 [Documentation] Verify that an admin can go to dashboard
28 Click Element xpath=//div[@class="sidebar"]/ul/li[1]/a[1]
```

Executing robot file:

```
PS C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot> robot -d results WIS_dashboard.robot
=====
WIS dashboard
=====
[ WARN ] The chromedriver version (128.0.6613.119) detected in PATH at C:\Python310\Scripts\chromedriver-win64\chromedriver-win64\chrome
on (129.0.6668.90); currently, chromedriver 129.0.6668.89 is recommended for chrome 129.*, so it is advised to delete the driver in P
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:59276/devtools/browser/3cc4b63-f647-486f-8b87-2830403e40cf
Go to Website :: This test case verify user is able to open the URL | PASS |
-----
Login to your account | PASS |
-----
Go to Dashboard :: Verify that an admin can go to dashboard | PASS |
-----
WIS dashboard | PASS |
3 tests, 3 passed, 0 failed
=====
Output: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\output.xml
Log: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\log.html
Report: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\report.html
```

After running the code, we can see that all the test cases passed.

Results and Report:

WIS dashboard Report

Generated

20241006 00:03:04 UTC+09:30

11 seconds ago

Summary Information

Status:

All tests passed

Start Time:

20241006 00:03:01 379

End Time:

20241006 00:03:04 814

Elapsed Time:

00:00:03 435

Log File:

[log.html](#)

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	3	3	0	0	00:00:03	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
WIS dashboard	3	3	0	0	00:00:03	

Test Details

AllTagsSuitesSearch

Suite:

WIS dashboard

Status:

3 tests total, 3 passed, 0 failed, 0 skipped

Start / End Time:

20241006 00:03:01 379 / 20241006 00:03:04 814

Elapsed Time:

00:00:03 435

Log File:

[log.html#1](#)

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS dashboard: Go to Website	This test case verify user is able to open the URL		PASS		00:00:02 560	20241006 00:03:01 696 20241006 00:03:04 258
WIS dashboard: Login to your account			PASS		00:00:00 453	20241006 00:03:04 258 20241006 00:03:04 711
WIS dashboard: Go to Dashboard	Verify that an admin can go to dashboard		PASS		00:00:00 102	20241006 00:03:04 711 20241006 00:03:04 813

▪ User Management Module: - (Synthia Islam)

➤ Manage Groups (View, Add, Edit, Delete): - (Synthia Islam)

Allowing the staff/user to manage User Group in Warehouse Inventory System (WIS):

Created 13 test cases for managing group functionalities which include viewing group details, adding new user groups, editing existing groups, and deleting the groups:

- Go to website
- Login with Admin username
- Go to User Management menu
- View group details by going to Manage Group submenu
- Click Add New Group button to go to Add New User Group page
- Add new user group by giving input details
- Then again Go to User Management Menu
- View group details again by going to Manage Group submenu
- Go to Edit User Group
- Edit the details and Save
- Then again Go to User Management Menu
- View group details by going to Manage Group submenu
- Delete the group

Writing automated testing code using Robot Framework:

```
WIS_ManageGroup.robot X
F: > Install > xampp > htdocs > synthia > warehouse-inventory-system-master > Robot > WIS_ManageGroup.robot
1  * Settings *
2  Library SeleniumLibrary
3  Library Collections
4
5  * Variables *
6  ${BROWSER}    chrome
7  ${URL}        http://localhost/warehouse-inventory-system-master/home.php
8  ${ADMIN_USER} admin
9  ${ADMIN_PASS} admin
10
11 * Test Cases *
12 Go to Website
13     [Documentation] This test case verify user is able to open the URL
14     Open Browser    ${URL}    ${Browser}
15     Page Should Contain    Welcome
16
17 Login to your account
18     Input Text    username    admin
19     Input Text    password    admin
20     Click Button    Login
21
22 Go to User Management
23     [Documentation] Verify that an admin can go to User Management
24     Click Element    xpath=//a[@class='submenu-toggle']/span[text()='User Management']
25     Sleep    1 seconds    # Pauses the test execution for 1 seconds
26
27 View Group Details
28     [Documentation] Verify that an admin can view group details
29     Click Link    xpath=//a[text()='Manage Groups']
30     Sleep    1 seconds    # Pauses the test execution for 1 seconds
31
32 Add New Group
33     [Documentation] Verify that an admin can add new group
34     Click Link    xpath=//a[@class='btn btn-info pull-right btn-sm']
35     Page Should Contain    Add new user Group
36     Sleep    1 seconds    # Pauses the test execution for 1 seconds
37
```

```

37
38 Add New User Group
39     Input Text    group-name    Default10
40     Input Text    group-level    10
41     Select From List By Label    xpath=//select[@name='status']    Active
42     Click Button    Update
43     Sleep    1 seconds    # Pauses the test execution for 1 seconds
44
45 Go to User Management Again
46     [Documentation]    Verify that an admin can go to User Management
47     Click Element    xpath=//a[@class='submenu-toggle']/span[text()='User Management']
48     Sleep    1 seconds    # Pauses the test execution for 1 seconds
49
50 View Group Details Again
51     [Documentation]    Verify that an admin can view group details
52     Click Link    xpath=//a[text()='Manage Groups']
53     Sleep    1 seconds    # Pauses the test execution for 1 seconds
54
55 Go to Edit Group
56     [Documentation]    Verify that an admin can edit group
57     Click Element    xpath=//table//tr[4]//td[5]//div[@class='btn-group']/a[@data-original-title='Edit']
58     Page Should Contain    Edit Group
59     Sleep    1 seconds    # Pauses the test execution for 1 seconds
60
61 Edit Existing User Group
62     Input Text    group-name    Default11
63     Input Text    group-level    11
64     Select From List By Label    xpath=//select[@name='status']    Deactive
65     Click Button    Update
66     Sleep    1 seconds    # Pauses the test execution for 1 seconds
67
68 Go to User Management Again From Edit Page
69     [Documentation]    Verify that an admin can go to User Management
70     Click Element    xpath=//a[@class='submenu-toggle']/span[text()='User Management']
71     Sleep    1 seconds    # Pauses the test execution for 1 seconds
72
73 View Group Details Again After Edit Operation
74     [Documentation]    Verify that an admin can view group details
75     Click Link    xpath=//a[text()='Manage Groups']
76     Sleep    1 seconds    # Pauses the test execution for 1 seconds
77
78 Go to Delete Group
79     [Documentation]    Verify that an admin can delete group
80     Click Element    xpath=//table//tr[4]//td[5]//div[@class='btn-group']/a[@data-original-title='Remove']
81     Page Should Contain    Group has been deleted
82     Sleep    1 seconds    # Pauses the test execution for 1 seconds
83

```

Executing robot file:

```

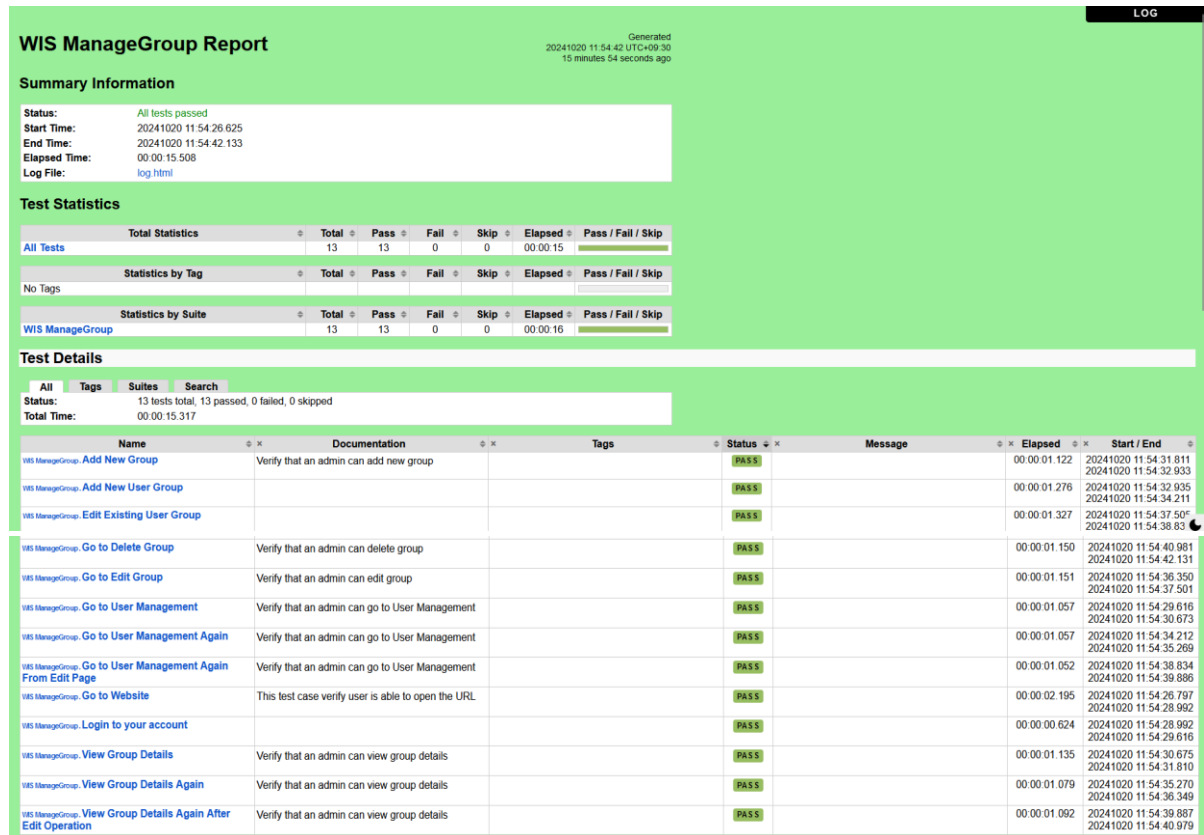
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Report: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_ManageGroup.robot
=====
WIS_ManageGroup
=====
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:59668/devtools/browser/7a5ae320-5392-4497-8a6e-0d81b7269550
Go to Website :: This test case verify user is able to open the URL    | PASS |
-----
Login to your account    | PASS |
-----
Go to User Management :: Verify that an admin can go to User Manag... | PASS |
-----
View Group Details :: Verify that an admin can view group details    | PASS |
-----
Add New Group :: Verify that an admin can add new group    | PASS |
-----
Add New User Group    | PASS |
-----
Go to User Management Again :: Verify that an admin can go to User... | PASS |
-----
View Group Details Again :: Verify that an admin can view group de... | PASS |
-----
Go to Edit Group :: Verify that an admin can edit group    | PASS |
-----
Edit Existing User Group    | PASS |
-----
Go to User Management Again From Edit Page :: Verify that an admin... | PASS |
-----
View Group Details Again After Edit Operation :: Verify that an ad... .Created TensorFlow Lite XNNPACK delegate for CPU.
View Group Details Again After Edit Operation :: Verify that an ad... | PASS |
-----
Go to Delete Group :: Verify that an admin can delete group    | PASS |
-----
WIS_ManageGroup    | PASS |
13 tests, 13 passed, 0 failed
=====
Output: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\output.xml
Log: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\log.html
Report: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html

```

After running the code, we can see that all the test cases passed.

Results and Report:



➤ Manage Users (View, Add, Edit, Delete):- (Synthia Islam)

Allowing the staff/user to manage User Group in Warehouse Inventory System (WIS):

Created 14 test cases for managing group functionalities which include viewing user details, adding new users, editing existing users, changing their password and deleting the users:

- Go to website
- Login with Admin username
- Go to User Management menu
- View user details by going to Manage Users submenu
- Click Add New User button to go to Add New User page
- Add new user by giving input details
- Then again Go to User Management Menu
- View user details again by going to Manage Users submenu
- Go to Edit User
- Edit the user details and update
- Change password and update
- Then again Go to User Management Menu
- View user details again by going to Manage Users submenu
- Delete the user

Writing automated testing code using Robot Framework:

```
WIS_ManageGroup.robot  WIS_ManageUsers.robot X
F:\> Install > xampp > htdocs > synthia > warehouse-inventory-system-master > Robot > WIS_ManageUsers.robot
1  * Settings *
2  Library SeleniumLibrary
3  Library Collections
4
5  * Variables *
6  ${BROWSER}    chrome
7  ${URL}         http://localhost/warehouse-inventory-system-master/home.php
8  ${ADMIN_USER}  admin
9  ${ADMIN_PASS}  admin
10
11 * Test Cases *
12 Go to Website
13     [Documentation] This test case verify user is able to open the URL
14     Open Browser  ${URL}  ${Browser}
15     Page Should Contain  Welcome
16
17 Login to your account
18     Input Text    username    admin
19     Input Text    password    admin
20     Click Button  Login
21
22 Go to User Management
23     [Documentation] Verify that an admin can go to User Management
24     Click Element xpath=//a[@class='submenu-toggle']/span[text()='User Management']
25     Sleep 1 seconds # Pauses the test execution for 1 seconds
26
27 View User Details
28     [Documentation] Verify that an admin can view user details
29     Click Link    xpath=//a[text()='Manage Users']
30     Sleep 1 seconds # Pauses the test execution for 1 seconds
31
32 Go to Add New User
33     [Documentation] Verify that an admin can add new user
34     Click Link    xpath=//a[@class='btn btn-info pull-right' and text()='Add New User']
35     Sleep 1 seconds # Pauses the test execution for 1 seconds
36
```

```
37 Add New User
38     Input Text    full-name    TestUser
39     Input Text    username    testuser
40     Input Text    password    1234
41     Select From List By Label  xpath=//select[@name='level']  User
42     Click Button  Add User
43     Sleep 1 seconds # Pauses the test execution for 1 seconds
44
45 Go to User Management Again
46     [Documentation] Verify that an admin can go to User Management
47     Click Element xpath=//a[@class='submenu-toggle']/span[text()='User Management']
48     Sleep 1 seconds # Pauses the test execution for 1 seconds
49
50 View User Details Again
51     [Documentation] Verify that an admin can view group details
52     Click Link    xpath=//a[text()='Manage Users']
53     Sleep 1 seconds # Pauses the test execution for 1 seconds
54
55 Go to Edit User
56     [Documentation] Verify that an admin can edit user
57     Click Element xpath=//table//tr[4]//td[7]//div[@class='btn-group']/a[@data-original-title='Edit']
58     Page Should Contain  Edit User
59     Sleep 1 seconds # Pauses the test execution for 1 seconds
60
61 Edit Existing User
62     Input Text    name    Test User
63     Input Text    username    testuser
64     Select From List By Label  xpath=//select[@name='level']  User
65     Select From List By Label  xpath=//select[@name='status']  Deactive
66     Sleep 1 seconds # Pauses the test execution for 1 seconds
67     Click Button  Update
68     Sleep 1 seconds # Pauses the test execution for 1 seconds
69
```

```

70 Edit Existing User Password
71 Input Text password 12345
72 Click Button Change
73 Sleep 1 seconds # Pauses the test execution for 1 seconds
74
75 Go to User Management Again From Edit Page
76 [Documentation] Verify that an admin can go to User Management
77 Click Element xpath=//a[@class='submenu-toggle']//span[text()='User Management']
78 Sleep 1 seconds # Pauses the test execution for 1 seconds
79
80 View User Details Again After Edit Operation
81 [Documentation] Verify that an admin can view group details
82 Click Link xpath=//a[text()='Manage Users']
83 Sleep 1 seconds # Pauses the test execution for 1 seconds
84
85 Delete User
86 [Documentation] Verify that an admin can delete user
87 Click Element xpath=//table//tr[4]//td[7]//div[@class='btn-group']//a[@data-original-title='Remove']
88 Sleep 1 seconds # Pauses the test execution for 1 seconds
89

```

Executing robot file:

```

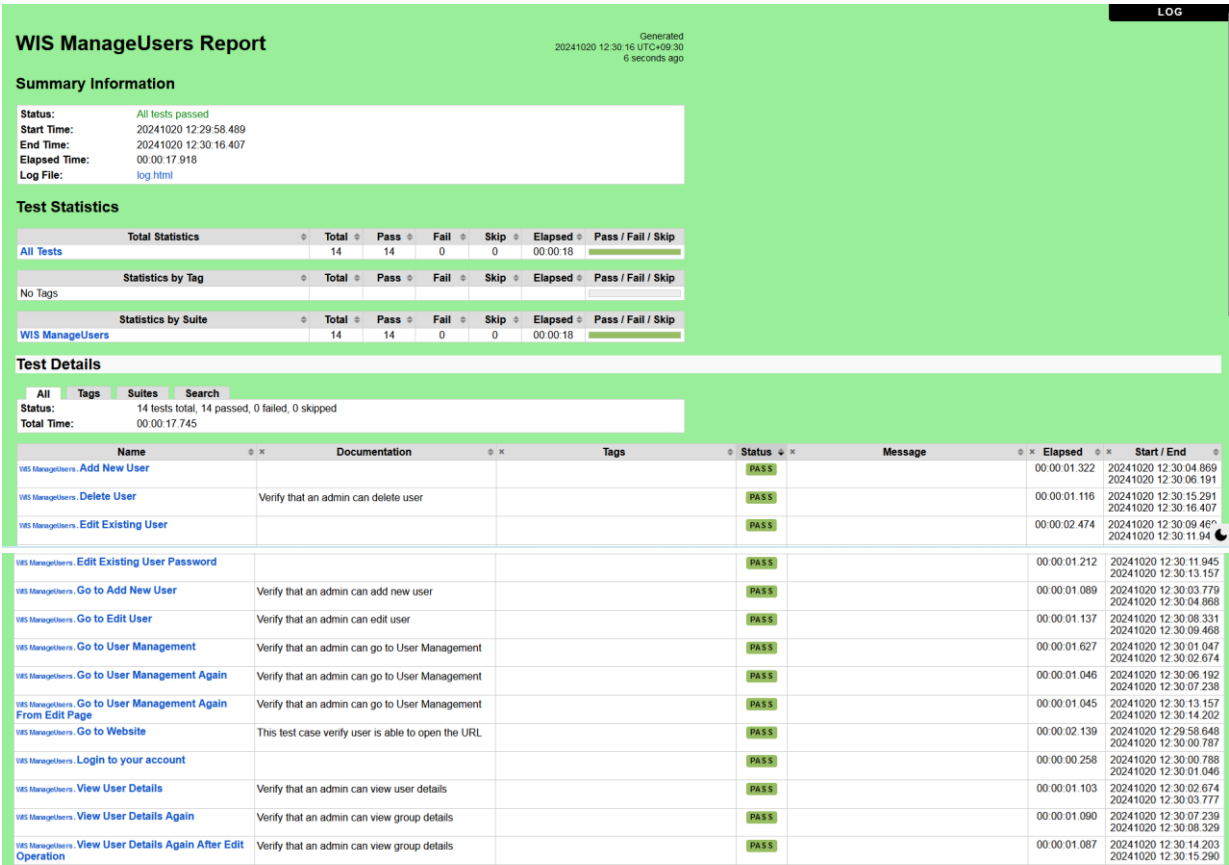
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_ManageUsers.robot

=====
WIS_ManageUsers
=====
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:59601/devtools/browser/5c00560e-7f06-4da7-88b2-e271b4606572
Go to Website :: This test case verify user is able to open the URL | PASS |
-----
Login to your account | PASS |
-----
Go to User Management :: Verify that an admin can go to User Manag... | PASS |
-----
View User Details :: Verify that an admin can view user details | PASS |
-----
Go to Add New User :: Verify that an admin can add new user | PASS |
-----
Add New User | PASS |
-----
Go to User Management Again :: Verify that an admin can go to User... | PASS |
-----
View User Details Again :: Verify that an admin can view group det... | PASS |
-----
Go to Edit User :: Verify that an admin can edit user | PASS |
-----
Edit Existing User | PASS |
-----
Edit Existing User Password | PASS |
-----
Go to User Management Again From Edit Page :: Verify that an admin... .Created TensorFlow Lite XNNPACK delegate for CPU.
Go to User Management Again From Edit Page :: Verify that an admin... | PASS |
-----
View User Details Again After Edit Operation :: Verify that an adm... | PASS |
-----
Delete User :: Verify that an admin can delete user | PASS |
-----
WIS_ManageUsers | PASS |
=====
14 tests, 14 passed, 0 failed
=====
Output: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\output.xml
Log: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\log.html
Report: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html

```

After running the code, we can see that all the test cases passed.

Results and Report:



Category Module: - (Synthia Islam)

Allowing the staff/user to manage Categories in the Warehouse Inventory System (WIS):

Created 7 test cases for managing categories functionalities, which include viewing category details, adding new categories, editing existing categories, and removing the categories:

- Go to website
- Login with Admin username
- Go to Categories menu
- Add new category
- Go to Edit Category page from the table
- Edit existing category by giving input details and update
- Remove Category

Writing automated testing code using Robot Framework:

```
WIS_Categories.robot X
F: > Install > xampp > htdocs > synthia > warehouse-inventory-system-master > Robot > WIS_Categories.robot
1  * Settings *
2  Library SeleniumLibrary
3  Library Collections
4
5  * Variables *
6  ${BROWSER}    chrome
7  ${URL}        http://localhost/warehouse-inventory-system-master/home.php
8  ${ADMIN_USER} admin
9  ${ADMIN_PASS} admin
10
11 * Test Cases *
12 Go to Website
13     [Documentation] This test case verify user is able to open the URL
14     Open Browser  ${URL}  ${Browser}
15     Page Should Contain  Welcome
16
17 Login to your account
18     Input Text    username    admin
19     Input Text    password    admin
20     Click Button  Login
21
22 Go to Categories
23     [Documentation] Verify that an admin can go to Categories
24     Click Element  xpath=//span[text()='Categories']
25     Sleep  1 seconds # Pauses the test execution for 1 seconds
26
27 Add New Category
28     Input Text    categorie-name    DefaultCategory
29     Click Button  Add categorie
30     Sleep  1 seconds # Pauses the test execution for 1 seconds
31
32 Go to Edit Category
33     [Documentation] Verify that an admin can edit Category
34     Click Element  xpath=//table//tr[1]//td[3]//div[@class='btn-group']/a[@data-original-title='Edit']
35     Sleep  1 seconds # Pauses the test execution for 1 seconds
36
37 Edit Category
38     Input Text    categorie-name    Default Category
39     Click Button  Update categorie
40     Sleep  1 seconds # Pauses the test execution for 1 seconds
41
42 Remove Category
43     [Documentation] Verify that an admin can edit Category
44     Click Element  xpath=//table//tr[1]//td[3]//div[@class='btn-group']/a[@data-original-title='Remove']
45     Sleep  1 seconds # Pauses the test execution for 1 seconds
46
```

Executing robot file:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_ManageUsers.robot

14 tests, 14 passed, 0 failed
=====
Output:  F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\output.xml
Log:     F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\log.html
Report:  F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_Categories.robot
=====
WIS Categories
=====
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:60115/devtools/browser/a94fcbbe-3db9-404a-b8ce-9308d3725fa6
Go to Website :: This test case verify user is able to open the URL      | PASS |
-----
Login to your account                                                    | PASS |
-----
Go to Categories :: Verify that an admin can go to Categories            | PASS |
-----
Add New Category                                                         | PASS |
-----
Go to Edit Category :: Verify that an admin can edit Category            | PASS |
-----
Edit Category                                                            | PASS |
-----
Remove Category :: Verify that an admin can edit Category                | PASS |
-----
WIS Categories                                                           | PASS |
7 tests, 7 passed, 0 failed
=====
Output:  F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\output.xml
Log:     F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\log.html
Report:  F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> 
```

After running the code, we can see that all the test cases passed.

Results and Report:

Generated
20241020 12:52:45 UTC+09:30
28 seconds ago

LOG

WIS Categories Report

Summary Information

Status: All tests passed

Start Time: 20241020 12:52:36.457

End Time: 20241020 12:52:45.926

Elapsed Time: 00:00:09.469

Log File: [log.html](#)

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	7	7	0	0	00:00:09	

Statistics by Tag

No Tags

Statistics by Suite

WIS Categories

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
7	7	0	0	00:00:09	

Test Details

All Tags Suites Search

Status: 7 tests total, 7 passed, 0 failed, 0 skipped

Total Time: 00:00:09.250

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS Categories - Add New Category			PASS		00:00:01.195	20241020 12:52:41.231 20241020 12:52:42.426
WIS Categories - Edit Category			PASS		00:00:01.197	20241020 12:52:43.563 20241020 12:52:44.760
WIS Categories - Go to Categories	Verify that an admin can go to Categories		PASS		00:00:01.366	20241020 12:52:39.987 20241020 12:52:41.223
WIS Categories - Go to Edit Category	Verify that an admin can edit Category		PASS		00:00:01.133	20241020 12:52:42.428 20241020 12:52:43.561
WIS Categories - Go to Website	This test case verify user is able to open the URL		PASS		00:00:02.924	20241020 12:52:36.663 20241020 12:52:39.587
WIS Categories - Login to your account			PASS		00:00:00.278	20241020 12:52:39.586 20241020 12:52:39.864
WIS Categories - Remove Category	Verify that an admin can edit Category		PASS		00:00:01.163	20241020 12:52:44.761 20241020 12:52:45.924

▪ Sales Module: - (Hussein Salami)

Allowing the staff/user to manage Sales in Warehouse Inventory System (WIS):

Created 8 test cases for managing categories functionalities which include viewing category details, adding new categories, editing existing categories, and removing the categories:

- Go to website
- Login with Admin username
- Go to Sales menu
- View sales details by going to Manage Sales submenu
- Add new sale
- Find sale product
- Go to Sales menu again
- Go to Add Sale submenu

Writing automated testing code using Robot Framework:

```
WIS_Sales.robot X
F: > Install > xampp > htdocs > synthia > warehouse-inventory-system-master > Robot > WIS_Sales.robot
1  * Settings *
2  Library SeleniumLibrary
3  Library Collections
4
5  * Variables *
6  ${BROWSER}    chrome
7  ${URL}         http://localhost/warehouse-inventory-system-master/home.php
8  ${ADMIN_USER}  admin
9  ${ADMIN_PASS}  admin
10
11 * Test Cases *
12 Go to Website
13     [Documentation] This test case verify user is able to open the URL
14     Open Browser    ${URL}    ${Browser}
15     Page Should Contain    Welcome
16
17 Login to your account
18     Input Text    username    admin
19     Input Text    password    admin
20     Click Button  Login
21
22 Go to Sales
23     [Documentation] Verify that an admin can go to Sales
24     Click Element  xpath=//a[@class='submenu-toggle']//span[text()='Sales']
25     Sleep 1 seconds # Pauses the test execution for 1 seconds
26
27 View Sales Details
28     [Documentation] Verify that an admin can view Sales details
29     Click Link     xpath=//a[text()='Manage Sales']
30     Sleep 1 seconds # Pauses the test execution for 1 seconds
31
32 Add New Sale
33     [Documentation] Verify that an admin can add new Sale
34     Click Link     xpath=//a[@class='btn btn-primary']
35     Page Should Contain    Sale Eidt
36     Sleep 1 seconds # Pauses the test execution for 1 seconds
37
```

```

38 Find New Sale Product
39     [Documentation] Verify that an admin can find sale product
40     Input Text    title    Product1
41     Click Button  Find It
42     Sleep 1 seconds # Pauses the test execution for 1 seconds
43
44 Go to Sales Again
45     [Documentation] Verify that an admin can go to Sales
46     Click Element xpath=//a[@class='submenu-toggle']/span[text()='Sales']
47     Sleep 1 seconds # Pauses the test execution for 1 seconds
48
49 Go to Add Sale
50     [Documentation] Verify that an admin can go to Add Sale
51     Click Link xpath=//a[text()='Add Sale']
52     Sleep 1 seconds # Pauses the test execution for 1 seconds
53

```

Executing robot file:

```

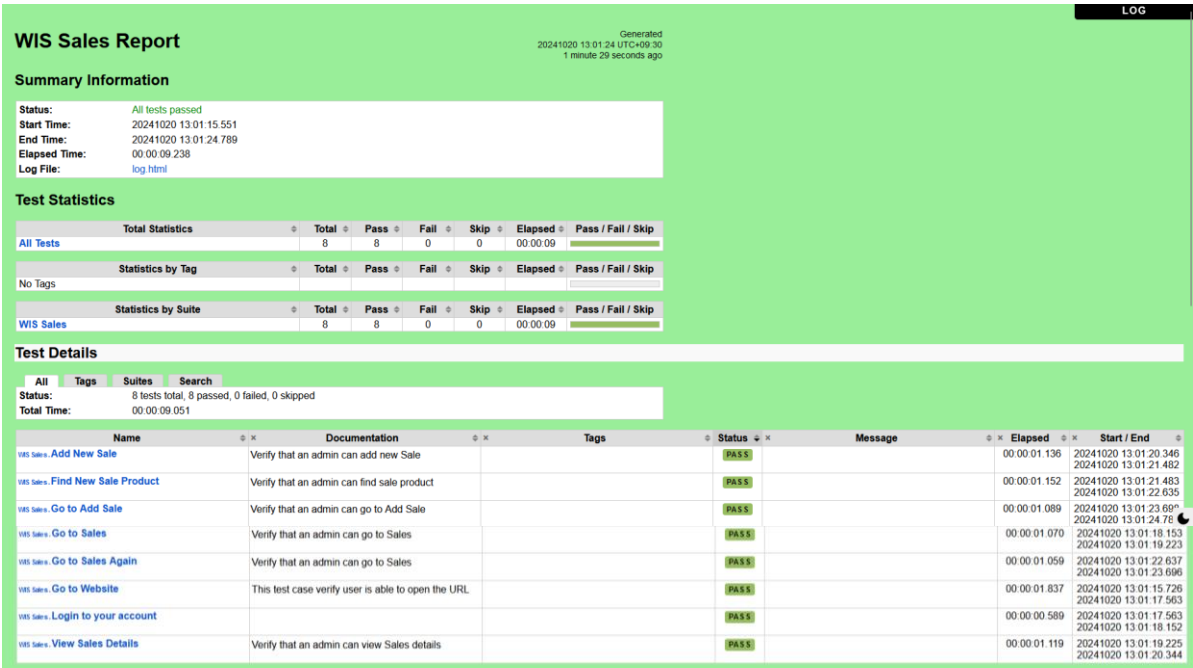
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\synth> cd F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_Sales.robot
=====
WIS Sales
=====
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:60255/devtools/browser/497a5560-a930-486c-8d0c-e5e696c3b9f5
Go to Website :: This test case verify user is able to open the URL | PASS |
-----
Login to your account | PASS |
-----
Go to Sales :: Verify that an admin can go to Sales | PASS |
-----
View Sales Details :: Verify that an admin can view Sales details | PASS |
-----
Add New Sale :: Verify that an admin can add new Sale | PASS |
-----
Find New Sale Product :: Verify that an admin can find sale product | PASS |
-----
Go to Sales Again :: Verify that an admin can go to Sales | PASS |
-----
Go to Add Sale :: Verify that an admin can go to Add Sale | PASS |
-----
WIS Sales | PASS |
8 tests, 8 passed, 0 failed
=====
Output: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\output.xml
Log: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\log.html
Report: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> 

```

After running the code, we can see that all the test cases passed.

Results and Report:



Sales Report Module: - (Hussein Salami, Synthia Islam)

Allowing the staff/user to generate and show sales report in the Warehouse Inventory System (WIS):

Created 10 test cases for generating and showing sales report functionalities, including daily report, monthly report, and reports between two specific dates:

- Go to website
- Login with Admin username
- Go to Sales Report menu
- Go to Sales By Dates submenu
- Generate report by selecting two dates
- Go Back to Previous Page from Report
- Go to Sales Report page again from sales by date page
- Go to Monthly sales submenu
- Go to Sales Report page again
- Go to Daily Sales submenu

Writing automated testing code using Robot Framework:

```
WIS_SalesReport.robot X
F:\> Install > xampp > htdocs > synthia > warehouse-inventory-system-master > Robot > WIS_SalesReport.robot

12  * Test Cases *
13  Go to Website
14      [Documentation] This test case verify user is able to open the URL
15      Open Browser    ${URL}    ${Browser}
16      Page Should Contain    Welcome
17
18  Login to your account
19      Input Text    username    admin
20      Input Text    password    admin
21      Click Button    Login
22
23  Go to Sales Report
24      [Documentation] Verify that an admin can go to Sales
25      Click Element    xpath=//a[@class='submenu-toggle']/span[text()='Sales Report']
26      Sleep    1 seconds    # Pauses the test execution for 1 seconds
27
28
29  View Date Range
30      [Documentation] Verify that an admin can view Sales details
31      Click Link    xpath=//a[text()='Sales by dates ']
32      Sleep    1 seconds    # Pauses the test execution for 1 seconds
33
34
35  Generate Sales by Dates
36      Input Text    start-date    2024-10-01
37      Click Element    xpath=//td[text()='1']    # Select the 1st day
38      Wait Until Element Is Not Visible    xpath=//div[@class='datepicker-days']    # Ensure the start calendar disappears
39      Input Text    start-date    2024-10-01
40      Input Text    end-date    2024-10-02
41      Click Element    xpath=//td[text()='2']    # Select the 2nd day
42      Wait Until Element Is Not Visible    xpath=//div[@class='datepicker-days']    # Ensure the end calendar disappears
43      Input Text    end-date    2024-10-02
44      Click Button    Generate Report
45      Sleep    3 seconds    # Pauses the test execution for 1 seconds
46
47  Go Back to Previous Page from Report
48      Go Back

50  Go to Sales Report for Date Range
51      [Documentation] Verify that an admin can go to Sales
52      Click Element    xpath=//a[@class='submenu-toggle']/span[text()='Sales Report']
53      Sleep    1 seconds    # Pauses the test execution for 1 seconds
54
55
56  View Monthly sales
57      [Documentation] Verify that an admin can view Monthly Sales details
58      Click Link    xpath=//a[text()='Monthly sales']
59      Sleep    1 seconds    # Pauses the test execution for 1 seconds
60
61
62  Go to Sales Report for Daily Sales
63      [Documentation] Verify that an admin can go to Sales
64      Click Element    xpath=//a[@class='submenu-toggle']/span[text()='Sales Report']
65      Sleep    1 seconds    # Pauses the test execution for 1 seconds
66
67
68  View Daily sales
69      [Documentation] Verify that an admin can view Daily Sales details
70      Click Link    xpath=//a[text()='Daily sales']
71      Sleep    1 seconds    # Pauses the test execution for 1 seconds
```

Executing robot file:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_SalesReport.robot

=====
WIS SalesReport
=====

Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:55867/devtools/browser/2be45f0c-dd97-416c-a70b-34a440d3422d
Go to Website :: This test case verify user is able to open the URL   | PASS |

-----

Login to your account                                              | PASS |

-----

Go to Sales Report :: Verify that an admin can go to Sales        | PASS |

-----

View Date Range :: Verify that an admin can view Sales details    | PASS |

-----

Generate Sales by Dates                                           .
Generate Sales by Dates                                           | PASS |

-----

Go Back to Previous Page from Report                             | PASS |

-----

Go to Sales Report for Date Range :: Verify that an admin can go t... | PASS |

-----

View Monthly sales :: Verify that an admin can view Monthly Sales ... | PASS |

-----

Go to Sales Report for Daily Sales :: Verify that an admin can go ... | PASS |

-----

View Daily sales :: Verify that an admin can view Daily Sales details | PASS |

-----

WIS SalesReport                                                  | PASS |

10 tests, 10 passed, 0 failed

=====

Output:  F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\output.xml
Log:      F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\log.html
Report:   F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot>
```

After running the code, we can see that all the test cases passed.

Results and Report:

LOG

WIS SalesReport Report

Generated 20241020 14:20:05 UTC+09:30 25 seconds ago

Summary Information

Status: All tests passed

Start Time: 20241020 14:19:52.418

End Time: 20241020 14:20:05.104

Elapsed Time: 00:00:12.686

Log File: log.html

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	10	10	0	0	00:00:13	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
WIS SalesReport	10	10	0	0	00:00:13	

Test Details

All Tags Suites Search

Status: 10 tests total, 10 passed, 0 failed, 0 skipped

Total Time: 00:00:12.523

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS SalesReport: Generate Sales by Dates			PASS		00:00:03.613	20241020 14:19:57.080 20241020 14:20:00.693
WIS SalesReport: Go Back to Previous Page from Report			PASS		00:00:00.046	20241020 14:20:00.693 20241020 14:20:00.739
WIS SalesReport: Go to Sales Report	Verify that an admin can go to Sales		PASS		00:00:01.066	20241020 14:19:54.914* 20241020 14:19:55.984
WIS SalesReport: Go to Sales Report for Daily Sales	Verify that an admin can go to Sales		PASS		00:00:01.051	20241020 14:20:02.926 20241020 14:20:03.971
WIS SalesReport: Go to Sales Report for Date Range	Verify that an admin can go to Sales		PASS		00:00:01.050	20241020 14:20:00.739 20241020 14:20:01.789
WIS SalesReport: Go to Website	This test case verify user is able to open the URL.		PASS		00:00:01.884	20241020 14:19:52.580 20241020 14:19:54.464
WIS SalesReport: Login to your account			PASS		00:00:00.450	20241020 14:19:54.465 20241020 14:19:54.915
WIS SalesReport: View Daily sales	Verify that an admin can view Daily Sales details		PASS		00:00:01.133	20241020 14:20:03.971 20241020 14:20:05.104
WIS SalesReport: View Date Range	Verify that an admin can view Sales details		PASS		00:00:01.098	20241020 14:19:55.982 20241020 14:19:57.080
WIS SalesReport: View Monthly sales	Verify that an admin can view Monthly Sales details		PASS		00:00:01.132	20241020 14:20:01.788 20241020 14:20:02.926

■ Media Module: - (Musrat Jahan)

Allowing the staff/user to explore medias in Warehouse Inventory System (WIS):

Created 5 test cases:

- Go to website
- Login with Admin username
- Go to Medias
- Upload Image
- Delete Upload Image

Writing automated testing code using Robot Framework:

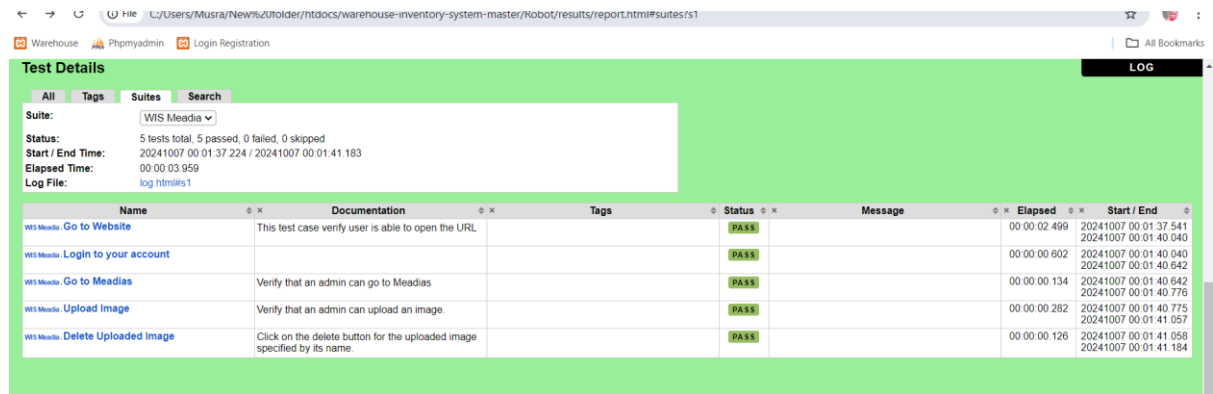
```

> Users > Musra > New folder > htdocs > warehouse-inventory-system-master > Robot > WIS_Media.robot > ...
Run Suite | Debug Suite | Load in Interactive Console
1  * Settings *
2  Library SeleniumLibrary
3  Library Collections
4
Load in Interactive Console
5  * Variables *
6  ${BROWSER}    chrome
7  ${URL}        http://localhost/warehouse-inventory-system-master/home.php
8  ${ADMIN_USER} admin
9  ${ADMIN_PASS} admin
10 ${FILE_INPUT_XPATH} //input[@type='file' and @name='file_upload']
11 ${UPLOAD_BUTTON_XPATH} //button[@type='submit' and @name='submit']
12 ${IMAGE_PATH}  C:/Users/Musra/OneDrive/Pictures/Book.png # Update this with the correct image path
13 ${image_name}  Book.png
14
15 * Test Cases *
Run | Debug | Run in Interactive Console
16 Go to Website
17 [Documentation] This test case verify user is able to open the URL
18 Open Browser  ${URL}  ${Browser}
19 Page Should Contain  Welcome
20
Run | Debug | Run in Interactive Console
21 Login to your account
22 Input Text      username  admin
23 Input Text      password  admin
24 Click Button    Login
25
Run | Debug | Run in Interactive Console
26 Go to Meadias
27 [Documentation] Verify that an admin can go to Meadias
28 Click Element   xpath=//div[@class='sidebar']/ul/li[5]//a[1]
29 Page Should Contain  All Photos
30
Run | Debug | Run in Interactive Console
31 Upload Image
32 [Documentation] Verify that an admin can upload an image.
33 # Wait for the file upload element to be visible
34 Wait Until Element Is Visible  xpath=//input[@type='file' and @name='file_upload']
35 Choose File  xpath=//input[@type='file' and @name='file_upload']  C:\\Users\\Musra\\OneDrive\\Pictures\\Book.png
36 Click Button  xpath=//button[@type='submit' and @name='submit']
37 Capture Page Screenshot  ${CURDIR}/Upload_Image_Screenshot.png # Adjust the path as needed
38
Run | Debug | Run in Interactive Console
40 Delete Uploaded Image
41 [Documentation] Click on the delete button for the uploaded image specified by its name.
42 ${image_xpath} Set Variable  //tr[td[contains(text(), '${image_name}')]//a[contains(@href, 'delete_media.php')]
43 Click Element  xpath=${image_xpath}
44 Page Should Contain  Photo has been deleted.
45
```

Executing robot file:

```
Log: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\log.html
Report: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\report.html
PS C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot> robot -d results WIS_Media.robot
=====
WIS Media
=====
[ WARN ] The chromedriver version (128.0.6613.119) detected in PATH at C:\Python310\Scripts\chromedriver-win64\chromedriver.exe might not be compatible with the detected chrome version (129.0.6668.90); currentl
y, chromedriver 129.0.6668.89 is recommended for chrome 129.*, so it is advised to delete the driver in PATH and retry
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:61394/devtools/browser/84280153-b076-420c-adab-d4253f3c4aaf
Go to Website :: This test case verify user is able to open the URL | PASS |
-----
Login to your account | PASS |
-----
Go to Meadias :: Verify that an admin can go to Meadias | PASS |
-----
Upload Image :: Verify that an admin can upload an image. | PASS |
-----
Delete Uploaded Image :: Click on the delete button for the upload... | PASS |
-----
WIS Media | PASS |
5 tests, 5 passed, 0 failed
=====
Output: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\output.xml
Log: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\log.html
Report: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\report.html
PS C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot>
```


Results and Report:



The screenshot shows a web browser window with the URL `C:/Users/Musra/New%20folder/htdocs/warehouse-inventory-system-master/Robot/results/report.html#suites/st`. The page title is 'Warehouse' and the user is logged in as 'Phipmyadmin'. The 'Test Details' section shows the suite 'WIS Meadia' with a status of '5 tests total, 5 passed, 0 failed, 0 skipped'. The start/end time is '20241007 00:01:37.224 / 20241007 00:01:41.183', the elapsed time is '00:00:03.959', and the log file is 'log.html#st1'. Below this is a table of test results.

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS Meadia .Go to Website	This test case verify user is able to open the URL		PASS		00:00:02.499	20241007 00:01:37.541 20241007 00:01:40.040
WIS Meadia .Login to your account			PASS		00:00:00.602	20241007 00:01:40.040 20241007 00:01:40.642
WIS Meadia .Go to Meadias	Verify that an admin can go to Meadias		PASS		00:00:00.134	20241007 00:01:40.642 20241007 00:01:40.776
WIS Meadia .Upload Image	Verify that an admin can upload an image.		PASS		00:00:00.282	20241007 00:01:40.775 20241007 00:01:41.057
WIS Meadia .Delete Uploaded Image	Click on the delete button for the uploaded image specified by its name.		PASS		00:00:00.126	20241007 00:01:41.058 20241007 00:01:41.184

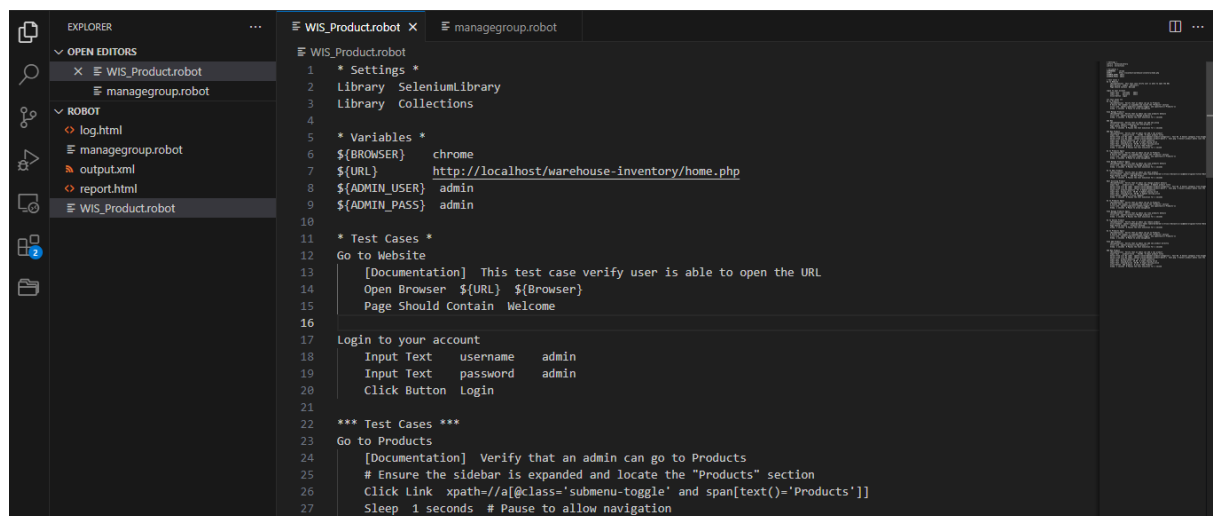
Product Module: - (Muhammad Asad Nawaz)

Allowing the staff/user to go to the Profile Settings dropdown from the top in Warehouse Inventory System (WIS):

Created 12 test cases:

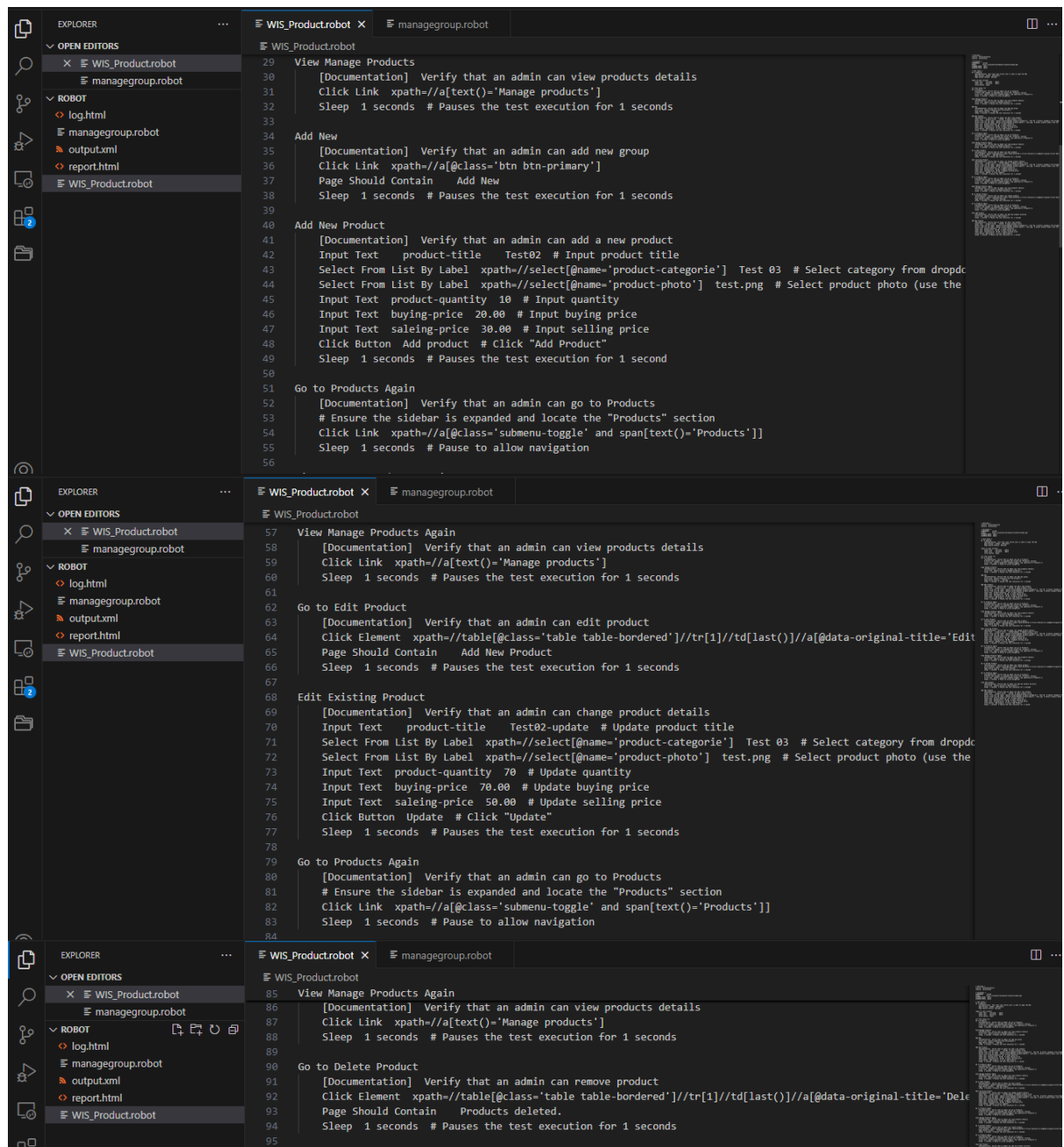
- Go to website
- Login with Admin username
- Go to Product Menu
- Go to Manage Product submenu
- Add new product
- Again go to Product Menu
- Again Go to Manage Product submenu
- Go to Edit Product
- Edit and update existing product
- Again go to Product Menu
- Again Go to Manage Product submenu
- Delete Product

Writing automated testing code using Robot Framework:

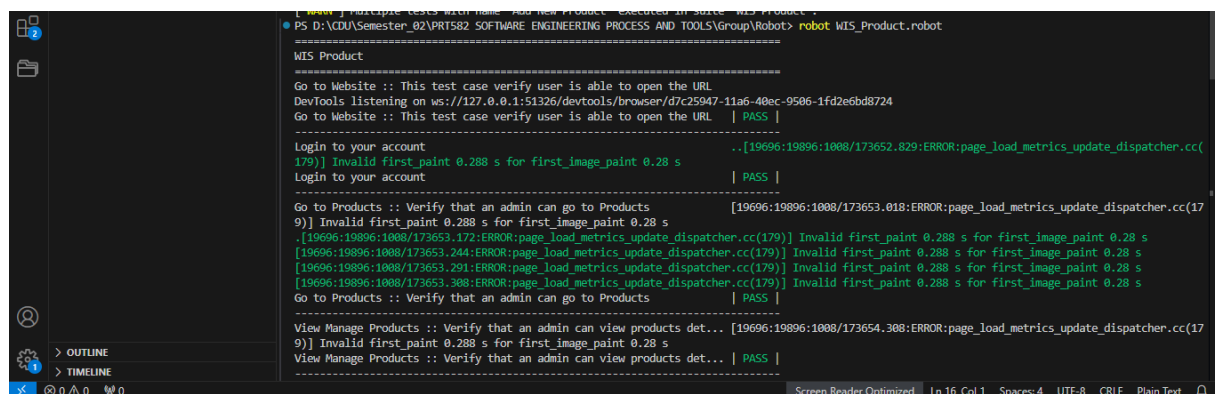


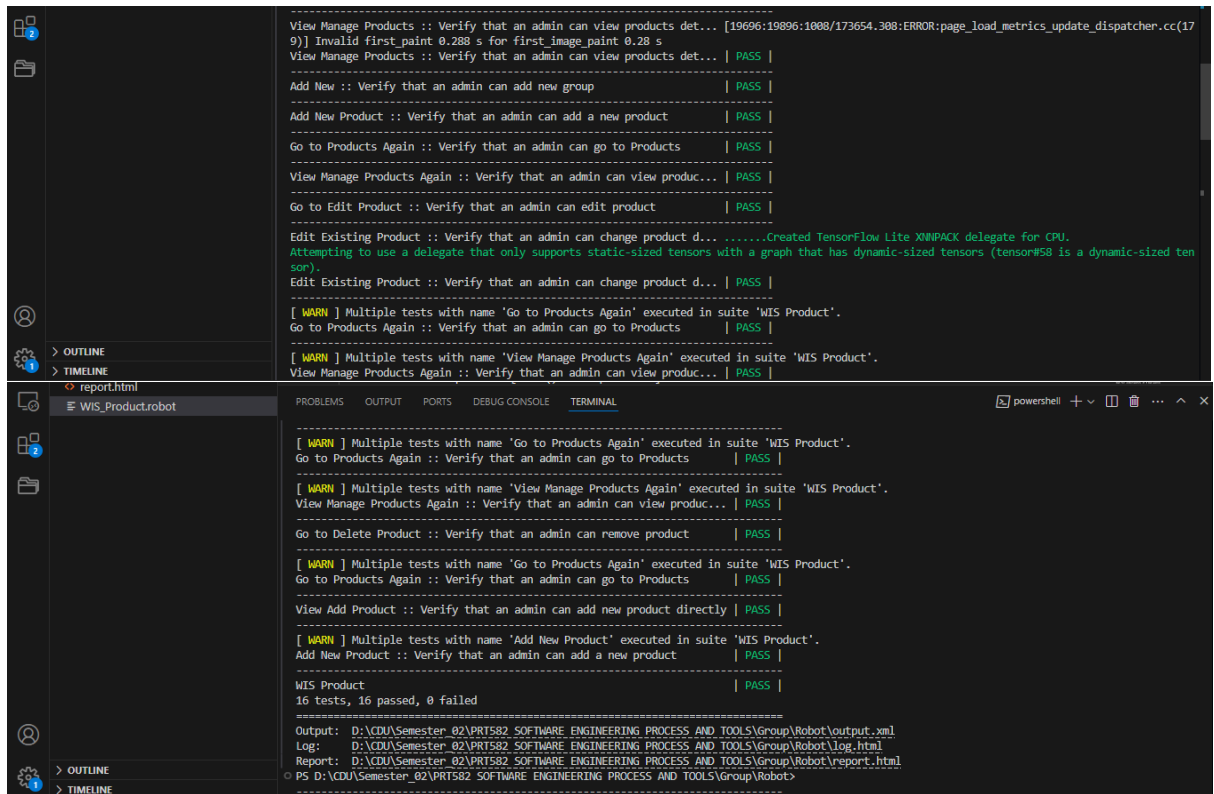
The screenshot shows an IDE with the Explorer on the left and the Editor on the right. The Explorer shows a project structure with 'WIS_Product.robot' selected. The Editor shows the code for 'WIS_Product.robot'.

```
1 * Settings *
2 Library SeleniumLibrary
3 Library Collections
4
5 * Variables *
6 ${BROWSER} chrome
7 ${URL} http://localhost/warehouse-inventory/home.php
8 ${ADMIN_USER} admin
9 ${ADMIN_PASS} admin
10
11 * Test Cases *
12 Go to Website
13 [Documentation] This test case verify user is able to open the URL
14 Open Browser ${URL} ${Browser}
15 Page Should Contain Welcome
16
17 Login to your account
18 Input Text username admin
19 Input Text password admin
20 Click Button Login
21
22 *** Test Cases ***
23 Go to Products
24 [Documentation] Verify that an admin can go to Products
25 # Ensure the sidebar is expanded and locate the "Products" section
26 Click Link xpath=//a[@class='submenu-toggle' and span[text()='Products']]
27 Sleep 1 seconds # Pause to allow navigation
```



Executing robot file:





Report:

← → ↺ 🏠

FileD:\CDU\Semester_02\PRTS82 SOFTWARE ENGINEERING PROCESS AND TOOLS\Group\Robot/re...🔍

🌐 ⚙️ 🔄 📄 📁 📌 ⬇️ 🌐 ⋮

LOG

WIS Product Report

Generated20241008 21:57:17 UTC+09:30
0 seconds ago

Summary Information

Status:

All tests passed

Start Time:

20241008 21:56:48.284

End Time:

20241008 21:57:17.676

Elapsed Time:

00:00:29.392

Log File:

[log.html](#)

Test Statistics

Total Statistics

Total

16

Pass

16

Fail

0

Skip

0

Elapsed

00:00:29

Pass / Fail / Skip

All Tests

Statistics by Tag

Total

Pass

Fail

Skip

Elapsed

Pass / Fail / Skip

No Tags

Statistics by Suite

Total

16

Pass

16

Fail

0

Skip

0

Elapsed

00:00:29

Pass / Fail / Skip

WIS Product

Test Details

AllTagsSuitesSearch

Suite:

Test:

Include:

Exclude:

SearchClearHelp

▪ Profile Settings Module: - (Muhammad Asad Nawaz, Synthia Islam)

Allowing the staff/user to go to the Profile Settings dropdown from the top in Warehouse Inventory System (WIS):

Created 9 test cases:

- Go to website
- Login with Admin username
- Click User Dropdown
- Go to Profile section
- Go to Edit Profile
- Upload image
- Update user Information
- Go to Change password
- Update password

Writing automated testing code using Robot Framework:

[We need to make sure to set the correct file path by giving “\\” in the path. Otherwise, it will give an error in Windows PC – (Synthia Islam)]

```
WIS_Setting.robot X
F: > Install > xampp >htdocs > synthia > warehouse-inventory-system-master > Robot > WIS_Setting.robot
7  ${URL}      http://localhost/warehouse-inventory-system-master/index.php
8  ${ADMIN_USER}  admin
9  ${ADMIN_PASS}  admin
10 ${FILE_INPUT_XPATH} //input[@type='file' and @name='file_upload']
11 ${UPLOAD_BUTTON_XPATH} //button[@type='submit' and @name='submit']
12 ${IMAGE_PATH} C:\\Users\\synth\\OneDrive\\Pictures\\1.png # Update this with the correct image path
13 ${image_name} Upload_Image_Screenshot.png
14
15 * Test Cases *
16 Go to Website
17     [Documentation] This test case verify user is able to open the URL
18     Open Browser  ${URL}  ${Browser}
19     Page Should Contain  Welcome
20
21 Login to your account
22     Input Text    username    admin
23     Input Text    password    admin
24     Click Button  Login
25
26 Click Admin User Dropdown
27     [Documentation] Click on the Admin User dropdown to test profile.
28     Click Element  //a[@class='toggle' and contains(span, 'Admin User')]
29
30
31 Click Profile Icon
32     [Documentation] Verify that an admin can open the Profile page
33     Click Element  xpath=//ul[@class='dropdown-menu']/a[@href='profile.php?id=1'] # Click on the Profile link
34     Sleep 1 second # Pause for a moment after navigation
35
36 Click Edit Admin User
37     [Documentation] Verify that an admin can open the Profile page
38     Click Element  xpath=//ul[@class='nav nav-pills nav-stacked']/a[@href='edit_account.php'] # Click on the Profile link
39     Sleep 1 second # Pause for a moment after navigation
40
```

```

41
42 Click Upload Image
43 [Documentation] Verify that an admin can upload an image.
44 # Wait for the file upload element to be visible
45 Wait Until Element Is Visible xpath=//input[@type='file' and @name='file_upload']
46 Choose File xpath=//input[@type='file' and @name='file_upload'] ${IMAGE_PATH}
47 Click Button xpath=//button[@type='submit' and @name='submit']
48 Capture Page Screenshot ${CURDIR}/${image_name} # Adjust the path as needed
49
50 Click Update Admin User Information
51 [Documentation] Verify that an admin can update user details.
52 Input Text name Admin User
53 Input Text username admin
54 Click Button update
55 Sleep 1 seconds # Pauses the test execution for 1 seconds
56
57 Click Change password
58 [Documentation] Verify that an admin can change the password
59 Click Element xpath=//a[@href='change_password.php' and @class='btn btn-danger pull-right'] # Click on the Profile link
60 Sleep 1 second # Pause for a moment after navigation
61
62
63 Click Update Password
64 [Documentation] Verify that an admin can change password.
65 Input password new-password Admin
66 Input password old-password admin
67 Click Button update
68 Sleep 1 seconds # Pauses the test execution for 1 seconds
69

```

Executing robot file:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_Setting.robot
=====
WIS Setting
=====
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:56002/devtools/browser/5dbf8dd9-b236-4194-9fda-0a3e40726f30
Go to Website :: This test case verify user is able to open the URL | PASS |
-----
Login to your account | PASS |
-----
Click Admin User Dropdown :: Click on the Admin User dropdown to t... | PASS |
-----
Click Profile Icon :: Verify that an admin can open the Profile page | PASS |
-----
Click Edit Admin User :: Verify that an admin can open the Profile... | PASS |
-----
Click Upload Image :: Verify that an admin can upload an image. | FAIL |
InvalidArgumentException: Message: invalid argument: File not found : C:\Users\Acer\Pictures\Screenshots est.png
(Session info: chrome=129.0.6668.101)
Stacktrace:
  GetHandleVerifier [0x00007FF68158B095+29557]
  (No symbol) [0x00007FF6814FFA50]
  (No symbol) [0x00007FF6813BB56A]
  (No symbol) [0x00007FF6814078E1]
  (No symbol) [0x00007FF68143718A]
  (No symbol) [0x00007FF681402E86]
  (No symbol) [0x00007FF6814373A0]
  (No symbol) [0x00007FF68145851C]
  (No symbol) [0x00007FF681436F33]
  (No symbol) [0x00007FF68140116F]
  (No symbol) [0x00007FF6814022D1]
  GetHandleVerifier [0x00007FF6818BC96D+3378253]
  GetHandleVerifier [0x00007FF681908497+3688311]
  GetHandleVerifier [0x00007FF6818FD1CB+3642539]
  GetHandleVerifier [0x00007FF68164A6B6+813462]
  (No symbol) [0x00007FF68150AB5F]
  (No symbol) [0x00007FF681506B74]

```

After running the code, we can see that one test case is failed, because image path needs to be defined from own local workspace which can be different in each laptop.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> robot -d files WIS_Setting.robot
=====
WIS Setting
=====
Go to Website :: This test case verify user is able to open the URL
DevTools listening on ws://127.0.0.1:56605/devtools/browser/c1cd79ae-d643-4d46-8f74-64eb0b305e65
Go to Website :: This test case verify user is able to open the URL | PASS |
-----
Login to your account | PASS |
-----
Click Admin User Dropdown :: Click on the Admin User dropdown to t... | PASS |
-----
Click Profile Icon :: Verify that an admin can open the Profile page | PASS |
-----
Click Edit Admin User :: Verify that an admin can open the Profile... | PASS |
-----
Click Upload Image :: Verify that an admin can upload an image. | PASS |
-----
Click Update Admin User Information :: Verify that an admin can up... | PASS |
-----
Click Change password :: Verify that an admin can change the password | PASS |
-----
Click Update Password :: Verify that an admin can change password. | PASS |
-----
WIS Setting | PASS |
9 tests, 9 passed, 0 failed
=====
Output: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\output.xml
Log: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\log.html
Report: F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot\files\report.html
PS F:\Install\xampp\htdocs\synthia\warehouse-inventory-system-master\Robot> 
```

After setting the right path, all test cases are passed now.

Results and Report:

LOG

WIS Setting Report

Generated
20241020 14:41:44 UTC+09:30
1 minute 42 seconds ago

Summary Information

Status:

All tests passed

Start Time:

20241020 14:41:36.209

End Time:

20241020 14:41:44.729

Elapsed Time:

00:00:08.520

Log File:

[log.html](#)

Test Statistics

Total Statistics

Total

9

Pass

9

Fail

0

Skip

0

Elapsed

00:00:08

Pass / Fail / Skip

Statistics by Tag

Total

9

Pass

9

Fail

0

Skip

0

Elapsed

00:00:08

Pass / Fail / Skip

No Tags

Statistics by Suite

Total

9

Pass

9

Fail

0

Skip

0

Elapsed

00:00:09

Pass / Fail / Skip

WIS Setting

Test Details

All Tags Suites Search

9 tests total, 9 passed, 0 failed, 0 skipped

Total Time: 00:00:08.358

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS Setting, Click Change password	Verify that an admin can change the password		PASS		00:00:01.079	20241020 14:41:42.452 20241020 14:41:43.531
WIS Setting, Click Admin User Dropdown	Click on the Admin User dropdown to test profile.		PASS		00:00:00.043	20241020 14:41:38.764 20241020 14:41:38.807
WIS Setting, Click Edit Admin User	Verify that an admin can open the Profile page		PASS		00:00:01.076	20241020 14:41:39.807 20241020 14:41:40.96
WIS Setting, Click Profile Icon	Verify that an admin can open the Profile page		PASS		00:00:01.083	20241020 14:41:38.807 20241020 14:41:39.890
WIS Setting, Click Update Admin User Information	Verify that an admin can update user details.		PASS		00:00:01.249	20241020 14:41:41.203 20241020 14:41:42.452
WIS Setting, Click Update Password	Verify that an admin can change password.		PASS		00:00:01.196	20241020 14:41:43.533 20241020 14:41:44.729
WIS Setting, Click Upload Image	Verify that an admin can upload an image.		PASS		00:00:00.234	20241020 14:41:40.968 20241020 14:41:41.202
WIS Setting, Go to Website	This test case verify user is able to open the URL.		PASS		00:00:01.790	20241020 14:41:36.364 20241020 14:41:38.154
WIS Setting, Login to your account			PASS		00:00:00.608	20241020 14:41:38.155 20241020 14:41:38.763

Ethical consideration:

The project was developed following ACS (Australian Computer Society) and IEEE/ethical ACM standards. There are similarities between the ACM and ACS Codes of Ethics; however, the ACM places more emphasis on cultural and societal contributions and the well-being of those concerned than the ACS. Taking all of this into account, our group's primary ethical concerns are:

- Integrity:** When developing User Requirements, it is essential to be upfront with customers and users about any limitations the product may have by design. Users should be able to rely on the product to

deliver on their job's essential duties and be alerted to any delays or discrepancies in data. Furthermore, product quality is the foremost factor that brings success, which decides a high portion of user satisfaction.

2. **The Primacy of the Public Interest:** Privacy regulations are most often linked to the development of databases and computer systems due to the importance of the public interest. Encryption should be used for data transmission and storage. Besides, a tester with a high opportunity to handle most data should always be aware of personal privacy regulations.
3. **Competence:** The software's reliability and quality must meet or exceed the standard in the industry if customers are to integrate it confidently into their operations. The Software Development Team, including the Quality Control team, is incentivized to cut expenses wherever possible, minimizing the time spent on testing, routine updates, and routine maintenance. Sometimes unethical software companies rush to market with inferior goods because of time limitations. Because of that, as a software tester, it is crucial to report all known issues found during the testing phase.
4. **Professionalism:** The atmosphere and culture of the office should be conducive to employees' health and happiness. Headaches and eye fatigue pose severe health and safety risks and must be addressed expertly. Everyone on the team should be given the same level of support and encouraged to give 100% to achieve the most significant potential outcomes.
5. **Honesty:** the QC engineer must always be honest when reporting bugs that could endanger the lives of users of the system. This will assist the development team in resolving or mitigating the issue that may influence the organization's credit.

Further work:

There are upcoming enhancements that will allow JMeter to entirely execute performance tests, as the high-security level of the Django Framework prevented the auto bot from interacting with the system. In the following project, our team will investigate further performance testing techniques to ensure that our system is not vulnerable to hacking or assault.

Lesson learned:

The testing team has faced numerous challenges when executing automated tests on the Robot Framework. Every project, website, and page's HTML structure will have a unique design and level of complexity. Consequently, reusing Test-Script only partially alleviates the time constraints of generating new test scripts. In the future, it will be necessary to more precisely estimate the time required to construct automated test scripts for the web platform utilizing Robot Framework. Additionally, while writing the script, there were some elements like buttons as an anchor tag inside each row in a specific column inside the table, which we were not able to fetch properly at first. However, after doing multiple trials and errors, we could do the testing successfully. In addition, to prevent impeding the project's progress, the testing team needs workshops in which team members can exchange their experiences and enhance their skills.

Conclusion:

In conclusion, in agile-based software development, it will be crucial to ensure product quality throughout all phases and phases. Consequently, the automation testing team will be essential in efficiently assessing product quality and minimizing expenses. This necessitates the usage of proxies; hence, software testing solutions such as Robot Framework are required. Even though there are still restrictions in the breadth of automated testing that prevent it from covering all features, software test scripts have helped solve a sizeable portion of the labour testers spend hours performing. In addition, the installation of numerous other add-on tools such as ZAP (Zed Attack Proxy) and JMeter has helped to evaluate the software's level of perfection and security, enabling the development team to improve security, anti-hacking, data safety, and user confidence.

Writing automated testing code using Robot Framework:

C: > Users > Musra > Downloads > Semester 2,2024 > Software Process > Project > Automation > @ WIS_logout.robot > {} Settings

```
Run Suite | Debug Suite | Load in Interactive Console
1  *** Settings ***
2  Documentation  Logout
3  Resource       WIS_login_resource.robot
4  Suite Setup    Go to Website
5  Suite Teardown Close Browser
6
7  Load in Interactive Console
8  *** Variables ***
9  ${VALID_USER}      admin
10 ${VALID_PASSWORD}  admin
11 ${INVALID_USER}    invalid_user
12 ${INVALID_PASSWORD} wrong_pw
13 ${EMPTY}           ${EMPTY}
14
15 *** Test Cases ***
16
17 Run | Debug | Run in Interactive Console
18 Logout
19   [Documentation]  Logout from the system
20   Login With Valid Credentials Should Succeed ${VALID_USER} ${VALID_PASSWORD}
21   Logout As Admin User
22
23 *** Keywords ***
24
25 Load in Interactive Console
26 Login With Valid Credentials Should Succeed
27   [Arguments] ${username} ${password}
28   Input username ${username} # Updated locator for username
29   Input passwords ${password} # Updated locator for password
30   Submit Credentials
31   Page Should Contain Welcome
32
33 Load in Interactive Console
34 Logout As Admin User
35   [Documentation]  Perform logout by clicking on the Admin User dropdown and then Logout.
36   Click Admin User Dropdown
37   Click Logout Link
38
39 Load in Interactive Console
40 Click Admin User Dropdown
41   [Documentation]  Click on the Admin User dropdown to reveal the logout option.
42   Click Element //a[@class='toggle' and contains(span, 'Admin User')]
43
44 Load in Interactive Console
45 Click Logout Link
46   [Documentation]  Click on the Logout link in the dropdown menu.
47   Click Element //li[@class='last']/a
48
49
50
51
```

Executing robot file:

```
Report: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\report.html
PS C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot> robot -d results WIS_logout.robot
[ ERROR ] Error in file 'C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\WIS_logout.robot' on line 12: Setting variable '${EMPTY}' failed: Recursive variable definition.
=====
WIS_logout :: Logout
=====
[ WARN ] The chromedriver version (128.0.6613.119) detected in PATH at C:\Python310\Scripts\chromedriver-win64\chromedriver-win64\chromedriver.exe might not be compatible with the detected chrome version (129.0.6668.90); currently, chromedriver 129.0.6668.89 is recommended for chrome 129.*, so it is advised to delete the driver in PATH and retry

DevTools listening on ws://127.0.0.1:59835/devtools/browser/cdbaad52-787d-465e-a8ff-77dedf1997a6
Logout :: Logout from the system | PASS |
-----
WIS_logout :: Logout | PASS |
1 test, 1 passed, 0 failed
=====
Output: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\output.xml
Log: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\log.html
Report: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\results\report.html
PS C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot>
```

3. Report for WIS Medias functionality in Robot framework

WIS Meadia Report

Generated
20241007 00:01:41 UTC+09:30
4 minutes 8 seconds ago

Summary Information

Status:	All tests passed
Start Time:	20241007 00:01:37.224
End Time:	20241007 00:01:41.183
Elapsed Time:	00:00:03.959
Log File:	log.html

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	5	5	0	0	00:00:04	
Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						
Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
WIS Meadia	5	5	0	0	00:00:04	

Test Details

AllTagsSuitesSearch

Suite:

Test:

Include:

Exclude:

Search

Clear

Help

Warehouse Phipmyadmin Login Registration

Test Details

LOG

Suite: WIS Meadia

Status: 5 tests total, 5 passed, 0 failed, 0 skipped

Start / End Time: 20241007 00:01:37.224 / 20241007 00:01:41.183

Elapsed Time: 00:00:03.959

Log File: log.html#s1

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS Meadia Go to Website	This test case verify user is able to open the URL		PASS		00:00:02.499	20241007 00:01:37.541 20241007 00:01:40.040
WIS Meadia Login to your account			PASS		00:00:00.602	20241007 00:01:40.040 20241007 00:01:40.642
WIS Meadia Go to Meadias	Verify that an admin can go to Meadias		PASS		00:00:00.134	20241007 00:01:40.642 20241007 00:01:40.776
WIS Meadia Upload Image	Verify that an admin can upload an image.		PASS		00:00:00.282	20241007 00:01:40.775 20241007 00:01:41.057
WIS Meadia Delete Uploaded Image	Click on the delete button for the uploaded image specified by its name.		PASS		00:00:00.126	20241007 00:01:41.058 20241007 00:01:41.184

Test Execution Log

SUITE WIS Meadia Full Name: WIS Meadia Source: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\WIS_Mediaia.robot Start / End / Elapsed: 20241007 00:01:37.224 / 20241007 00:01:41.183 / 00:00:03.959 Status: 5 tests total, 5 passed, 0 failed, 0 skipped	00:00:03.959
TEST Go to Website	00:00:02.499
TEST Login to your account	00:00:00.602
TEST Go to Meadias Full Name: WIS Meadia Go to Meadias Documentation: Verify that an admin can go to Meadias Start / End / Elapsed: 20241007 00:01:40.642 / 20241007 00:01:40.776 / 00:00:00.134 Status: PASS	00:00:00.134
KEYWORD SeleniumLibrary Click Element xpath=//div[@class="sidebar"]/ul/li[5]/a[1]	00:00:00.127
KEYWORD SeleniumLibrary Page Should Contain All Photos	00:00:00.007
TEST Upload Image Full Name: WIS Meadia Upload Image Documentation: Verify that an admin can upload an image. Start / End / Elapsed: 20241007 00:01:40.775 / 20241007 00:01:41.057 / 00:00:00.282 Status: PASS	00:00:00.282
KEYWORD SeleniumLibrary Wait Until Element Is Visible xpath=//input[@type="file" and @name="file_upload"]	00:00:00.019
KEYWORD SeleniumLibrary Choose File xpath=//input[@type="file" and @name="file_upload"] C:\Users\Musra\OneDrive\Pictures\Book.png	00:00:00.022
KEYWORD SeleniumLibrary Click Button xpath=//button[@type="submit" and @name="submit"]	00:00:00.123
KEYWORD SeleniumLibrary Capture Page Screenshot C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\Upload_Image_Screenshot.png	00:00:00.111
TEST Delete Uploaded Image Full Name: WIS Meadia Delete Uploaded Image Documentation: Click on the delete button for the uploaded image specified by its name. Start / End / Elapsed: 20241007 00:01:41.058 / 20241007 00:01:41.184 / 00:00:00.126 Status: PASS	00:00:00.126
KEYWORD \$[image_xpath] = Outem.Set Variable //tr[td[contains(text()), '\$[image_name)']]/a[contains(@href, 'delete_media.php)']	00:00:00.000
KEYWORD SeleniumLibrary Click Element xpath=\$[image_xpath]	00:00:00.114
KEYWORD SeleniumLibrary Page Should Contain Photo has been deleted	00:00:00.012

4. Report for WIS Logout functionality in Robot framework

Test Details

LOG

Suite: WIS logout

Status: 1 test total, 1 passed, 0 failed, 0 skipped

Documentation: Logout

Start / End Time: 20241006 00:17:44.248 / 20241006 00:17:49.557

Elapsed Time: 00:00:05.309

Log File: log.html#s1

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS logout Logout	Logout from the system		PASS		00:00:00.666	20241006 00:17:46.780 20241006 00:17:47.446

LOG

WIS logout Report

Generated
20241006 00:17:49 UTC+09:30
34 seconds ago

Summary Information

Status:All tests passed

Documentation:Logout

Start Time:20241006 00:17:44.248

End Time:20241006 00:17:49.557

Elapsed Time:00:00:05.309

Log File:log.html

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	1	1	0	0	00:00:01	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
WIS logout	1	1	0	0	00:00:05	

Test Details

AllTagsSuitesSearch

Suite:WIS logout

Status:1 test total, 1 passed, 0 failed, 0 skipped

Documentation:Logout

Start / End Time:20241006 00:17:44.248 / 20241006 00:17:49.557

Elapsed Time:00:00:05.309

Log File:log.html#s1

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS logout: Logout	Logout from the system		PASS		00:00:00.666	20241006 00:17:46.780 20241006 00:17:47.446

←→↺

FileC:/Users/Musra/New%20folder/htdocs/warehouse-inventory-system-master/Robot/results/report.html

Warehouse

Phpmadmin

Login Registration

WIS Meadia Report

Generated
20241007 00:01:41 UTC+09:30
4 minutes 8 seconds ago

Summary Information

Status:All tests passed

Start Time:20241007 00:01:37.224

End Time:20241007 00:01:41.183

Elapsed Time:00:00:03.959

Log File:log.html

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	5	5	0	0	00:00:04	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
WIS Meadia	5	5	0	0	00:00:04	

Test Details

AllTagsSuitesSearch

Suite:

Test:

Include:

Exclude:

SearchClearHelp

FileC:/Users/Musra/New%20folder/htdocs/warehouse-inventory-system-master/Robot/results/report.html#suites/s1

Warehouse

Phpmadmin

Login Registration

All Bookmarks

Test Details

AllTagsSuitesSearch

Suite:WIS Meadia

Status:5 tests total, 5 passed, 0 failed, 0 skipped

Start / End Time:20241007 00:01:37.224 / 20241007 00:01:41.183

Elapsed Time:00:00:03.959

Log File:log.html#s1

Name	Documentation	Tags	Status	Message	Elapsed	Start / End
WIS Meadia: Go to Website	This test case verify user is able to open the URL		PASS		00:00:02.499	20241007 00:01:37.541 20241007 00:01:40.040
WIS Meadia: Login to your account			PASS		00:00:00.602	20241007 00:01:40.040 20241007 00:01:40.642
WIS Meadia: Go to Meadias	Verify that an admin can go to Meadias		PASS		00:00:00.134	20241007 00:01:40.642 20241007 00:01:40.776
WIS Meadia: Upload Image	Verify that an admin can upload an image.		PASS		00:00:00.282	20241007 00:01:40.776 20241007 00:01:41.057
WIS Meadia: Delete Uploaded Image	Click on the delete button for the uploaded image specified by its name.		PASS		00:00:00.126	20241007 00:01:41.058 20241007 00:01:41.184

Products: (These Test cases will not work until category and media photo will be added in previous section. These need to be improved by adding category and media test cases also so that it can work)

Logs

←→↺🏠🔍D:\CDU\Semester_02\PRT582%20SOFTWARE%20ENGINEERING%20PROCESS%20AND%20TOOLS\Group\Robot\lo...☆🔗⚙️🕒🔒📄🗑️📥🔴⋮

Generated20241008 21:42:59 UTC+09:3011 minutes 08 seconds ago

REPORT

WIS Setting Log

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	9	9	0	0	00:00:13	

Statistics by Tag

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags					

Statistics by Suite

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
WIS Setting	9	9	0	0	00:00:13

Test Execution Log

SUITEWIS Setting00:00:13.008

Full Name:WIS Setting

Source:D:\CDU\Semester_02\PRT582 SOFTWARE ENGINEERING PROCESS AND TOOLS\Group\Robot\WIS_Setting.robot

Start / End / Elapsed:20241008 21:42:46.935 / 20241008 21:42:59.943 / 00:00:13.008

Status:9 tests total, 9 passed, 0 failed, 0 skipped

TESTGo to Website	00:00:03.738
TESTLogin to your account	00:00:01.133
TESTClick Admin User Dropdown	00:00:00.137
TESTClick Profile Icon	00:00:01.199
TESTClick Edit Admin User	00:00:01.227
TESTClick Upload Image	00:00:00.537
TESTClick Update Admin User Information	00:00:01.079
TESTClick Change password	00:00:01.257
TESTClick Update Password	00:00:01.059

Report

←→↺🏠🔍D:\CDU\Semester_02\PRT582%20SOFTWARE%20ENGINEERING%20PROCESS%20AND%20TOOLS\Group\Robot\re...☆🔗⚙️🕒🔒📄🗑️📥🔴⋮

Generated20241008 21:42:59 UTC+09:3012 minutes 57 seconds ago

LOG

WIS Setting Report

Summary Information

Status:All tests passed

Start Time:20241008 21:42:46.935

End Time:20241008 21:42:59.943

Elapsed Time:00:00:13.008

Log File:log.html

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	9	9	0	0	00:00:13	

Statistics by Tag

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags					

Statistics by Suite

Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
WIS Setting	9	9	0	0	00:00:13

Test Details

AllTagsSuitesSearch

Suite:

Test:

Include:

Exclude:

SearchClearHelp

Test Execution Log

- SUITE WIS Meadia		00:00:03.959
Full Name: WIS Meadia		
Source: C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\WIS_Media.robot		
Start / End / Elapsed: 20241007 00:01:37.224 / 20241007 00:01:41.183 / 00:00:03.959		
Status: 5 tests total, 5 passed, 0 failed, 0 skipped		
+ TEST Go to Website		00:00:02.499
+ TEST Login to your account		00:00:00.602
- TEST Go to Meadias		00:00:00.134
Full Name: WIS Meadia Go to Meadias		
Documentation: Verify that an admin can go to Meadias		
Start / End / Elapsed: 20241007 00:01:40.642 / 20241007 00:01:40.776 / 00:00:00.134		
Status: PASS		
+ KEYWORD SeleniumLibrary Click Element	xpath=//div[@class="sidebar"]//ul//li[5]//a[1]	00:00:00.127
+ KEYWORD SeleniumLibrary Page Should Contain	All Photos	00:00:00.007
- TEST Upload Image		00:00:00.282
Full Name: WIS Meadia Upload image		
Documentation: Verify that an admin can upload an image.		
Start / End / Elapsed: 20241007 00:01:40.775 / 20241007 00:01:41.057 / 00:00:00.282		
Status: PASS		
+ KEYWORD SeleniumLibrary Wait Until Element is Visible	xpath=//input[@type="file" and @name="file_upload"]	00:00:00.019
+ KEYWORD SeleniumLibrary Choose File	xpath=//input[@type="file" and @name="file_upload"] C:\Users\Musra\OneDrive\Pictures\Book.png	00:00:00.022
+ KEYWORD SeleniumLibrary Click Button	xpath=//button[@type="submit" and @name="submit"]	00:00:00.123
+ KEYWORD SeleniumLibrary Capture Page Screenshot	C:\Users\Musra\New folder\htdocs\warehouse-inventory-system-master\Robot\Upload_Image_Screenshot.png	00:00:00.111
- TEST Delete Uploaded Image		00:00:00.126
Full Name: WIS Meadia Delete Uploaded Image		
Documentation: Click on the delete button for the uploaded image specified by its name.		
Start / End / Elapsed: 20241007 00:01:41.058 / 20241007 00:01:41.184 / 00:00:00.126		
Status: PASS		
+ KEYWORD \${image_xpath} = BuiltIn.Set Variable	//tr[td[contains(text()), \${image_name}]]//a[contains(@href, 'delete_media.php')]	00:00:00.000
+ KEYWORD SeleniumLibrary Click Element	xpath=\${image_xpath}	00:00:00.114
+ KEYWORD SeleniumLibrary Page Should Contain	Photo has been deleted.	00:00:00.012